Joseph Argbaiye

Big Data Analytics

Student ID: 5025912

Installation, Configuration and code workflow Documentation:

This document provides a step-by-step guide to the project implementation. It explains the installation and configuration of each tool used and documents the key processes required to establish the complete system. Also, the codes attributed to the models and streaming pipeline. This code should be treated as for supporting documentation for my assessment and not for public reproducibility without my consent.

Implementation Content Flow:

- ✓ WSL and Ubuntu environment
- ✓ Bitcoin Core installation and synchronization
- ✓ ZMQ and RPC configuration
- ✓ Kafka installation, topic and consumer
- ✓ Packages Installation
- ✓ ML environment, models and live processing
- ✓ SQLite schema and live persistence
- ✓ Grafana installation and data-source configuration
- ✓ Dashboard and investigator view
- ✓ End-to-end transaction trace
- ✓ Service files, requirements and reproducibility information

WSL and Ubuntu Environment:

This guide retains the full step-by-step sequence while grouping it into five practical phases for easier use during implementation and evidence collection.

Purpose of the setup:

The purpose of this setup is to create a Linux environment inside Windows 11 using Windows Subsystem for Linux version 2 (WSL 2). The finished system provides both command-line Linux access and a complete **graphical Linux desktop** that can be reached through **Windows Remote Desktop** if not using **VMware**:

Windows 11 - **WSL 2** - **Ubuntu 24.04 LTS** - **Linux user:** admp - **Hostname**: Joseph - **systemd** - **XFCE Desktop** - **XRDP Server** :3390 - **WSL UI** - **Windows Remote Desktop** -> localhost:3390

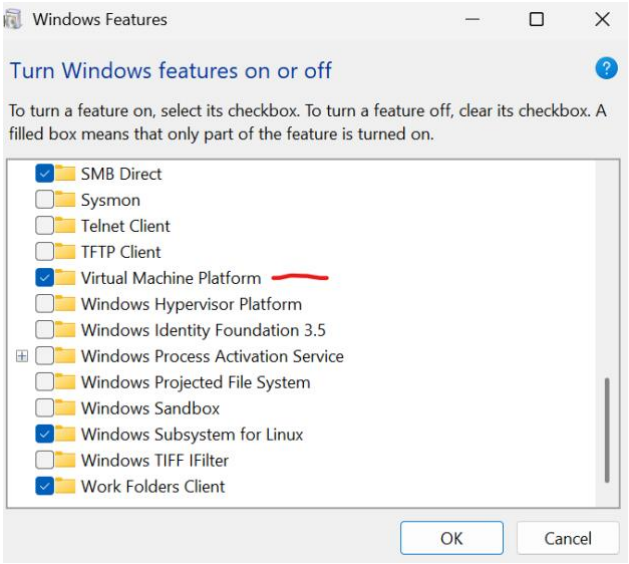
The arrangement above is suitable when Windows remains the host operating system, but Linux-based services and tools need to be installed, configured and managed in a realistic Linux environment. WSL 2 uses a real Linux kernel in a lightweight virtualised environment. Unlike a conventional virtual machine, it does not require the user to manually create a separate VM, virtual disk or hypervisor console. This

©Joseph Aragbaiye Project, 2026

procedure is also widely considered for industry standard by so many organizations on a windows driven system.

1. **Install WSL2 on Powershell, WSLUI, and Ubuntu 24.04 from Windows App:**

Before installation, ensure that your virtual machine service is on. To do this, on your window desktop find the search beside the windows start menu. Click on the search and type windows feature plus enter. When the windows folder comes up; enable this feature: **'Virtual Machine Platform'** with **'Windows Subsystem for Linux'.**

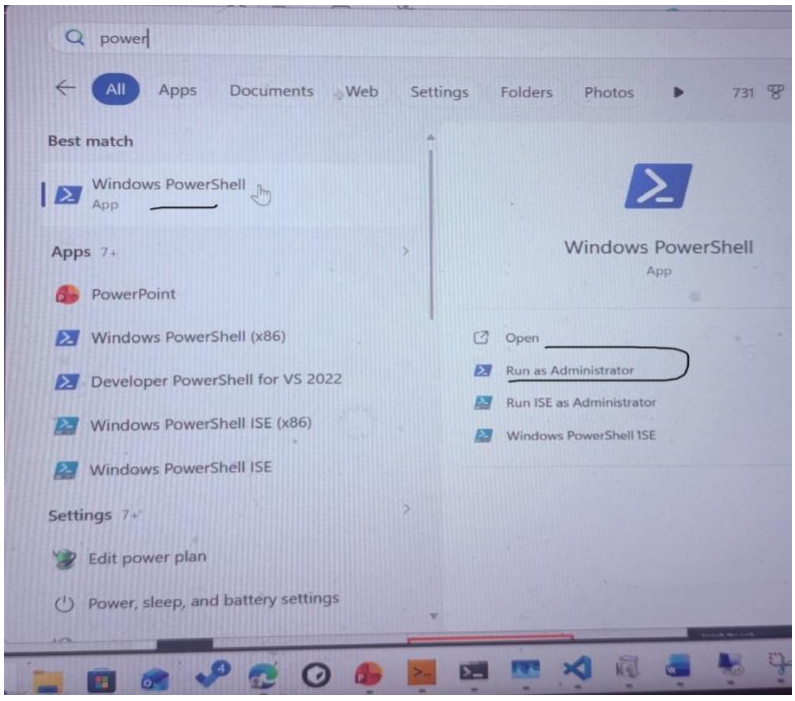


If prompted to restart your system, do so.

After that has been checked. We can now install our **Linux WSL environment**, **Ubuntu 24:04, WSLUI environment.**

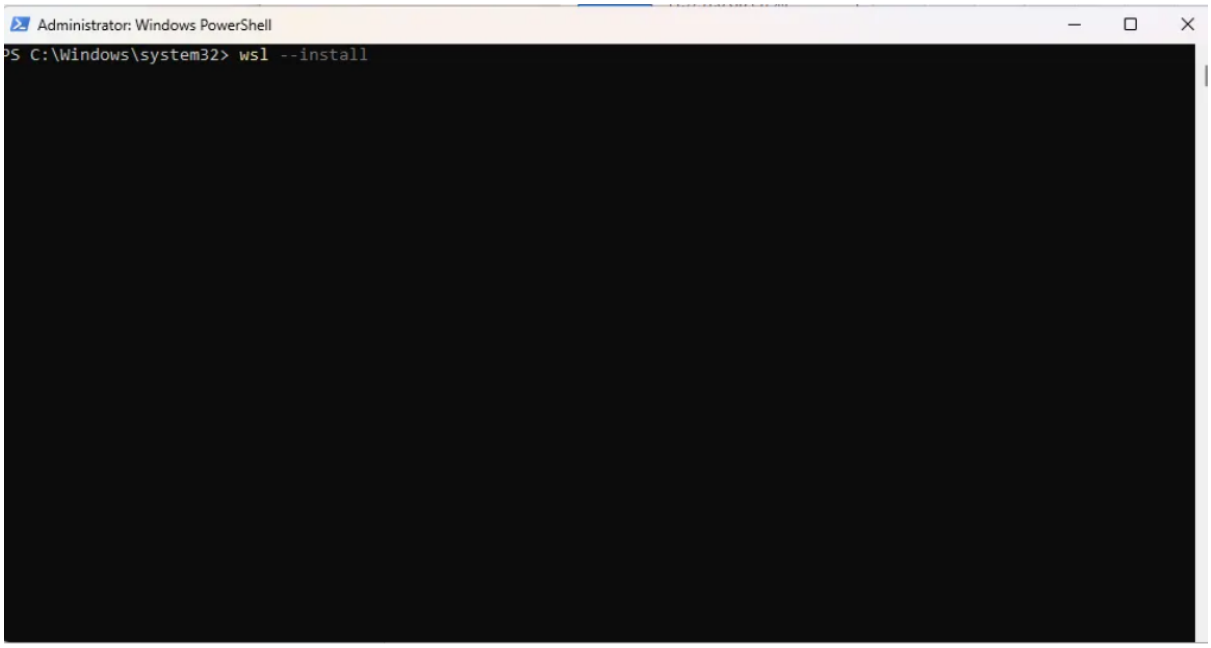
First thing to do before installing the **WSL2** on our PowerShell is to check if WSL has already been installed on your pc.

To verify if WSL has been installed on your computer. Run your **PowerShell as Administrator:**

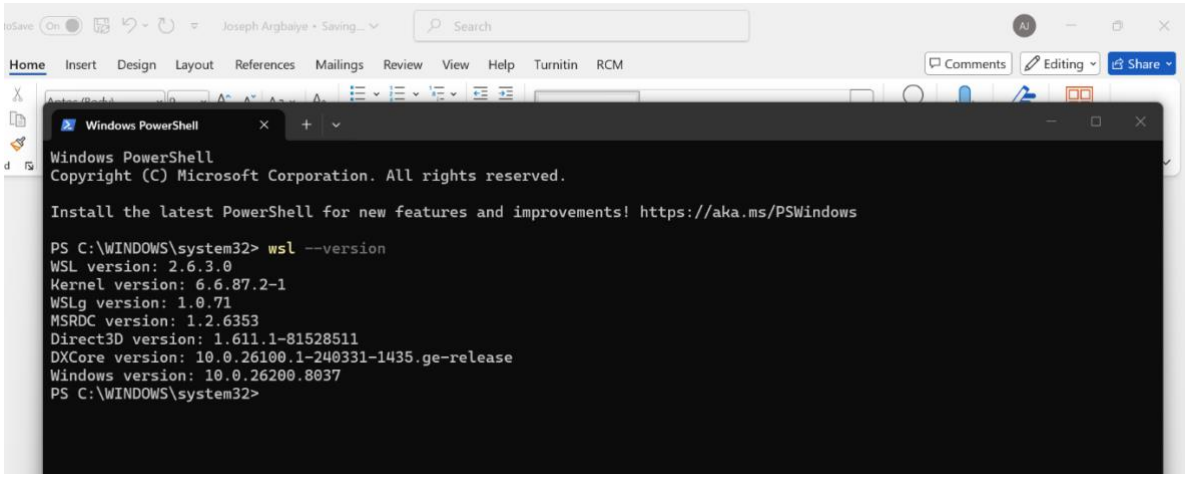


©Joseph Aragbaiye Project, 2026

When you run your **PowerShell** as administrator, you should have this window PowerShell opened:



Now, run the command **WSL –-version** in your PowerShell. For me, I have that already installed on my computer with the **version 2.6.3.0** .

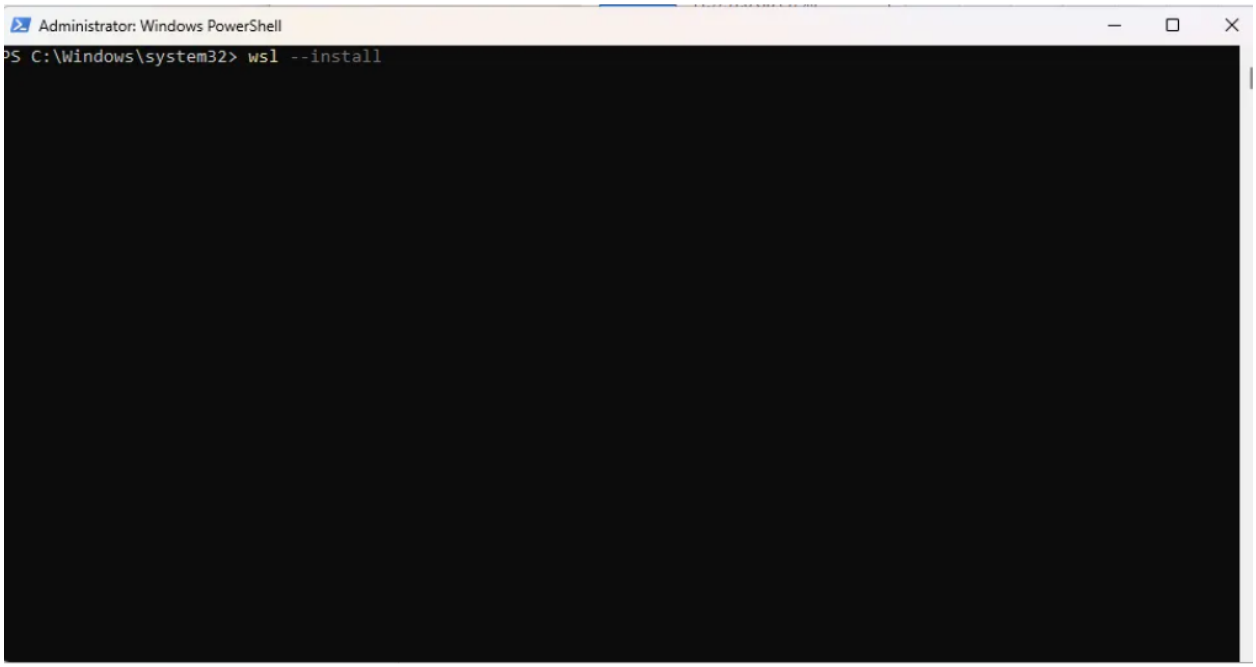


If you have not installed **WSL** on your **PC** before, look out for the phrase **“the windows Subsystem for Linux is not installed**. If that has been confirmed that your windows have no record of WSL being installed, do the following instruction:

Install WSL:

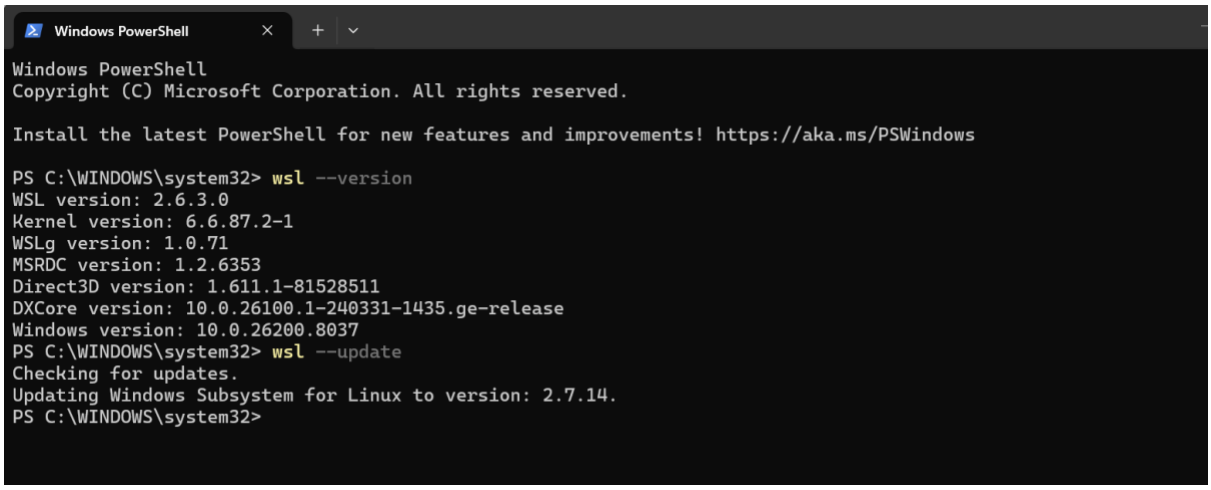
Open your **PowerShell** and type in the command **wsl --install:**

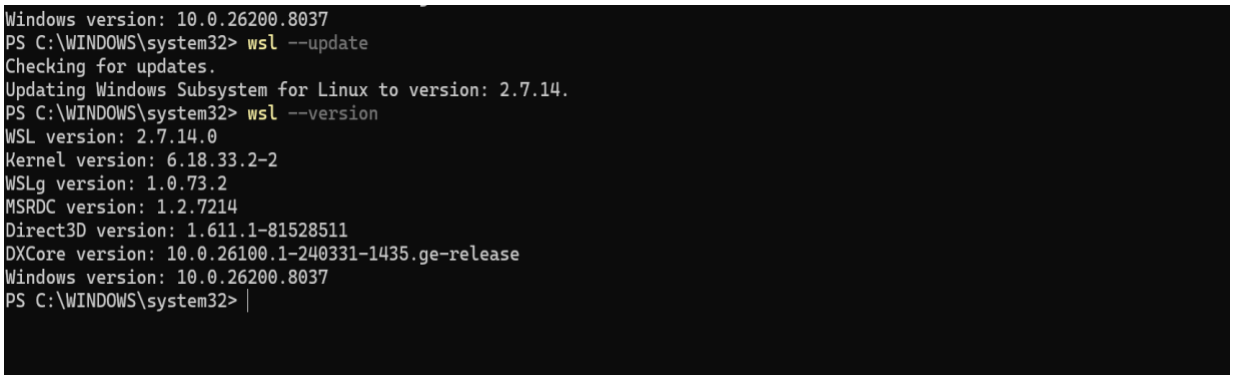
©Joseph Aragbaiye Project, 2026



Then check your version if you have downloaded the latest **WSL**: Just as I did above with the command **wsl –version.** For you, you should have the latest WSL downloaded on your computer. The latest wsl version as of today's date is **version 2.7.14.**

Since I have the previous version, I will upgrade to the latest version **version 2.7.14** with the command **wsl –upgrade**





©Joseph Aragbaiye Project, 2026

To confirm you have installed the latest updated **wsl** ; run the command **wsl –-update** again: and you should have the below output screenshot.

```
PS C:\WINDOWS\system32> wsl --update
Checking for updates.
The most recent version of Windows Subsystem for Linux is already installed.
PS C:\WINDOWS\system32>
```

Now, we must check out for the categorized version of wsl we are using. We have **WSL1** & **WSL2**. It is recommended to use **WSL2** version. To check the categorized status, we should run the command **wsl –status.**

```
PS C:\WINDOWS\system32> wsl --status
Default Distribution: Ubuntu
Default Version: 2
PS C:\WINDOWS\system32>
```

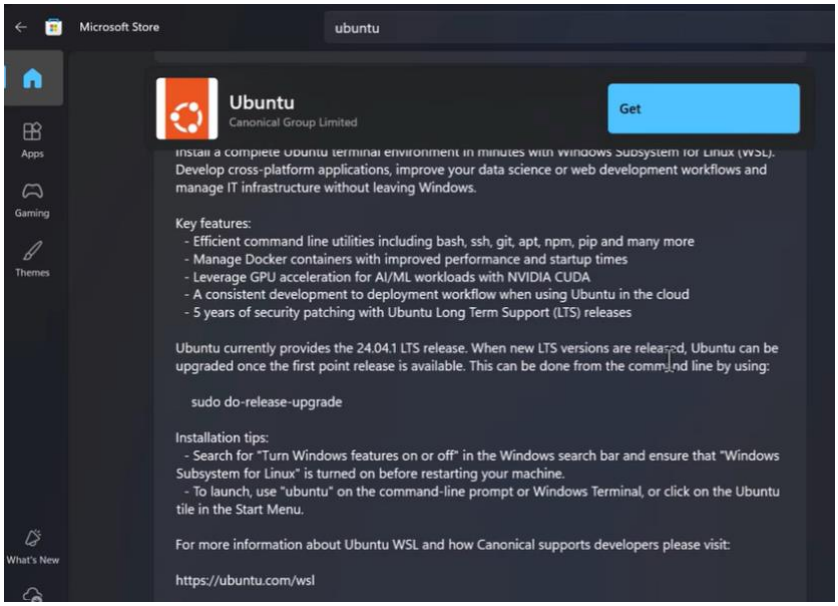
If you see **version: 1** you must change it to **version: 2**

To change it to **version 2**, you are to run this command:

wsl –default-version-2

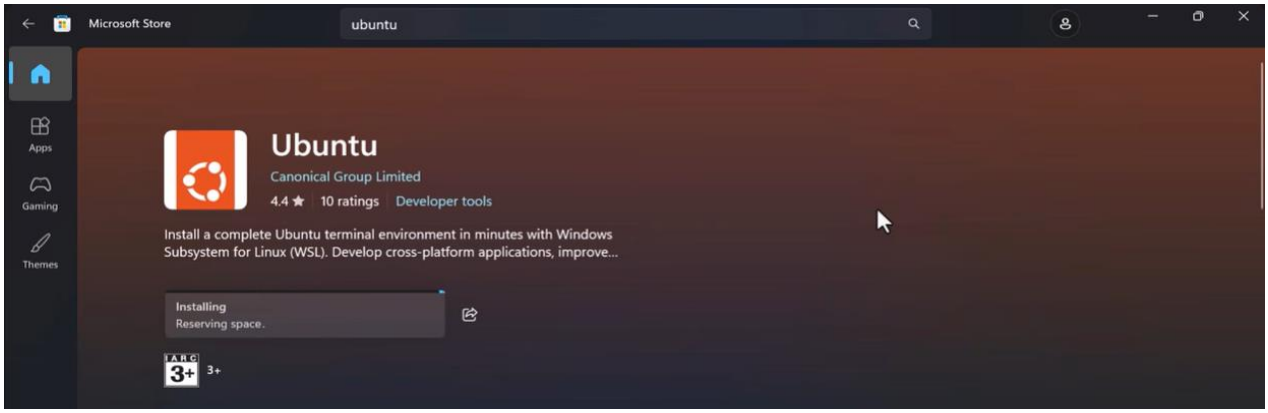
The next step is installing **ubuntu 24.04 OS** on the window desktop.

On your windows, navigate to **Microsoft store App.** On the App store, search for Ubuntu. Now get the ubuntu

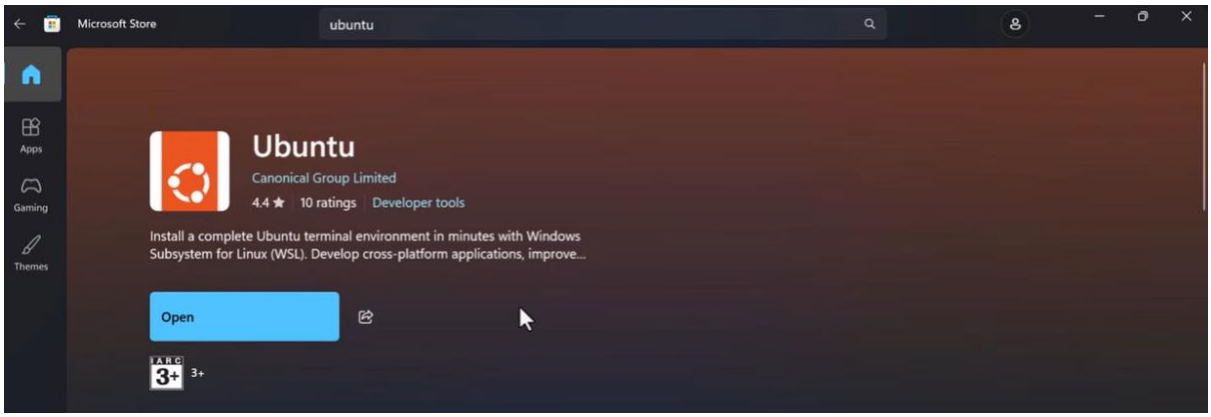


Wait while it installs....

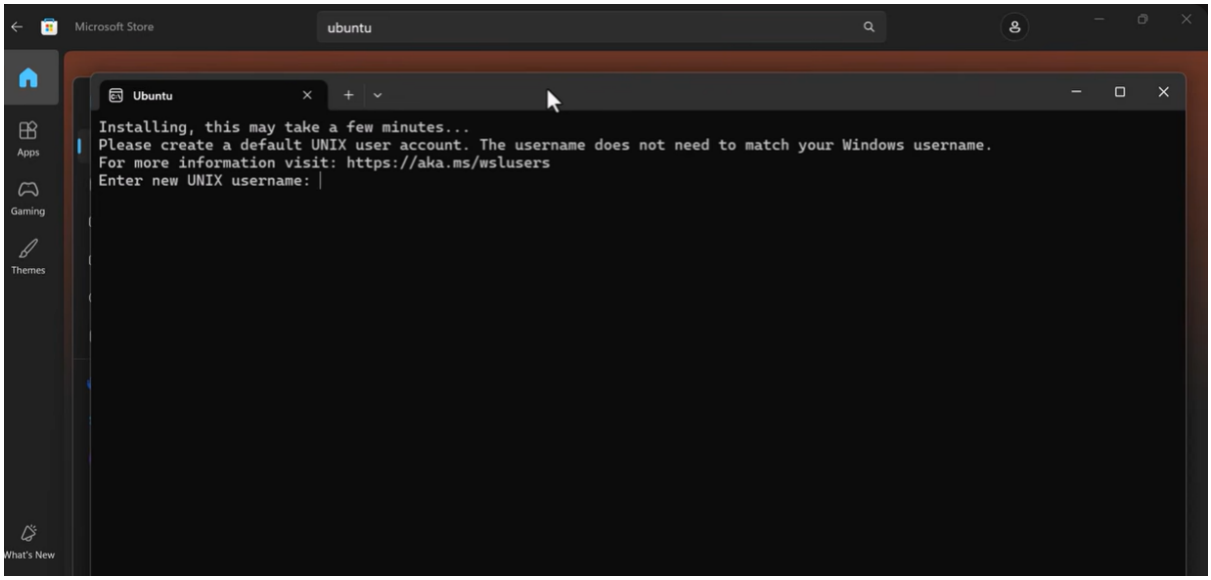
©Joseph Aragbaiye Project, 2026



After the download has completed…. You can open.



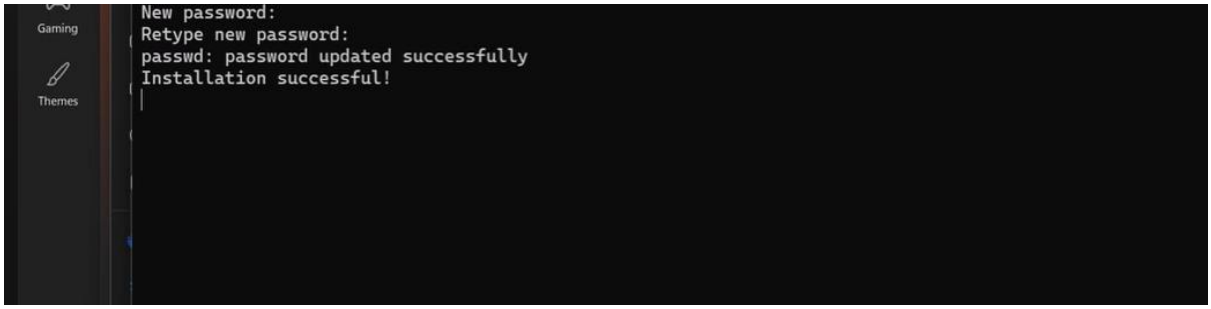
Create a **Username** for your installed Ubuntu



Now create a **password** for your Ubuntu User

Note: Linux terminals do not normally display dots, asterisks or other characters while a password is being typed; this is normal.

©Joseph Aragbaiye Project, 2026



After then, run the command for any update left...

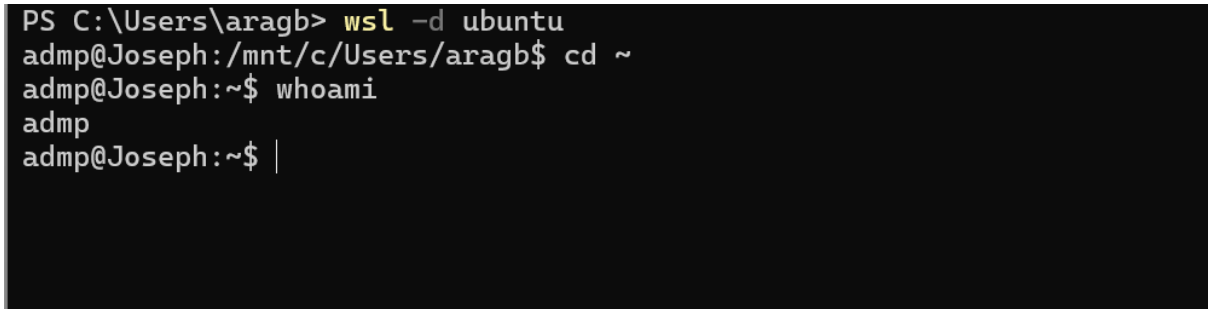
sudo apt update

You should get that following for password verification (**admp** is my chosen **username**, you can decide what username you want from the creation of the username of your account):

[sudo] password for admp:

The account created at this stage is the Linux account used throughout the remainder of the guide.

Inside Ubuntu, verify the current user. That defines you as a user.



As for my own expected result of my username: **admp**

How the hostname Joseph is obtained:

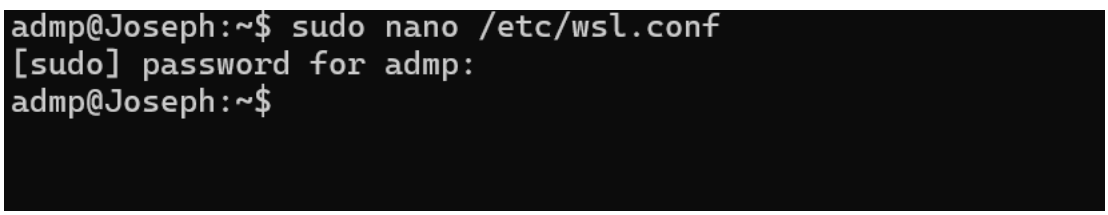
WSL can use the Windows computer name as its Linux hostname. If the Windows PC is named Joseph, Ubuntu may therefore already show **admp@Joseph.** For a reproducible configuration, the hostname can be set explicitly in **/etc/wsl.conf.**

Configure /etc/wsl.conf:

Inside Ubuntu, open the WSL distribution configuration file:

sudo nano /etc/wsl.conf

It will prompt for your password, so enter your password:



©Joseph Aragbaiye Project, 2026

Enter the following configuration in your nano file configuration window but you have to adjust the hostname exactly to the username created for yourself and your local system username:

[boot] systemd=true

[network] hostname=Joseph

[user] default=admp



The boot section enables systemd. The network section sets the hostname to Joseph. The user section makes admp the default user when the distribution starts. In nano, save with **Ctrl + O**, press Enter, and exit with **Ctrl + X.**

Restart WSL after changing wsl.conf

Configuration changes to **/etc/wsl.conf** are applied after the WSL virtual environment is stopped and started again. To exit run the command:

exit

```
admp@Joseph:~$ exit
logout
PS C:\Users\aragb>
```

The above exit from the **ubuntu environment**.

To shut-down the ubuntu:

Run the command in PowerShell: **wsl -- shutdown**

To start the distribution again, run the command: **wsl**

Verify username, hostname and systemd Inside Ubuntu, verify each configured item:

Expected output:

©Joseph Aragbaiye Project, 2026

admp

Joseph

systemd

```
admp@Joseph:~$ whoami
hostname
ps -p 1 -o comm=
admp
Joseph
systemd
admp@Joseph:~$
```

Return to Windows PowerShell and run:

wsl --list –verbose

```
PS C:\WINDOWS\system32> wsl --list --verbose
  NAME            STATE           VERSION
* Ubuntu          Running         2
```

What WSL UI:

WSL UI is not the same product as Microsoft WSL. WSL is the Windows subsystem that runs Linux distributions. WSL UI is a third-party graphical management application that provides a Windows interface for viewing and managing WSL distributions.

Windows 11 -> **Microsoft WSL** -> **Ubuntu** -> **WSL UI manages / monitors the distribution**

WSL UI does not replace WSL; it sits above the WSL environment as a management layer.

Install WSL UI:

Two ways to Install WSL... through **command line PowerShell**

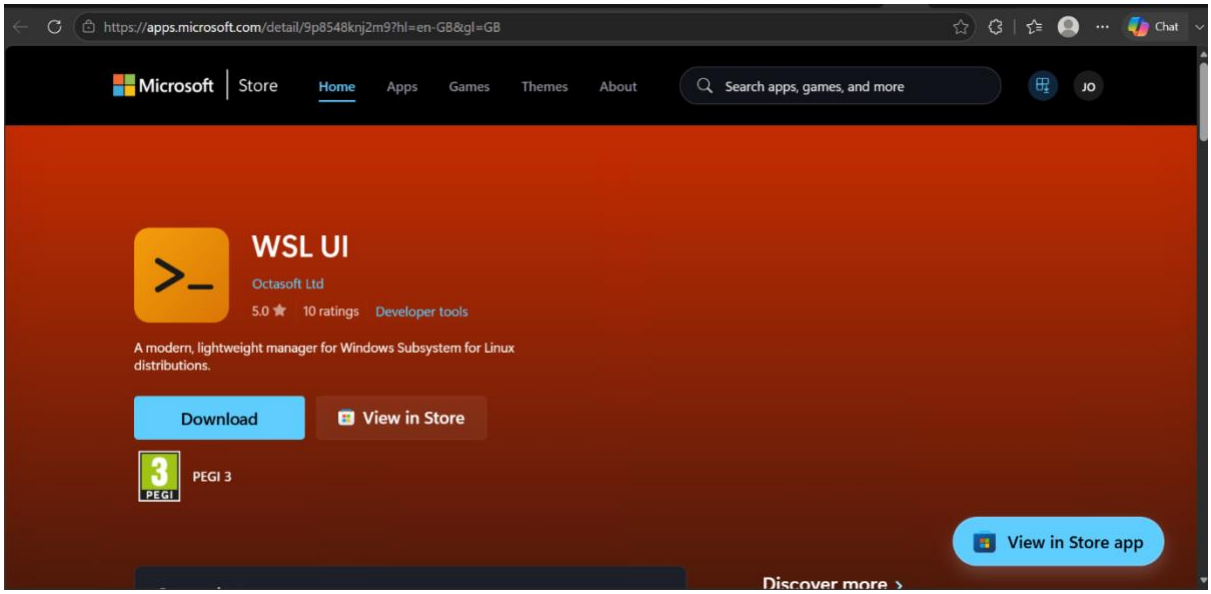
Open **Windows PowerShell** and install **WSL UI** from the **Microsoft Store** source **through Winget:**

winget install "WSL UI" --source msstore

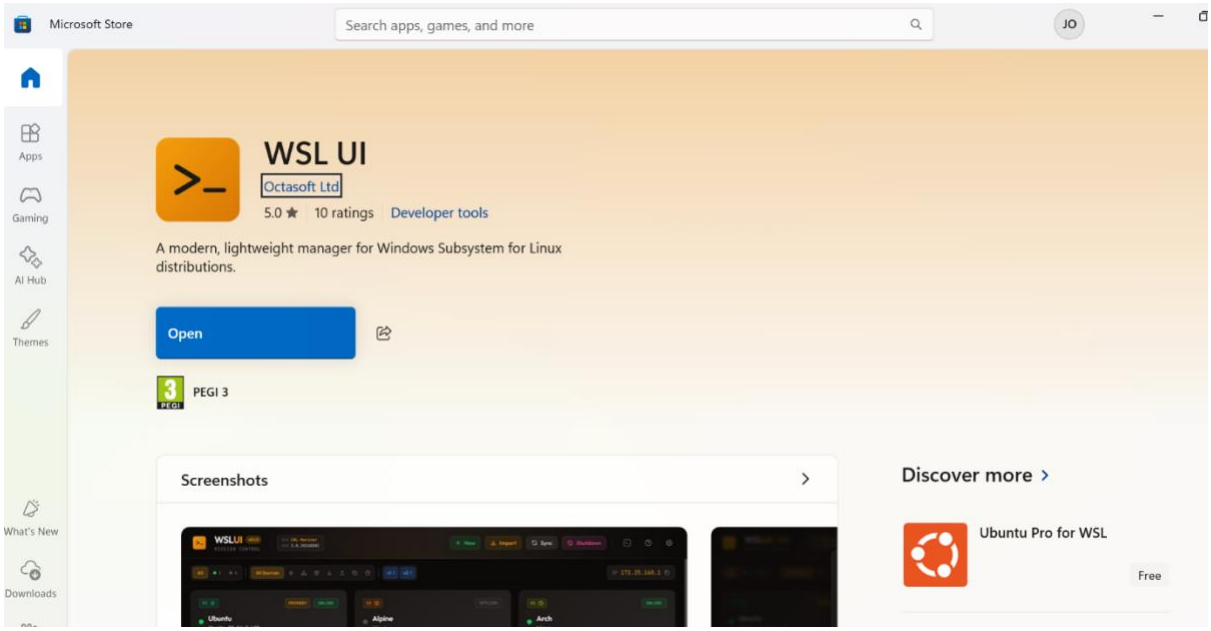
Or

Go to the **Microsoft App** and download **WSL UI:**

©Joseph Aragbaiye Project, 2026

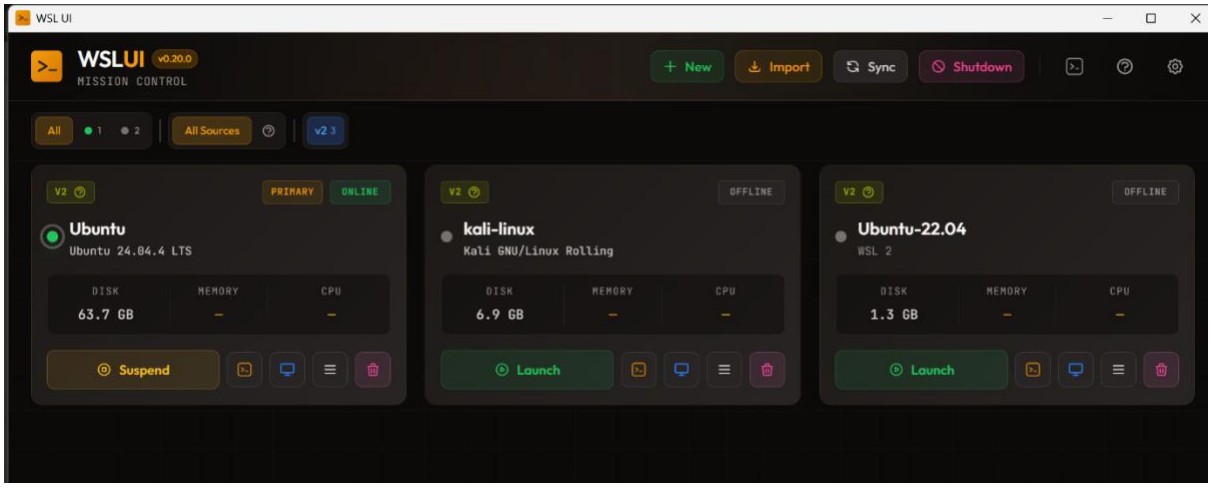


Click view in store and from there you can download on your PC:



After installation, open Start -> WSL UI. An MSI installer or portable build can also be used if required.

©Joseph Aragbaiye Project, 2026



You should have a view of the Linux distributions you have installed just like mine are above.

Difference between a Linux terminal and a Linux desktop :

A standard Ubuntu WSL installation provides a Linux command-line environment. The presence of a shell prompt such as admp@Joseph:~$ or whatever username and hostname it is you have considered does not mean a complete Linux desktop environment has been installed.

Normally available after WSL Ubuntu install:

Linux kernel and filesystem

Bash shell and Linux utilities

apt package manager

Linux services

Requires additional desktop configuration:

Complete desktop environment

Desktop panels, file manager and window manager

XRDP desktop session

Full Remote Desktop experience

WSLg versus **a full desktop**

Full Remote Desktop Experience Modern WSL includes WSLg for launching individual Linux graphical applications on the Windows desktop. XRDP with XFCE has a different purpose: it provides a complete Linux desktop session reached through the Remote Desktop Protocol.

Technology:

Primary use**: WSLg** Run individual **Linux GUI** applications alongside Windows applications

XFCE + XRDP in WSlg. Install **XFCE** Provide a complete separate **Linux desktop session**

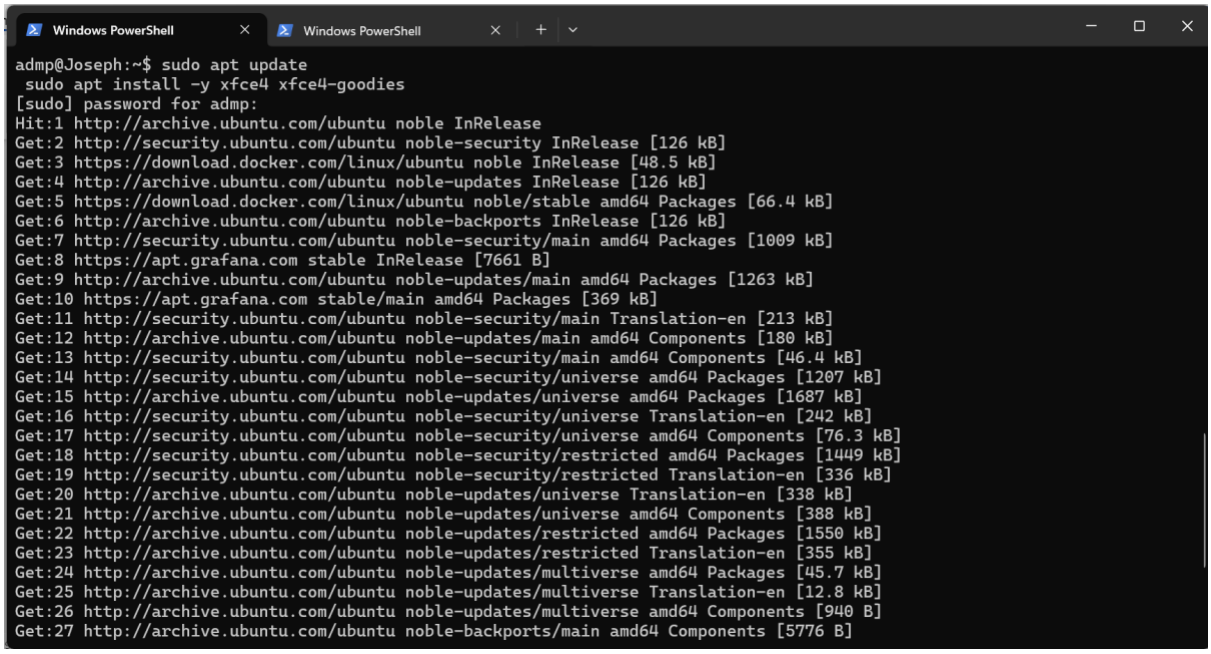
©Joseph Aragbaiye Project, 2026

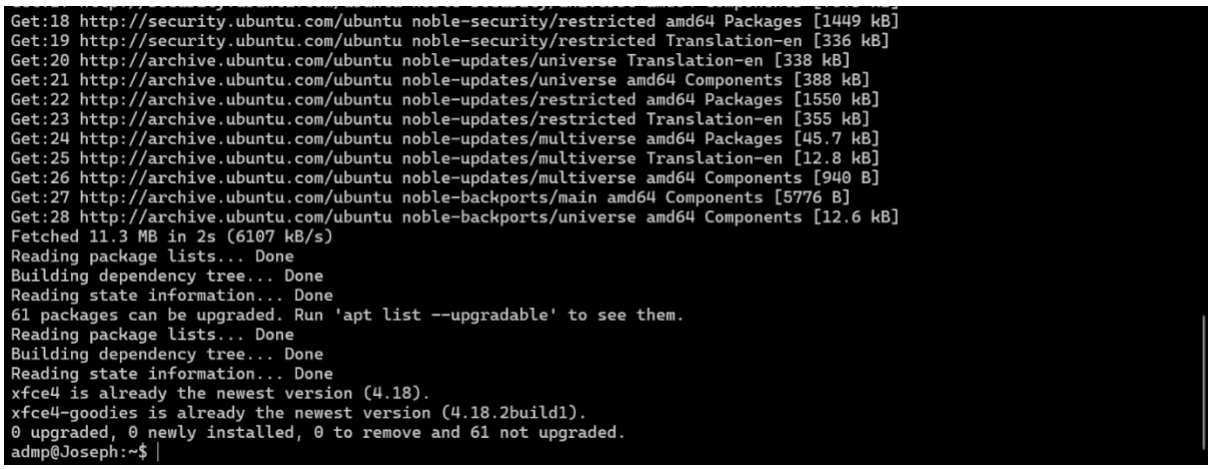
Install XFCE

Provide a complete separate Linux desktop session Inside Ubuntu, update package metadata and install the XFCE desktop environment and supporting packages:

sudo apt update

sudo apt install -y xfce4 xfce4-goodies



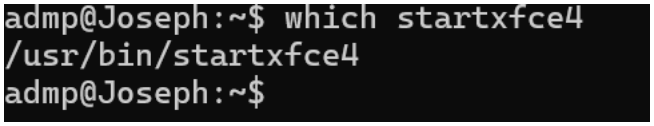


XFCE is lightweight and therefore well suited to WSL.

Verify its startup command:

which startxfce4

Expected to see: /usr/bin/startxfce4



Check installed XFCE packages if required:

©Joseph Aragbaiye Project, 2026

dpkg -l | grep xfce4 | head -20

```
admp@Joseph:~$ dpkg -l | grep xfce4 | head -20
ii  libxfce4panel-2.0-4                    4.18.4-1ubuntu0.1          amd64        Xfce4 panel
library (GTK3 variant)
ii  libxfce4ui-2-0:amd64                   4.18.4-1build4             amd64        widget libra
ry for Xfce - Gtk+3 variant
ii  libxfce4ui-common                      4.18.4-1build4             all          common files
 for libxfce4ui
ii  libxfce4ui-utils                       4.18.4-1build4             amd64        Utility file
s for libxfce4ui
ii  libxfce4util-bin                       4.18.1-2build3             amd64        tools for li
bxfce4util
ii  libxfce4util-common                    4.18.1-2build3             all          common files
 for libxfce4util
ii  libxfce4util7:amd64                    4.18.1-2build3             amd64        Utility func
tions library for Xfce4
ii  xfce4                                  4.18                       all          Meta-package
 for the Xfce Lightweight Desktop Environment
ii  xfce4-appfinder                        4.18.0-1build2             amd64        Application
finder for the Xfce4 desktop environment
ii  xfce4-battery-plugin:amd64             1.1.5-1build2              amd64        battery moni
tor plugin for the Xfce4 panel
ii  xfce4-clipman                          2:1.6.5-1build2            amd64        clipboard hi
story utility
ii  xfce4-clipman-plugin                   2:1.6.5-1build2            amd64        clipboard hi
story plugin for Xfce panel
ii  xfce4-cpufreq-plugin:amd64             1.2.8-2build2              amd64        cpufreq info
rmation plugin for the Xfce4 panel
ii  xfce4-cpugraph-plugin:amd64            1.2.10-1build2             amd64        CPU load gra
ph plugin for the Xfce4 panel
```

```
ii  xfce4-clipman-plugin                   2:1.6.5-1build2            amd64        clipboard hi
story plugin for Xfce panel
ii  xfce4-cpufreq-plugin:amd64             1.2.8-2build2              amd64        cpufreq info
rmation plugin for the Xfce4 panel
ii  xfce4-cpugraph-plugin:amd64            1.2.10-1build2             amd64        CPU load gra
ph plugin for the Xfce4 panel
ii  xfce4-dict                             0.8.6-1build2              amd64        Dictionary p
lugin for Xfce4 panel
ii  xfce4-diskperf-plugin:amd64            2.7.0-1build2              amd64        disk perform
ance display plugin for the Xfce4 panel
ii  xfce4-fsguard-plugin:amd64             1.1.3-1build2              amd64        filesystem m
onitor plugin for the Xfce4 panel
ii  xfce4-genmon-plugin:amd64              4.1.1-1build2              amd64        Generic Moni
tor for the Xfce4 panel
ii  xfce4-goodies:amd64                    4.18.2build1               amd64        enhancements
 for the Xfce4 Desktop Environment
ii  xfce4-helpers                          4.18.4-0ubuntu3            amd64        helpers for
the Xfce4 desktop environment
admp@Joseph:~$
```

Install XRDP:

XRDP provides a Linux Remote Desktop Protocol server.

Install XRDP, its Xorg integration and D-Bus X11 support:

Run command below to install:

sudo apt install -y xrdp xorgxrdp dbus-x11

```
admp@Joseph:~$  sudo apt install -y xrdp xorgxrdp dbus-x11
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
xrdp is already the newest version (0.9.24-4).
xorgxrdp is already the newest version (1:0.9.19-1).
dbus-x11 is already the newest version (1.14.10-4ubuntu4.1).
0 upgraded, 0 newly installed, 0 to remove and 61 not upgraded.
admp@Joseph:~$
```

Package	Purpose
Xrdp	RDP server that accepts Remote Desktop

©Joseph Aragbaiye Project, 2026

Xorgxrdp	XRDP integration with the Xorg display server
dbus-x11	D-Bus support for X11 graphical desktop sessions

Verify XRDP installation:

xrdp –version

```
admp@Joseph:~$  sudo apt install -y xrdp xorgxrdp dbus-x11
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
xrdp is already the newest version (0.9.24-4).
xorgxrdp is already the newest version (1:0.9.19-1).
dbus-x11 is already the newest version (1.14.10-4ubuntu4.1).
0 upgraded, 0 newly installed, 0 to remove and 61 not upgraded.
admp@Joseph:~$ xrdp --version
xrdp 0.9.24
  A Remote Desktop Protocol Server.
  Copyright (C) 2004-2020 Jay Sorg, Neutrino Labs, and all contributors.
  See https://github.com/neutrinolabs/xrdp for more information.

  Configure options:
      --enable-ipv6
      --enable-jpeg
      --enable-fuse
      --enable-rfxcodec
      --enable-opus
      --enable-painter
      --enable-vsock
      --build=x86_64-linux-gnu
      --prefix=/usr
      --includedir=${prefix}/include
      --mandir=${prefix}/share/man
      --infodir=${prefix}/share/info
      --sysconfdir=/etc
      --localstatedir=/var
```

```
      --enable-fuse
      --enable-rfxcodec
      --enable-opus
      --enable-painter
      --enable-vsock
      --build=x86_64-linux-gnu
      --prefix=/usr
      --includedir=${prefix}/include
      --mandir=${prefix}/share/man
      --infodir=${prefix}/share/info
      --sysconfdir=/etc
      --localstatedir=/var
      --disable-silent-rules
      --libdir=${prefix}/lib/x86_64-linux-gnu
      --libexecdir=${prefix}/lib/x86_64-linux-gnu
      --disable-maintainer-mode
      --disable-dependency-tracking
      --with-socketdir=/run/xrdp/sockdir
      build_alias=x86_64-linux-gnu
      CFLAGS=-g -O2 -fno-omit-frame-pointer -mno-omit-leaf-frame-pointer -ffile-prefix-map=/build/xrdp-Fn5kxo/xrdp-0.9.2
4=. -flto=auto -ffat-lto-objects -fstack-protector-strong -fstack-clash-protection -Wformat -Werror=format-security -fcf
-protection -fdebug-prefix-map=/build/xrdp-Fn5kxo/xrdp-0.9.24=/usr/src/xrdp-0.9.24-4
      LDFLAGS=-Wl,-Bsymbolic-functions -flto=auto -ffat-lto-objects -Wl,-z,relro -Wl,-z,now -Wl,--as-needed
      CPPFLAGS=-Wdate-time -D_FORTIFY_SOURCE=3
      PKG_CONFIG_PATH=/build/xrdp-Fn5kxo/xrdp-0.9.24/pkgconfig

  Compiled with OpenSSL 3.0.13 30 Jan 2024
admp@Joseph:~$
```

©Joseph Aragbaiye Project, 2026

A successful installation prints an XRDP version and identifies it as a Remote Desktop Protocol server. This proves the package is installed, but it does not yet prove the XRDP service is running.

Why XRDP is moved from port 3389

TCP port 3389 is the standard RDP port and may already be associated with Windows Remote Desktop services. Using 3389 for both Windows and the WSL XRDP server can create a port or session conflict or cause the Remote Desktop client to connect to the wrong service. Therefore, I used TCP 3390 for XRDP inside WSL.

Change XRDP port to 3390

Change the configured XRDP listening port:

sudo sed -i 's/^port=3389$/port=3390/' /etc/xrdp/xrdp.ini

Verify the setting:

```
admp@Joseph:~$ grep '^port=' /etc/xrdp/xrdp.ini
port=3390
port=-1
port=-1
port=ask5900
port=ask3389
admp@Joseph:~$
```

Configure XRDP certificate access:

XRDP uses certificate files such as **/etc/xrdp/cert.pem and /etc/xrdp/key.pem.** Add the **xrdp service account** to the **ssl-cert group:**

Run this command on your Linux environment: because I have already added the xrdp service reason you likely see the user **'xrdp' is already a member of 'ssl-cert'.** For you, it will newly add your xrdp service to the ssl-cert group.

sudo adduser xrdp ssl-cert

```
admp@Joseph:~$ sudo adduser xrdp ssl-cert
[sudo] password for admp:
info: The user `xrdp' is already a member of `ssl-cert'.
admp@Joseph:~$
```

Check membership: The result should include ssl-cert.

Run the command below to check membership:

groups xrdp

```
admp@Joseph:~$ groups xrdp
xrdp : xrdp ssl-cert
admp@Joseph:~$
```

Then verify that the private key is readable:

sudo -u xrdp test -r /etc/xrdp/key.pem && echo "XRDP TLS key readable" || echo "XRDP TLS key NOT readable"

©Joseph Aragbaiye Project, 2026

```
admp@Joseph:~$ sudo -u xrdp test -r /etc/xrdp/key.pem && echo "XRDP TLS key readable" || echo "XRDP TLS key NOT
readable"
XRDP TLS key readable
admp@Joseph:~$
```

Configure XFCE as the **XRDP desktop:**

XRDP needs to know which desktop session to start for the Linux user. A basic XFCE configuration should be run on the command line:

echo "startxfce4" > ~/.xsession

chmod +x ~/.xsession

cat ~/.xsession

The expected content is **startxfce4.** The file is stored as /home/admp/.xsession

```
admp@Joseph:~$ echo "startxfce4" > ~/.xsession
chmod +x ~/.xsession
cat ~/.xsession
startxfce4
admp@Joseph:~$
```

To avoid black screen after xrdp login:

Backup and explicitly force the XRDP session to use X11:

Run the command below:

cp ~/.xsession ~/.xsession.backup 2>/dev/null || true

cat > ~/.xsession <<'EOF' #!/bin/sh

XRDP_DISPLAY="$DISPLAY"

unset WAYLAND_DISPLAY

unset WAYLAND_SOCKET

unset DBUS_SESSION_BUS_ADDRESS

export DISPLAY="$XRDP_DISPLAY"

export XDG_SESSION_TYPE=x11

export XDG_CURRENT_DESKTOP=XFCE

export DESKTOP_SESSION=xfce

export GDK_BACKEND=x11

export QT_QPA_PLATFORM=xcb

exec dbus-run-session -- startxfce4

EOF

chmod +x ~/.xsession

cat ~/.xsession

©Joseph Aragbaiye Project, 2026

Remove cached XFCE session data before reconnecting:

rm -rf ~/.cache/sessions/*

On a fresh system this may remove nothing, but it is safe and can prevent stale session data from affecting a new XRDP login.

Configure XRDP to start automatically under systemd:

```
admp@Joseph:~$ sudo systemctl enable xrdp
[sudo] password for admp:
Synchronizing state of xrdp.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable xrdp
admp@Joseph:~$
```

Restart both the session manager and the XRDP server:

```
admp@Joseph:~$ sudo systemctl restart xrdp-sesman
sudo systemctl restart xrdp
admp@Joseph:~$
```

xrdp-sesman manages Linux graphical login sessions; xrdp accepts incoming RDP client connections.

Inspect the full-service status:

```
admp@Joseph:~$ systemctl status xrdp --no-pager
● xrdp.service - xrdp daemon
     Loaded: loaded (/usr/lib/systemd/system/xrdp.service; enabled; preset: enabled)
     Active: active (running) since Wed 2026-09-16 06:46:31 BST; 2min 14s ago
       Docs: man:xrdp(8)
             man:xrdp.ini(5)
    Process: 12666 ExecStartPre=/bin/sh /usr/share/xrdp/socksetup (code=exited, status=0/SUCCESS)
    Process: 12675 ExecStart=/usr/sbin/xrdp $XRDP_OPTIONS (code=exited, status=0/SUCCESS)
   Main PID: 12676 (xrdp)
      Tasks: 1 (limit: 9356)
     Memory: 852.0K (peak: 2.3M)
        CPU: 73ms
     CGroup: /system.slice/xrdp.service
             └─12676 /usr/sbin/xrdp

Sep 16 06:46:29 Joseph systemd[1]: Starting xrdp.service - xrdp daemon...
Sep 16 06:46:30 Joseph xrdp[12675]: [INFO ] address [0.0.0.0] port [3390] mode 1
Sep 16 06:46:30 Joseph xrdp[12675]: [INFO ] listening to port 3390 on 0.0.0.0
Sep 16 06:46:30 Joseph xrdp[12675]: [INFO ] xrdp_listen_pp done
Sep 16 06:46:30 Joseph systemd[1]: xrdp.service: Can't open PID file /run/xrdp/xrdp.pid (yet?) after start: No …irectory
Sep 16 06:46:31 Joseph systemd[1]: Started xrdp.service - xrdp daemon.
Sep 16 06:46:32 Joseph xrdp[12676]: [INFO ] starting xrdp with pid 12676
Sep 16 06:46:32 Joseph xrdp[12676]: [INFO ] address [0.0.0.0] port [3390] mode 1
Sep 16 06:46:32 Joseph xrdp[12676]: [INFO ] listening to port 3390 on 0.0.0.0
Sep 16 06:46:32 Joseph xrdp[12676]: [INFO ] xrdp_listen_pp done
Hint: Some lines were ellipsized, use -l to show in full.
admp@Joseph:~$
```

A healthy service reports Active: active (running). For compact verification use:

```
admp@Joseph:~$ systemctl is-active xrdp
systemctl is-active xrdp-sesman
active
active
admp@Joseph:~$
```

Both should return active.

Check the active listening socket:

sudo ss -ltnp | grep 3390

A healthy result resembles the below output:

©Joseph Aragbaiye Project, 2026

```
admp@Joseph:~$ sudo ss -ltnp | grep 3390
LISTEN 0      2                  *:3390             *:*    users:(("xrdp",pid=12676,fd=11))
admp@Joseph:~$
```

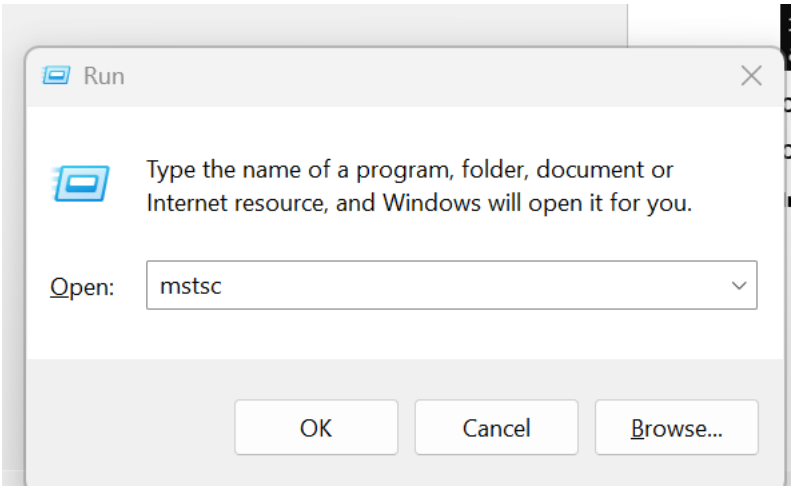
This is stronger evidence than simply showing the package version because it proves that the XRDP process is running and accepting network connections.

Clear any cached sudo authentication and test the password interactively:

```
admp@Joseph:~$ sudo -k
sudo whoami
[sudo] password for admp:
root
admp@Joseph:~$
```

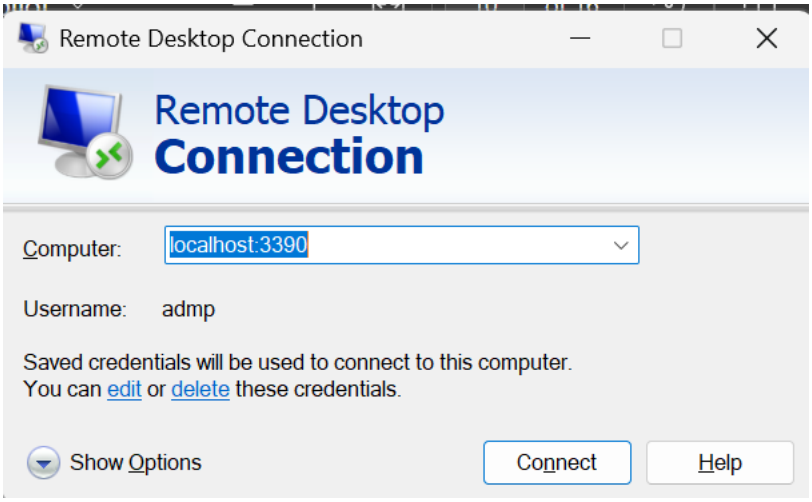
Connect to GUI using Windows Remote Desktop

On Windows, press **Win + R**, type **mstsc** IN THE RUN box then press Enter.



In the Computer field enter:

localhost:3390



At the **XRDP** login screen select the **Xorg** session and authenticate with the Linux credentials(your own username and password):

©Joseph Aragbaiye Project, 2026

Session: Xorg

Username: admp

Password:



After configuration, run this consolidated verification block inside Ubuntu:

echo "=== USER ==="

whoami echo "=== HOSTNAME ===" hostname

echo "=== INIT SYSTEM ===" ps -p 1 -o comm=

echo "=== OPERATING SYSTEM ===" grep PRETTY_NAME /etc/os-release

echo "=== XFCE ===" which startxfce4

echo "=== XRDP VERSION ===" xrdp --version | head -1

echo "=== XRDP STATUS ===" systemctl is-active xrdp

echo "=== XRDP SESSION MANAGER ==="

systemctl is-active xrdp-sesman

echo "=== XRDP CONFIGURED PORT ==="

grep '^port=' /etc/xrdp/xrdp.ini

echo "=== XRDP LISTENING PORT ==="

sudo ss -ltnp | grep 3390

echo "=== DESKTOP SESSION CONFIGURATION ==="

©Joseph Aragbaiye Project, 2026

cat ~/.xsession

A healthy result should show:

admp,

Joseph,

systemd,

Ubuntu 24.04.x LTS,

/usr/bin/startxfce4, active XRDP services and port=3390

In the conclusion of my lab setup:

The prompt **admp@Joseph:~$** is not created by WSL UI.

The username admp is created during Ubuntu first run setup, while Joseph is the hostname, either inherited from the Windows computer name or explicitly from **Windows 11 WSL 2 / WSL UI / XFCE / XRDP** Implementation setup configuration through **/etc/wsl.conf.**

WSL UI is a separate graphical management layer. XFCE provides the desktop, XRDP provides remote desktop connectivity, and WSL 2 provides the Linux execution platform.

The use of port 3390 is deliberate so that the WSL XRDP service does not compete with Windows Remote Desktop services on the conventional RDP port 3389.

The use of Xorg and an explicit X11-oriented .xsession is also deliberate because WSLg / Wayland environment variables can otherwise cause an authenticated XRDP session to show only a black screen.

Reference sources:

The implementation is based on standard Microsoft WSL administration concepts and WSL UI vendor documentation.

These sources are useful for verifying commands and behaviour as the products evolve. Microsoft Learn –

Install WSL **- https://learn.microsoft.com/en-us/windows/wsl/install**

Microsoft Learn - Basic WSL commands - **https://learn.microsoft.com/en-us/windows/wsl/basic-commands**

Microsoft Learn - Advanced settings configuration in WSL - **https://learn.microsoft.com/en-us/windows/wsl/wsl config**

Microsoft Learn - Run Linux GU apps with WSL – **https://learn.microsoft.com/en-us/windows/wsl/tutorials/gui**

apps Microsoft Learn - Set up a WSL development environment - **https://learn.microsoft.com/en-us/windows/wsl/setup/environment**

WSL UI - Downloads - https://wsl-ui.octasoft.co.uk/downloads

WSL UI - **User Guide - https://wsl-ui.octasoft.co.uk/docs/user-guide**

©Joseph Aragbaiye Project, 2026

Bitcoin Core 31.1 on WSL2

Simple Step-by-Step Installation, Configuration, Wallet, Historical Blockchain and Real Time Mempool Unconfirmed Transaction.

Platform: Windows 11

We will install the Bitcoin Core local node in the Linux Ubuntu environment on WSL2. The version we will install is Bitcoin Core 31.1 from a bitcoin network Mainnet. For the user of the bitcoin node will be admp which is my local Linux environment for my lab… Your own Linux user will likely differ mine depending on the name given to your Linux as a user. The purpose is Historical Blockchain + live mempool + wallet + RPC + ZMQ + future python/Kafka analytics.

Note: I will keep the code documentation and clear as possible so to understand why you are doing what you are doing when you run each command. In each part you will be instructed to run a command, I will be answering the question why? Why you did what you have been instructed to do.

To understand the flow, I will start with the question **what?**

1. What Are We Building?

We are installing **Bitcoin Core 31.1** inside Ubuntu running through WSL2.

What is the benefit for that?

The completed system will provide:

- ✓ Historical Bitcoin blockchain data
- ✓ Confirmed transaction data
- ✓ Live unconfirmed transaction data from the mempool
- ✓ Transaction fees
- ✓ Transaction size
- ✓ Transaction virtual size (vsize)
- ✓ Transaction weight
- ✓ Wallet functionality
- ✓ Local RPC access
- ✓ ZMQ notifications
- ✓ A foundation for a future Python and Kafka analytics system

©Joseph Aragbaiye Project, 2026

Why?

Bitcoin Core will be the main source of Live mempool Bitcoin data collection system replacing Third-Party Mempool API. Kafka can stream directly from our Bitcoin Core node, and the models can detect anomality's in the transaction and store the detected transaction in SQLite and virtualize them.

2. Important Configuration Decisions

We are configuring Bitcoin Core with:

Bitcoin Core 31.1

Bitcoin Mainnet

Wallet enabled

txindex=1

Mempool enabled

RPC enabled

ZMQ enabled

Unpruned blockchain

systemd service

Why?

This configuration is designed to support both:

- **Historical blockchain analysis**
- **Live mempool monitoring**

We are deliberately keeping the blockchain **unpruned** because the project requires access to historical block data.

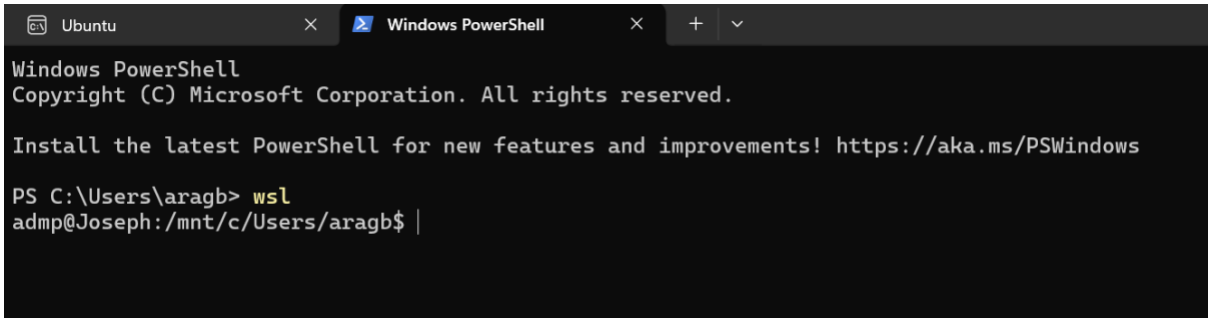
Important: An unpruned Bitcoin node requires significant disk space. Make sure the WSL/Windows storage has enough capacity before starting synchronization.

3. Enter Ubuntu

From Windows PowerShell, run:

wsl

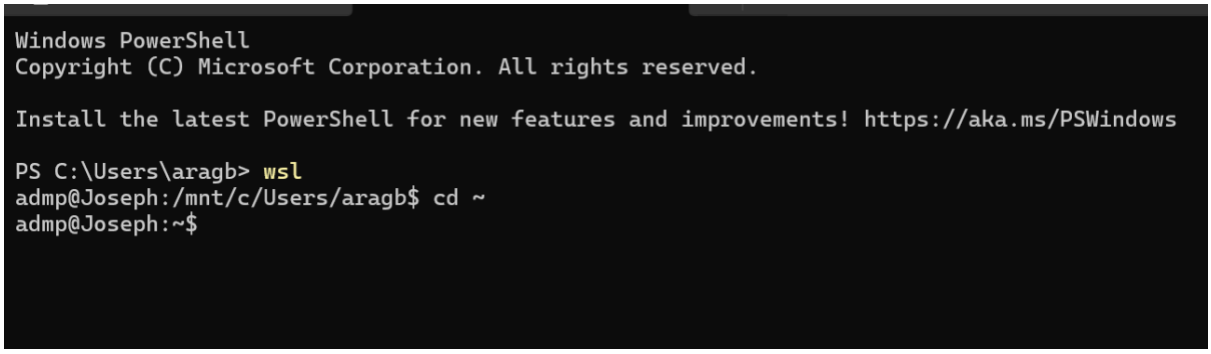
©Joseph Aragbaiye Project, 2026



Or open Ubuntu directly from Windows.

You should see something like:

admp@Joseph:~$



Why?

All of the Bitcoin Core commands in this guide are Linux commands. They should be run inside the Ubuntu WSL2 environment.

4. Check the Current User

Run:

Whoami

Expected:

admp



©Joseph Aragbaiye Project, 2026

Why?

We need to confirm that we are installing and configuring Bitcoin Core under the intended Linux account.

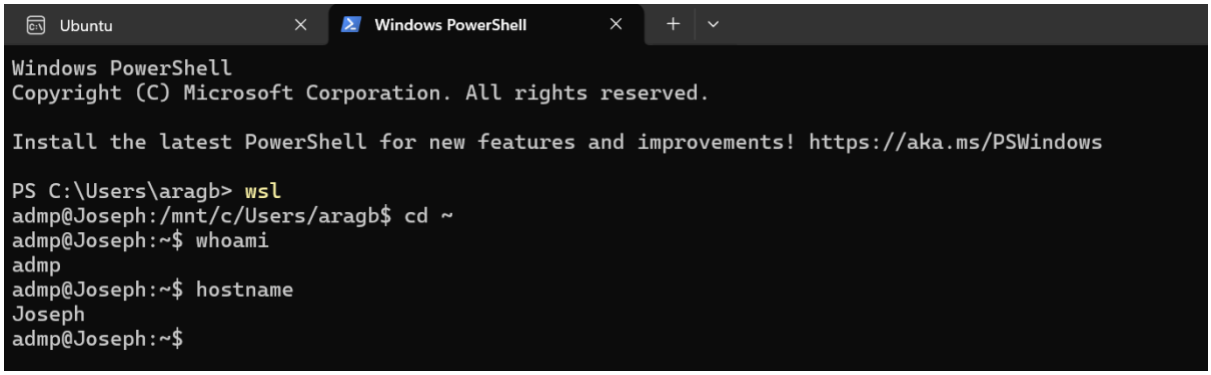
5. Check the Hostname

Run:

hostname

You may see (watever your hostname is, you see):

Joseph



Why?

This confirms which Linux/WSL environment you are currently working in.

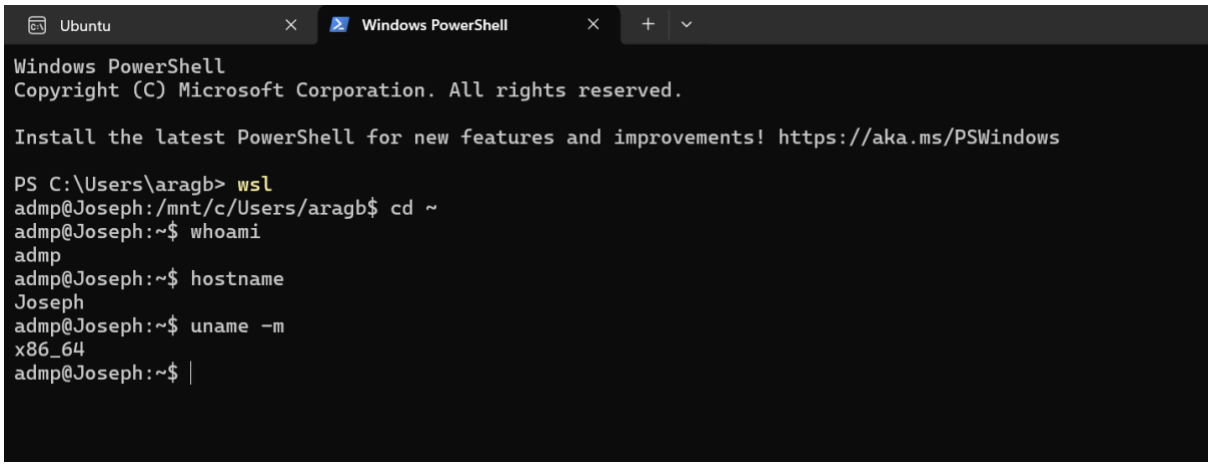
6. Check the CPU Architecture

Run:

uname -m

Expected on a normal x86-64 computer:

x86_64



©Joseph Aragbaiye Project, 2026

Why?

Bitcoin Core provides different packages for different CPU architectures.

For an x86_64 computer, we need the:

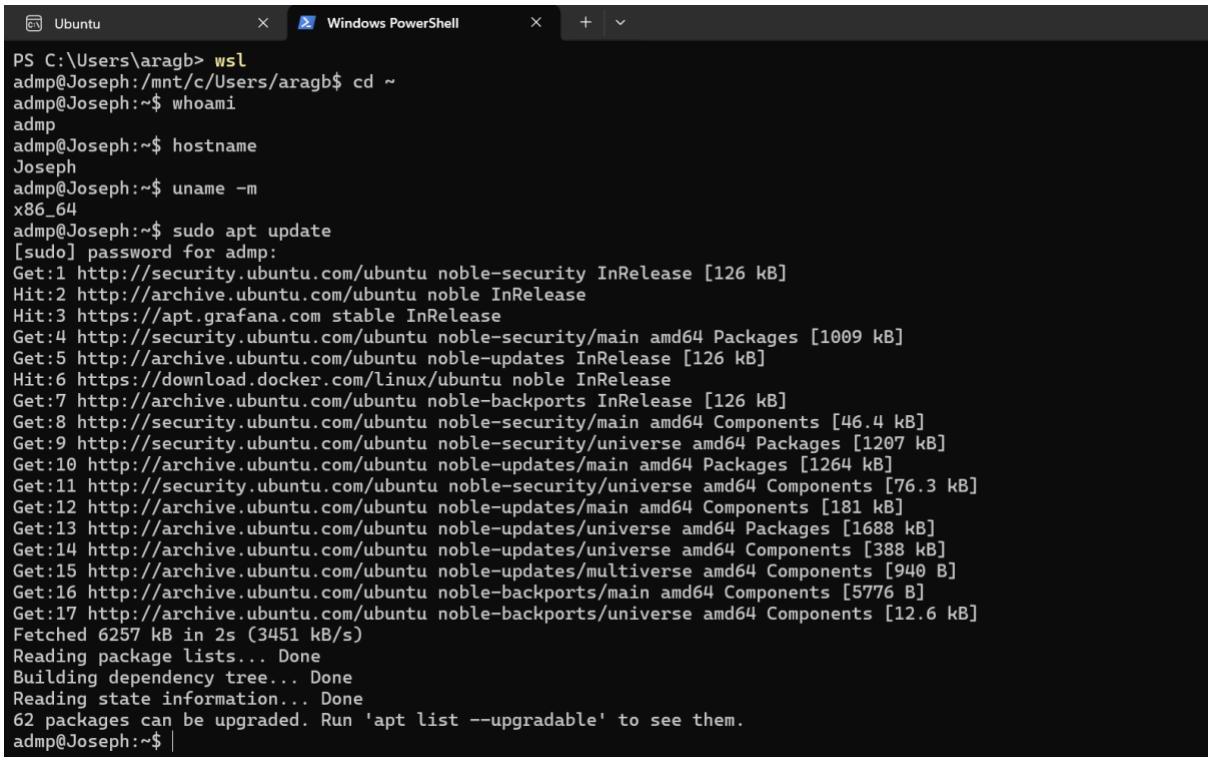
x86_64-linux-gnu

Bitcoin Core package.

7. Update Ubuntu

Run:

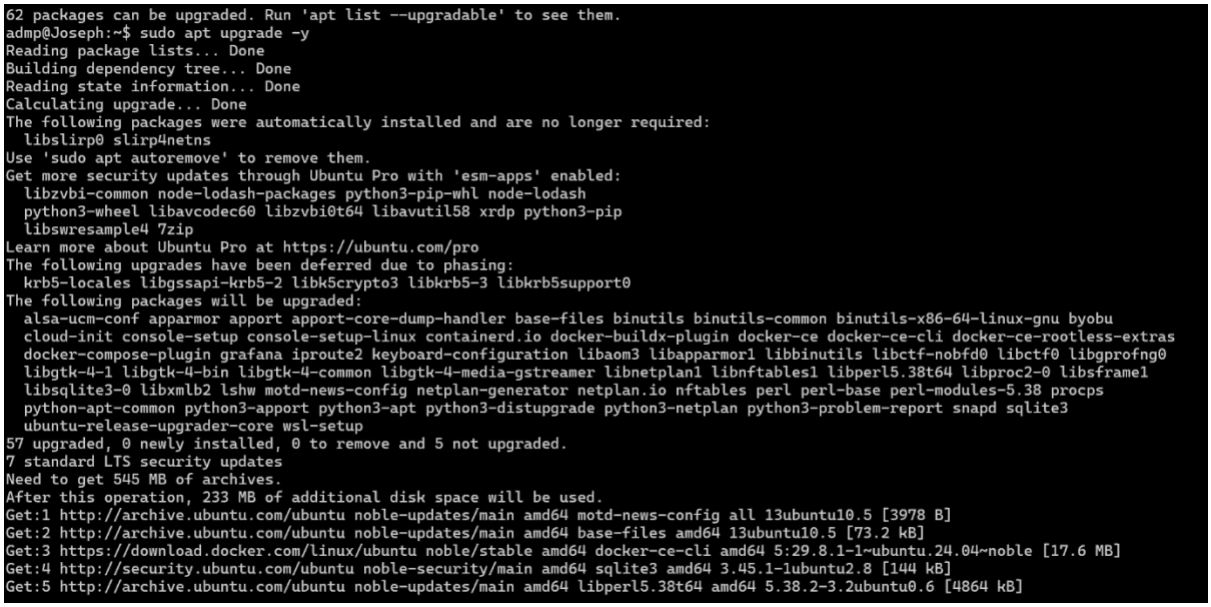
sudo apt update



Then:

sudo apt upgrade -y

©Joseph Aragbaiye Project, 2026



Why?

apt update refreshes Ubuntu's package information.

apt upgrade installs available updates for packages already installed on the system.

Keeping the operating system updated reduces problems when installing and running Bitcoin Core.

8. Install Required Tools

Run:

sudo apt install -y \

curl \

jq \

gnupg \

git \

ca-certificates \

tar \

wget

©Joseph Aragbaiye Project, 2026

```
admp@Joseph:~$ sudo apt install -y \
    curl \
    jq \
    gnupg \
    git \
    ca-certificates \
    tar \
    wget
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
curl is already the newest version (8.5.0-2ubuntu10.13).
jq is already the newest version (1.7.1-3ubuntu0.24.04.2).
gnupg is already the newest version (2.4.4-2ubuntu17.6).
git is already the newest version (1:2.43.0-1ubuntu7.3).
ca-certificates is already the newest version (20260601~24.04.1).
tar is already the newest version (1.35+dfsg-3ubuntu0.4).
wget is already the newest version (1.21.4-1ubuntu4.5).
The following packages were automatically installed and are no longer required:
  libslirp0 slirp4netns
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 5 not upgraded.
admp@Joseph:~$
```

Why?

These programs are needed for different parts of the installation:

Tool	Purpose
curl	Downloads Bitcoin Core files
jq	Reads and formats Bitcoin Core JSON output
gnupg	Verifies cryptographic signatures
git	Downloads the Bitcoin Core signature repository
ca-certificates	Helps verify HTTPS connections
tar	Extracts the Bitcoin Core archive
wget	Alternative download utility

9. Verify the Required Tools

Run:

curl --version | head -1

```
admp@Joseph:~$ curl --version | head -1
curl 8.5.0 (x86_64-pc-linux-gnu) libcurl/8.5.0 OpenSSL/3.0.13 zlib/1.3 brotli/1.1.0 zstd/1.5.5 libidn2/2.3.7 libpsl/0.21.2 (+libidn2/
2.3.7) libssh/0.10.6/openssl/zlib nghttp2/1.59.0 librtmp/2.3 OpenLDAP/2.6.10
admp@Joseph:~$
```

Run:

gpg --version | head -1

©Joseph Aragbaiye Project, 2026

```
admp@Joseph:~$ gpg --version | head -1
gpg (GnuPG) 2.4.4
admp@Joseph:~$
```

Run:

git –version

```
admp@Joseph:~$ git --version
git version 2.43.0
admp@Joseph:~$
```

Run:

jq –version

```
admp@Joseph:~$ jq --version
jq-1.7
admp@Joseph:~$
```

Why?

We are confirming that the required programs were installed successfully before continuing.

10. Create the Bitcoin Core Installation Directory

Run:

mkdir -p ~/bitcoin-install

```
admp@Joseph:~$ mkdir -p ~/bitcoin-install
admp@Joseph:~$
```

Then:

cd ~/bitcoin-install

```
admp@Joseph:~$ mkdir -p ~/bitcoin-install
admp@Joseph:~$ cd ~/bitcoin-install
admp@Joseph:~/bitcoin-install$
```

Then:

Pwd

Expected:

/home/admp/bitcoin-install

```
admp@Joseph:~/bitcoin-install$ pwd
/home/admp/bitcoin-install
admp@Joseph:~/bitcoin-install$
```

©Joseph Aragbaiye Project, 2026

Why?

This gives us a dedicated location for the Bitcoin Core installation files.

It is important to understand the difference between the two directories:

~/bitcoin-install

```
admp@Joseph:~/bitcoin-install$ ~/bitcoin-install
-bash: /home/admp/bitcoin-install: Is a directory
admp@Joseph:~/bitcoin-install$
```

This contains downloaded and extracted Bitcoin Core installation files.

~/.bitcoin

```
admp@Joseph:~/bitcoin-install$ ~/.bitcoin
-bash: /home/admp/.bitcoin: Is a directory
admp@Joseph:~/bitcoin-install$
```

This contains Bitcoin Core's actual blockchain, configuration, wallet, indexes, logs, and other node data.

11. Select Bitcoin Core 31.1

Run:

```
VERSION=31.1
```

Why?

This stores the Bitcoin Core version we want to install in a variable.

That allows later commands to automatically use version 31.1.

12. Automatically Select the Correct Architecture

Run:

```
ARCH=$(uname -m)


case "$ARCH" in
  x86_64)
    BTC_ARCH="x86_64-linux-gnu"
    ;;
  aarch64|arm64)
    BTC_ARCH="aarch64-linux-gnu"
    ;;
```

©Joseph Aragbaiye Project, 2026

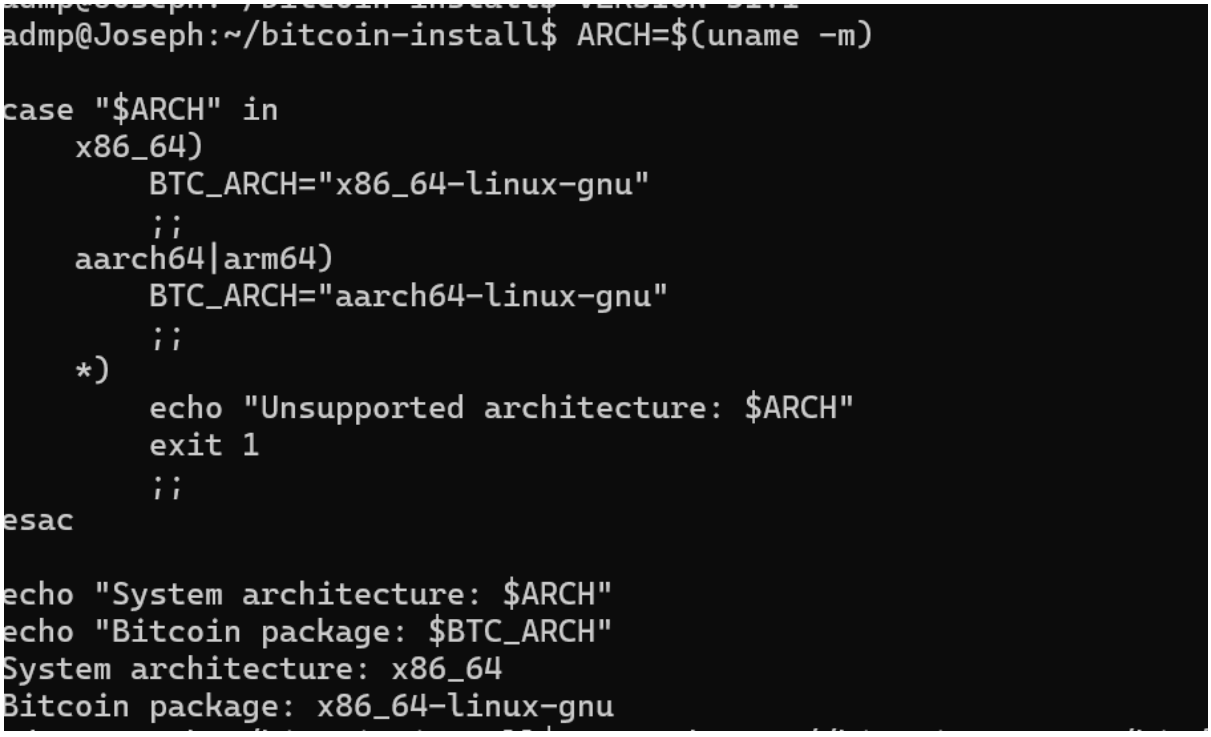
```
  *)
    echo "Unsupported architecture: $ARCH"
    exit 1
    ;;
esac

echo "System architecture: $ARCH"
echo "Bitcoin package: $BTC_ARCH"
```

Expected on your system:

System architecture: x86_64

Bitcoin package: x86_64-linux-gnu



Why?

This prevents us from accidentally downloading a Bitcoin Core package intended for a different CPU architecture.

13. Set the Bitcoin Core Download Location

Run:

BASE="https://bitcoincore.org/bin/bitcoin-core-${VERSION}"

©Joseph Aragbaiye Project, 2026

```
admp@Joseph:~/bitcoin-install$ BASE="https://bitcoincore.org/bin/bitcoin-core-${VERSION}"
admp@Joseph:~/bitcoin-install$
```

Why?

This creates a variable containing the official Bitcoin Core release location.

The following download commands will use this variable.

14. Download Bitcoin Core

Run:

curl -fLO "${BASE}/bitcoin-${VERSION}-${BTC_ARCH}.tar.gz"

```
admp@Joseph:~/bitcoin-install$ curl -fLO "${BASE}/bitcoin-${VERSION}-${BTC_ARCH}.tar.gz"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  2 86.1M    2 2463k    0     0   551k      0  0:02:39  0:00:04  0:02:35  551k
```

Why?

This downloads the Bitcoin Core 31.1 Linux binary archive.

Do not install or extract it yet.

We first verify that the downloaded file is authentic and has not been corrupted.

15. Download the SHA-256 Checksum

Run:

curl -fLO "${BASE}/SHA256SUMS"

```
admp@Joseph:~/bitcoin-install$ curl -fLO "${BASE}/SHA256SUMS"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  2956  100  2956    0     0   5053      0 --:--:-- --:--:-- --:--:--  5052
admp@Joseph:~/bitcoin-install$
```

Why?

SHA256SUMS contains the official SHA-256 checksums for the Bitcoin Core release files.

We will use it to check that our downloaded archive matches the published file.

16. Download the Signed Checksum

Run:

curl -fLO "${BASE}/SHA256SUMS.asc"

©Joseph Aragbaiye Project, 2026

```
admp@Joseph:~/bitcoin-install$ curl -fLO "${BASE}/SHA256SUMS.asc"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  8687  100  8687    0     0  14710      0 --:--:-- --:--:-- --:--:-- 14723
admp@Joseph:~/bitcoin-install$
```

Why?

SHA256SUMS.asc contains a cryptographic signature for the checksum file.

This allows us to perform another verification step using GPG.

17. Check the Downloaded Files

Run:

ls -lh \

"bitcoin-${VERSION}-${BTC_ARCH}.tar.gz" \

SHA256SUMS \

SHA256SUMS.asc

Why?

We are confirming that all three required files were downloaded.

You should see:

bitcoin-31.1-x86_64-linux-gnu.tar.gz

SHA256SUMS

SHA256SUMS.asc

```
admp@Joseph:~/bitcoin-install$ ls -lh \
    "bitcoin-${VERSION}-${BTC_ARCH}.tar.gz" \
    SHA256SUMS \
    SHA256SUMS.asc
-rw-r--r-- 1 admp admp 2.9K Sep 16 23:22 SHA256SUMS
-rw-r--r-- 1 admp admp 8.5K Sep 16 23:24 SHA256SUMS.asc
-rw-r--r-- 1 admp admp  87M Sep 16 23:20 bitcoin-31.1-x86_64-linux-gnu.tar.gz
admp@Joseph:~/bitcoin-install$
```

18. Verify the SHA-256 Checksum

Run:

sha256sum --ignore-missing --check SHA256SUMS

A successful result should look like:

bitcoin-31.1-x86_64-linux-gnu.tar.gz: OK

```
admp@Joseph:~/bitcoin-install$ sha256sum --ignore-missing --check SHA256SUMS
bitcoin-31.1-x86_64-linux-gnu.tar.gz: OK
admp@Joseph:~/bitcoin-install$
```

©Joseph Aragbaiye Project, 2026

Why?

This checks whether the downloaded Bitcoin Core archive matches the official checksum.

If you see:

OK

the downloaded archive matches the checksum published for that file.

If you see:

FAILED

Stop. Do not install the archive.

Delete the archive:

rm -f "bitcoin-${VERSION}-${BTC_ARCH}.tar.gz"

Then download it again.

Important

Do **not** type:

bitcoin-31.1-x86_64-linux-gnu.tar.gz: OK

That is output from the command.

It is not a command.

19. Download the Bitcoin Core Signature Repository

Run:

git clone https://github.com/bitcoin-core/guix.sigs.git

```
admp@Joseph:~/bitcoin-install$ git clone https://github.com/bitcoin-core/guix.sigs.git
Cloning into 'guix.sigs'...
remote: Enumerating objects: 18087, done.
remote: Counting objects: 100% (229/229), done.
remote: Compressing objects: 100% (74/74), done.
remote: Total 18087 (delta 173), reused 176 (delta 154), pack-reused 17858 (from 3)
Receiving objects: 100% (18087/18087), 6.40 MiB | 5.62 MiB/s, done.
Resolving deltas: 100% (8193/8193), done.
Updating files: 100% (5972/5972), done.
admp@Joseph:~/bitcoin-install$
```

Why?

The repository contains information used to verify Bitcoin Core release builds and signatures.

20. Import the Builder Keys

Run:

gpg --import guix.sigs/builder-keys/*

©Joseph Aragbaiye Project, 2026

```
admp@Joseph:~/bitcoin-install$ gpg --import guix.sigs/builder-keys/*
gpg: key 188CBB2648416AD5: 4 signatures not checked due to missing keys
gpg: key 188CBB2648416AD5: ".0xB10C <b10c@b10c.me>" not changed
gpg: key A5E0907A0380E6C3: "CoinForensics (SigningKey) <59567284+coinforensics@users.noreply.github.com>" not changed
gpg: key 2EBB056FD847F8A7: 12 signatures not checked due to missing keys
gpg: key 2EBB056FD847F8A7: "Stephan Oeste (it) <it@oeste.de>" not changed
gpg: key 57FF9BDBCC301009: 42 signatures not checked due to missing keys
gpg: key 57FF9BDBCC301009: "Sjors Provoost <sjors@sprovoost.nl>" not changed
gpg: key 9303B33A305224CB: 15 signatures not checked due to missing keys
gpg: key 9303B33A305224CB: "sedited <seb.kung@gmail.com>" not changed
gpg: key 17565732E08E5E41: "Ava Chow <me@achow101.com>" not changed
gpg: key D7CC770B81FD22A8: 2 signatures not checked due to missing keys
gpg: key D7CC770B81FD22A8: "Ben Carman <benthecarman@live.com>" not changed
gpg: key 1C2491FFEB0EF770: 2 signatures not checked due to missing keys
gpg: key 1C2491FFEB0EF770: "Cory Fields <cfields@bitcoinfoundation.org>" not changed
gpg: key E13FC145CD3F4304: 13 signatures not checked due to missing keys
gpg: key E13FC145CD3F4304: "Antoine Poinsot <darosior@protonmail.com>" not changed
gpg: key 10F2A3D26839321E: "davidgumberg <davidzgumberg@gmail.com>" not changed
gpg: key C37B1C1D44C786EE: "Duncan Dean <duncangleeddean@gmail.com>" not changed
gpg: key 944D35F9AC3DB76A: 14 signatures not checked due to missing keys
gpg: key 944D35F9AC3DB76A: "Michael Ford (bitcoin-otc) <fanquake@gmail.com>" not changed
gpg: key 8F617F1200A6D25C: 6 signatures not checked due to missing keys
gpg: key 8F617F1200A6D25C: "Gloria Zhao <gloriazhao@berkeley.edu>" not changed
gpg: key 8E4256593F177720: 1 signature not checked due to a missing key
gpg: key 8E4256593F177720: "Oliver Gugger <gugger@gmail.com>" not changed
gpg: key 410108112E7EA81F: "Hennadii Stepanov (GitHub key) <32963518+hebasto@users.noreply.github.com>" not changed
gpg: key 0E3908F364989888: "Abubakar Sadiq Ismail <abubakarsadiqismail@proton.me>" not changed
gpg: key D11BD4F33F1DB499: "jackielove4u <jackielove4u@hotmail.com>" not changed
gpg: key 8ADCB558C4F33D65: "josibake@protonmail.com <josibake@protonmail.com>" not changed
gpg: key F62711DBDCA8AE56: "Dimitri <kvaciral@protonmail.com>" not changed
gpg: Note: third-party key signatures using the SHA1 algorithm are rejected
gpg: (use option "--allow-weak-key-signatures" to override)
```

Why?

This imports the public keys required to verify the signatures associated with the Bitcoin Core release process.

21. Verify the GPG Signature

Run:

gpg --verify SHA256SUMS.asc SHA256SUMS

```
admp@Joseph:~/bitcoin-install$ gpg --verify SHA256SUMS.asc SHA256SUMS
gpg: Signature made Tue Jul  7 01:09:57 2026 BST
gpg:                using RSA key 152812300785C96444D3334D17565732E08E5E41
gpg:                issuer "me@achow101.com"
gpg: Good signature from "Ava Chow <me@achow101.com>" [unknown]
gpg:                 aka "Ava Chow <github@achow101.com>" [unknown]
gpg:                 aka "Ava Chow <achow101@pm.me>" [unknown]
gpg: WARNING: The key's User ID is not certified with a trusted signature!
gpg:          There is no indication that the signature belongs to the owner.
Primary key fingerprint: 1528 1230 0785 C964 44D3  334D 1756 5732 E08E 5E41
gpg: Signature made Tue Jul  7 09:53:43 2026 BST
gpg:                using RSA key CFB16E21C950F67FA95E558F2EEB9F5CC09526C1
gpg:                issuer "fanquake@gmail.com"
gpg: Good signature from "Michael Ford (bitcoin-otc) <fanquake@gmail.com>" [unknown]
gpg: Signature notation: manu=2,2.5+1.12,0,3
gpg: WARNING: The key's User ID is not certified with a trusted signature!
gpg:          There is no indication that the signature belongs to the owner.
Primary key fingerprint: E777 299F C265 DD04 7930  70EB 944D 35F9 AC3D B76A
     Subkey fingerprint: CFB1 6E21 C950 F67F A95E  558F 2EEB 9F5C C095 26C1
gpg: Signature made Tue Jul  7 18:20:17 2026 BST
gpg:                using RSA key D1DBF2C4B96F2DEBF4C16654410108112E7EA81F
gpg:                issuer "hebasto@gmail.com"
gpg: Good signature from "Hennadii Stepanov (GitHub key) <32963518+hebasto@users.noreply.github.com>" [unknown]
gpg:                 aka "Hennadii Stepanov (hebasto) <hebasto@gmail.com>" [unknown]
gpg: WARNING: The key's User ID is not certified with a trusted signature!
gpg:          There is no indication that the signature belongs to the owner.
Primary key fingerprint: D1DB F2C4 B96F 2DEB F4C1  6654 4101 0811 2E7E A81F
gpg: Signature made Tue Jul  7 09:42:48 2026 BST
gpg:                using RSA key 0AD83877C1F0CD1EE9BD660AD7CC770B81FD22A8
```

Why?

We already checked that the downloaded archive matches its checksum.

This second check verifies the signature on the checksum file.

©Joseph Aragbaiye Project, 2026

In simple terms:

SHA-256 verification

Checks the downloaded file.

GPG verification

Checks the signed checksum information.

You should see valid/good signature information.

A GPG trust warning is not necessarily the same thing as an invalid cryptographic signature.

22. Extract Bitcoin Core

Only continue after the verification steps succeed.

Run:

tar -xzf "bitcoin-${VERSION}-${BTC_ARCH}.tar.gz"

```
admp@Joseph:~/bitcoin-install$ tar -xzf "bitcoin-${VERSION}-${BTC_ARCH}.tar.gz"
admp@Joseph:~/bitcoin-install$
```

Why?

The Bitcoin Core archive contains the executable programs we need.

This command extracts them so they can be installed.

---

23. Check the Extracted Bitcoin Core Programs

Run:

ls -lh "bitcoin-${VERSION}/bin/"

You should see programs such as:

bitcoind

bitcoin-cli

bitcoin-qt

bitcoin-tx

bitcoin-util

©Joseph Aragbaiye Project, 2026

```
admp@Joseph:~/bitcoin-install$ ls -lh "bitcoin-${VERSION}/bin/"
total 81M
-rwxr-xr-x 1 admp admp 1.9M Jul  6 14:09 bitcoin
-rwxr-xr-x 1 admp admp 2.6M Jul  6 14:09 bitcoin-cli
-rwxr-xr-x 1 admp admp  42M Jul  6 14:09 bitcoin-qt
-rwxr-xr-x 1 admp admp 4.7M Jul  6 14:09 bitcoin-tx
-rwxr-xr-x 1 admp admp 4.3M Jul  6 14:09 bitcoin-util
-rwxr-xr-x 1 admp admp 8.9M Jul  6 14:09 bitcoin-wallet
-rwxr-xr-x 1 admp admp  18M Jul  6 14:09 bitcoind
admp@Joseph:~/bitcoin-install$
```

Why?

We are confirming that the Bitcoin Core archive contains the expected programs.

The important programs are:

bitcoind

The Bitcoin Core node itself.

bitcoin-cli

The command-line program used to communicate with Bitcoin Core.

bitcoin-qt

The graphical Bitcoin Core application.

bitcoin-tx

A Bitcoin transaction utility.

bitcoin-util

Additional Bitcoin Core utilities.

24. Install Bitcoin Core

Run:

```
sudo install -m 0755 \
  "bitcoin-${VERSION}/bin/"* \
  /usr/local/bin/
```

```
admp@Joseph:~/bitcoin-install$ sudo install -m 0755 \
    "bitcoin-${VERSION}/bin/"* \
    /usr/local/bin/
[sudo] password for admp:
admp@Joseph:~/bitcoin-install$
```

Why?

This copies the Bitcoin Core programs into:

©Joseph Aragbaiye Project, 2026

/usr/local/bin/

This is a standard location for locally installed Linux programs.

25. Refresh the Shell

Run:

hash -r

```
[sudo] password for admp:
admp@Joseph:~/bitcoin-install$ hash -r
admp@Joseph:~/bitcoin-install$
```

Why?

Your shell may remember where commands were previously found.

hash -r refreshes that information so Bash can find the newly installed Bitcoin Core programs.

26. Check Where Bitcoin Core Was Installed

Run:

which bitcoind

Expected:

/usr/local/bin/bitcoind

```
admp@Joseph:~/bitcoin-install$ which bitcoind
/usr/local/bin/bitcoind
admp@Joseph:~/bitcoin-install$
```

Run:

which bitcoin-cli

Expected:

/usr/local/bin/bitcoin-cli

```
admp@Joseph:~/bitcoin-install$ which bitcoin-cli
/usr/local/bin/bitcoin-cli
admp@Joseph:~/bitcoin-install$
```

Why?

This confirms that Linux can find the Bitcoin Core programs.

27. Check the Bitcoin Core Version

Run:

©Joseph Aragbaiye Project, 2026

bitcoind –version

```
admp@Joseph:~/bitcoin-install$ bitcoind --version
Bitcoin Core daemon version v31.1.0 bitcoind
Copyright (C) 2009-2026 The Bitcoin Core developers

Please contribute if you find Bitcoin Core useful. Visit
<https://bitcoincore.org/> for further information about the software.
The source code is available from <https://github.com/bitcoin/bitcoin>.

This is experimental software.
Distributed under the MIT software license, see the accompanying file COPYING
or <https://opensource.org/license/MIT>
admp@Joseph:~/bitcoin-install$
```

Then:

bitcoin-cli --version

Expected:

Bitcoin Core version v31.1.0

```
admp@Joseph:~/bitcoin-install$ bitcoind --version
Bitcoin Core daemon version v31.1.0 bitcoind
Copyright (C) 2009-2026 The Bitcoin Core developers

Please contribute if you find Bitcoin Core useful. Visit
<https://bitcoincore.org/> for further information about the software.
The source code is available from <https://github.com/bitcoin/bitcoin>.

This is experimental software.
Distributed under the MIT software license, see the accompanying file COPYING
or <https://opensource.org/license/MIT>
admp@Joseph:~/bitcoin-install$ bitcoin-cli --version
Bitcoin Core RPC client version v31.1.0
Copyright (C) 2009-2026 The Bitcoin Core developers

Please contribute if you find Bitcoin Core useful. Visit
<https://bitcoincore.org/> for further information about the software.
The source code is available from <https://github.com/bitcoin/bitcoin>.

This is experimental software.
Distributed under the MIT software license, see the accompanying file COPYING
or <https://opensource.org/license/MIT>
admp@Joseph:~/bitcoin-install$
```

and:

Bitcoin Core RPC client version v31.1.0

Why?

We want to confirm that the correct Bitcoin Core version was installed.

At this point:

Installed

©Joseph Aragbaiye Project, 2026

does not mean:

Configured

and it does not mean:

Synchronized

Those are separate stages.

28. Create the Bitcoin Data Directory

Run:

mkdir -p ~/.bitcoin

```
admp@Joseph:~/bitcoin-install$ mkdir -p ~/.bitcoin
admp@Joseph:~/bitcoin-install$
```

Why?

Bitcoin Core needs a dedicated directory for its data and configuration.

For user admp, this will normally be:

/home/admp/.bitcoin

---

29. Secure the Bitcoin Data Directory

Run:

chmod 700 ~/.bitcoin

```
admp@Joseph:~/bitcoin-install$ chmod 700 ~/.bitcoin
admp@Joseph:~/bitcoin-install$
```

Why?

This restricts access to the directory.

The directory can contain sensitive information, especially wallet-related information.

30. Check the Directory

Run:

ls -ld ~/.bitcoin

```
admp@Joseph:~/bitcoin-install$ ls -ld ~/.bitcoin
drwx------ 6 admp admp 4096 Sep 16 23:55 /home/admp/.bitcoin
admp@Joseph:~/bitcoin-install$
```

Why?

©Joseph Aragbaiye Project, 2026

This confirms that the directory exists and allows us to inspect its permissions.

31. Create the Bitcoin Core Configuration File

Run:

nano ~/.bitcoin/bitcoin.conf

Why?

bitcoin.conf tells Bitcoin Core how we want the node to operate.

Paste the following:

Bitcoin Core 31.1

Windows 11 / WSL2 / Ubuntu 24.04

Bitcoin Mainnet

Historical Blockchain + Mempool + Wallet + ZMQ

NETWORK

Use Bitcoin Mainnet.

chain=main

Allow Bitcoin Core to accept peer-to-peer connections.

listen=1

RPC

Enable the JSON-RPC server.

server=1

Only make RPC available locally.

rpcbind=127.0.0.1

Only allow local RPC connections.

rpcallowip=127.0.0.1

Number of RPC worker threads.

©Joseph Aragbaiye Project, 2026

```
rpcthreads=16

# Maximum RPC work queue.
rpcworkqueue=128

# WALLET
# Keep Bitcoin Core wallet functionality enabled.
disablewallet=0

# HISTORICAL TRANSACTION INDEX
# Maintain an index for historical transaction lookup.
txindex=1

# MEMPOOL
# Save the mempool between restarts.
persistmempool=1
# Maximum mempool size in MB.
maxmempool=10000

# DATABASE CACHE
# Memory available for database caching.
dbcache=4096

# ZMQ
# Publish raw transaction notifications.
zmqpubrawtx=tcp://127.0.0.1:28332

# Publish raw block notifications.
zmqpubrawblock=tcp://127.0.0.1:28333
```

Save the file:

Ctrl+O

©Joseph Aragbaiye Project, 2026

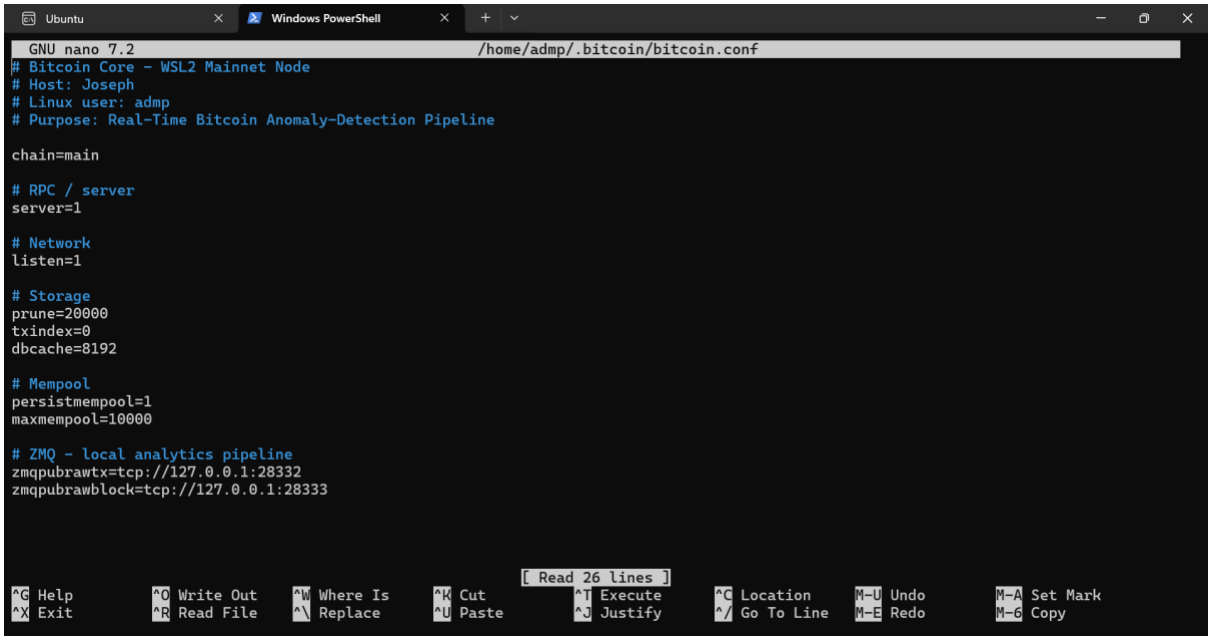
Press:

Enter

Exit:

Ctrl+X

This is my own configuration below....



32. Secure the Configuration File

Run:

chmod 600 ~/.bitcoin/bitcoin.conf

```
admp@Joseph:~/bitcoin-install$ chmod 600 ~/.bitcoin/bitcoin.conf
admp@Joseph:~/bitcoin-install$
```

Why?

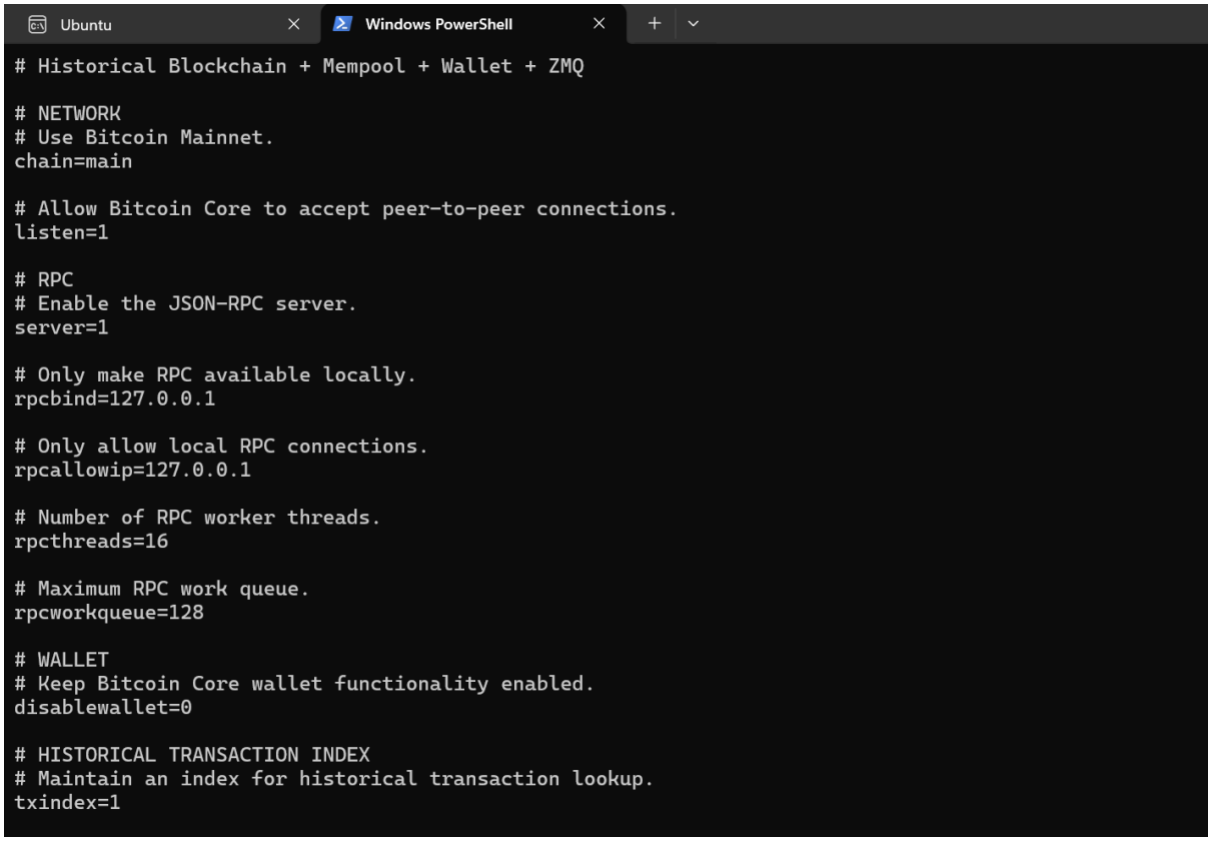
The configuration file should not be readable or writable by other normal users.

33. Check the Configuration

Run:

cat ~/.bitcoin/bitcoin.conf

©Joseph Aragbaiye Project, 2026



Why?

This allows us to confirm that the configuration was saved correctly.

34. Why Is the Wallet Enabled?

Our configuration contains:

disablewallet=0

Why?

We want Bitcoin Core to provide wallet functionality in addition to blockchain and mempool functionality.

Later we can use:

bitcoin-cli createwallet "main"

and:

bitcoin-cli -rpcwallet=main getnewaddress

The wallet is separate from the mempool.

35. Why Is txindex=1 **Enabled?**

Our configuration contains:

txindex=1

Why?

©Joseph Aragbaiye Project, 2026

The project requires historical transaction analysis.

The transaction index makes it possible for Bitcoin Core to efficiently locate historical transactions by transaction ID.

Historical transaction information can include:

- TXID
- Inputs
- Outputs
- Size
- Virtual size
- Weight
- Fee
- Block information

This information can later be used by the analytics system.

36. Why Are We Not Using prune=20000**?**

We are deliberately **not** adding:

prune=20000

Why?

Pruning saves disk space by removing older block data shortly after been synchronized. For my project, I used prune based on limited storage resources... and for reason, focused on real time anomaly detection. Further development; the important of keeping historical data whilst on a real time is highly recommendable.

For production level, therefore, the configuration should use:

txindex=1

and does not contain:

prune=

For this project, you are to remove the txindex=1. Replace it with prune = 100000

Important

Do not combine this historical-node design with a pruning configuration.

An unpruned node requires considerably more storage.

©Joseph Aragbaiye Project, 2026

37. Start Bitcoin Core Manually

Run:

bitcoind -datadir="$HOME/.bitcoin" -daemon

Expected:

Bitcoin Core starting

Mine has already been running … more reason you see 'Bitcoin core is probably already running'

```
admp@Joseph:~/bitcoin-install$ bitcoind -datadir="$HOME/.bitcoin" -daemon
Error: Cannot obtain a lock on directory /home/admp/.bitcoin. Bitcoin Core is probably already running.
admp@Joseph:~/bitcoin-install$
```

Why?

We are testing Bitcoin Core manually before creating the systemd service.

This makes it easier to identify configuration, permission, database, or installation problems.

38. Wait for Bitcoin Core RPC

Run:

bitcoin-cli -rpcwait getblockchaininfo

```
admp@Joseph:~/bitcoin-install$ bitcoin-cli -rpcwait getblockchaininfo
{
  "chain": "main",
  "blocks": 930052,
  "headers": 967337,
  "bestblockhash": "00000000000000000009c663b5298d84d772b032d31b68d6c9dbccb9786c774",
  "bits": "1701e605",
  "target": "00000000000000000001e6050000000000000000000000000000000000000000",
  "difficulty": 148258433855481.3,
  "time": 1767036006,
  "mediantime": 1767033932,
  "verificationprogress": 0.9152061623361076,
  "initialblockdownload": true,
  "chainwork": "0000000000000000000000000000000000000001026100013dfd8ac765f37c1c",
  "size_on_disk": 11371133392,
  "pruned": true,
  "pruneheight": 923882,
  "automatic_pruning": true,
  "prune_target_size": 20971520000,
  "warnings": [
  ]
}
admp@Joseph:~/bitcoin-install$
```

Why?

bitcoin-cli communicates with Bitcoin Core through RPC.

The -rpcwait option tells bitcoin-cli to wait until Bitcoin Core's RPC interface is ready.

39. Check Blockchain Synchronization

©Joseph Aragbaiye Project, 2026

Run:

bitcoin-cli getblockchaininfo | jq '{

  chain,

  blocks,

  headers,

  verificationprogress,

  initialblockdownload,

  pruned

}'

```
admp@Joseph:~/bitcoin-install$ bitcoin-cli getblockchaininfo | jq '{
    chain,
    blocks,
    headers,
    verificationprogress,
    initialblockdownload,
    pruned
}'
{
  "chain": "main",
  "blocks": 930118,
  "headers": 967337,
  "verificationprogress": 0.9153668407350073,
  "initialblockdownload": true,
  "pruned": true
}
admp@Joseph:~/bitcoin-install$
```

Why?

This tells us:

- Which Bitcoin network we are using
- How many blocks have been processed
- How many headers are known
- Approximate synchronization progress
- Whether initial synchronization is still happening
- Whether the node is pruned

During synchronization you may see:

initialblockdownload: true

That is normal.

©Joseph Aragbaiye Project, 2026

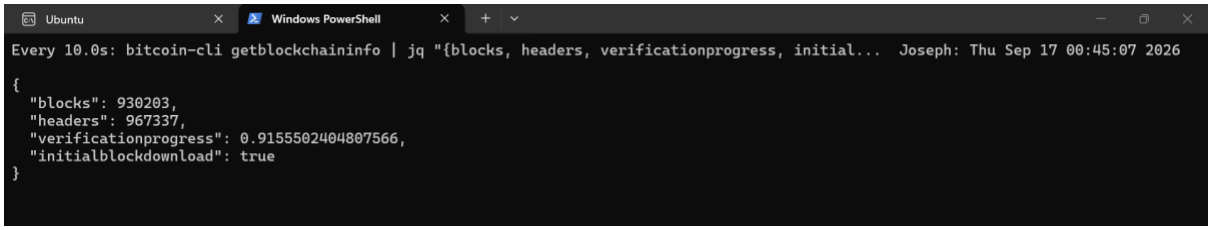
The final state should eventually be:

initialblockdownload: false

40. Monitor Synchronization

Run:

watch -n 10 'bitcoin-cli getblockchaininfo | jq "{blocks, headers, verificationprogress, initialblockdownload}"'



Why?

This automatically refreshes the synchronization information every 10 seconds.

Stop the display with:

Ctrl+C

This does **not** stop Bitcoin Core.

41. Check Bitcoin Network Connections

Run:

bitcoin-cli getnetworkinfo | jq '{

  connections,

  connections_in,

  connections_out,

  networkactive

}'

©Joseph Aragbaiye Project, 2026

```
admp@Joseph:~/bitcoin-install$ bitcoin-cli getnetworkinfo | jq '{
    connections,
    connections_in,
    connections_out,
    networkactive
}'
{
  "connections": 10,
  "connections_in": 0,
  "connections_out": 10,
  "networkactive": true
}
admp@Joseph:~/bitcoin-install$
```

Why?

Bitcoin Core needs connections to other Bitcoin nodes to exchange blocks and transactions.

You may see something such as:

connections: 8

connections_in: 0

connections_out: 8

networkactive: true

The exact number of connections will change over time.

Mine is presently :

connections: 10

connections_in: 0

connections_out: 10

networkactive: true

You can also do this below instead of the above:

42. Count the Connected Peers

Run:

bitcoin-cli getpeerinfo | jq 'length'

```
admp@Joseph:~/bitcoin-install$ bitcoin-cli getpeerinfo | jq 'length'
10
admp@Joseph:~/bitcoin-install$
```

Why?

This provides a simple count of the peers currently known to Bitcoin Core.

43. Check the Mempool

Run:

bitcoin-cli getmempoolinfo

©Joseph Aragbaiye Project, 2026

```
admp@Joseph:~/bitcoin-install$ bitcoin-cli getmempoolinfo
{
  "loaded": true,
  "size": 0,
  "bytes": 0,
  "usage": 0,
  "total_fee": 0.00000000,
  "maxmempool": 10000000000,
  "mempoolminfee": 0.00000100,
  "minrelaytxfee": 0.00000100,
  "incrementalrelayfee": 0.00000100,
  "unbroadcastcount": 0,
  "fullrbf": true,
  "permitbaremultisig": true,
  "maxdatacarriersize": 100000,
  "limitclustercount": 64,
  "limitclustersize": 101000,
  "optimal": true
}
admp@Joseph:~/bitcoin-install$
```

Why?

The mempool contains valid unconfirmed transactions currently known to your Bitcoin Core node.

You may see:

size: 0

That is not automatically an error.

It means your local node currently has no unconfirmed transactions in its mempool.

It does **not** mean that the entire Bitcoin network has no unconfirmed transactions.

The entire blockchain (bitcoin) network transaction has to fully synchronize from 2009 till date before you can start to stream mempool unconfirmed transaction on a real-time. You do need to configure or do anything, it happens on itself when the synchronization is done.

44. Display a Clean Mempool Summary

Run:

bitcoin-cli getmempoolinfo | jq '{

loaded,

size,

bytes,

usage,

total_fee,

mempoolminfee

©Joseph Aragbaiye Project, 2026

}'

```
admp@Joseph:~/bitcoin-install$ bitcoin-cli getmempoolinfo | jq '{
    loaded,
    size,
    bytes,
    usage,
    total_fee,
    mempoolminfee
}'
{
  "loaded": true,
  "size": 0,
  "bytes": 0,
  "usage": 0,
  "total_fee": 0E-8,
  "mempoolminfee": 0.00000100
}
admp@Joseph:~/bitcoin-install$
```

Why?

This gives us the most useful mempool information without displaying unnecessary fields.

45. Read Mempool Transactions

When the mempool contains transactions, run:

bitcoin-cli getrawmempool true | jq '.[] | {

txid,

size,

vsize,

weight,

fee_sat: (.fees.base * 100000000),

fee_rate_sat_vb: ((.fees.base * 100000000) / .vsize)

}'

Why?

This extracts the fields needed for transaction-fee analysis.

For each transaction, we can obtain:

TXID

size

vsize

weight

©Joseph Aragbaiye Project, 2026

fee

fee rate

46. Understand vsize

vsize means **virtual transaction size**.

It is measured in virtual bytes:

vB

Why?

Bitcoin transaction fee rates are normally expressed as:

satoshis per virtual byte

or:

sat/vB

For example:

Fee = 36,960 sat

vsize = 168 vB

The fee rate is:

36,960 / 168

which equals:

220 sat/vB

47. Check Historical Blockchain Data

For example, retrieve the hash of block 924800:

bitcoin-cli getblockhash 924800

```
admp@Joseph:~/bitcoin-install$ bitcoin-cli getblockhash 924800
0000000000000000000a8ada48f29edc61459739b712cba9a4905606c1afc17
admp@Joseph:~/bitcoin-install$
```

Why?

Bitcoin Core identifies blocks using block hashes.

We need the block hash before asking Bitcoin Core for the full block.

48. Retrieve Historical Block 924800

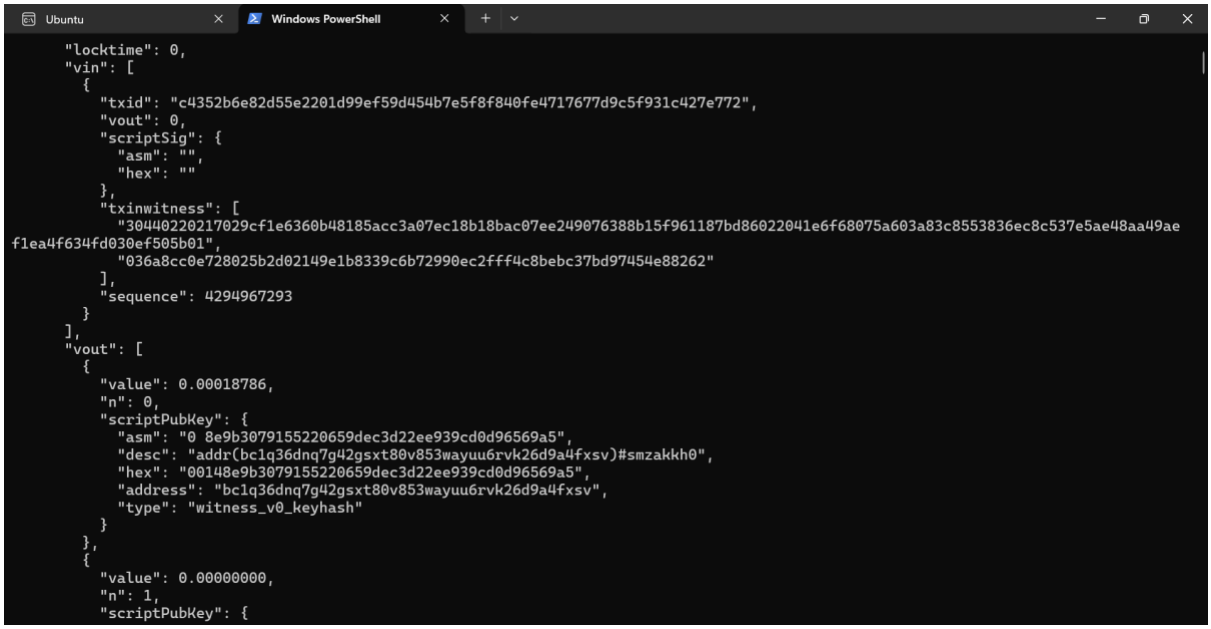
Run:

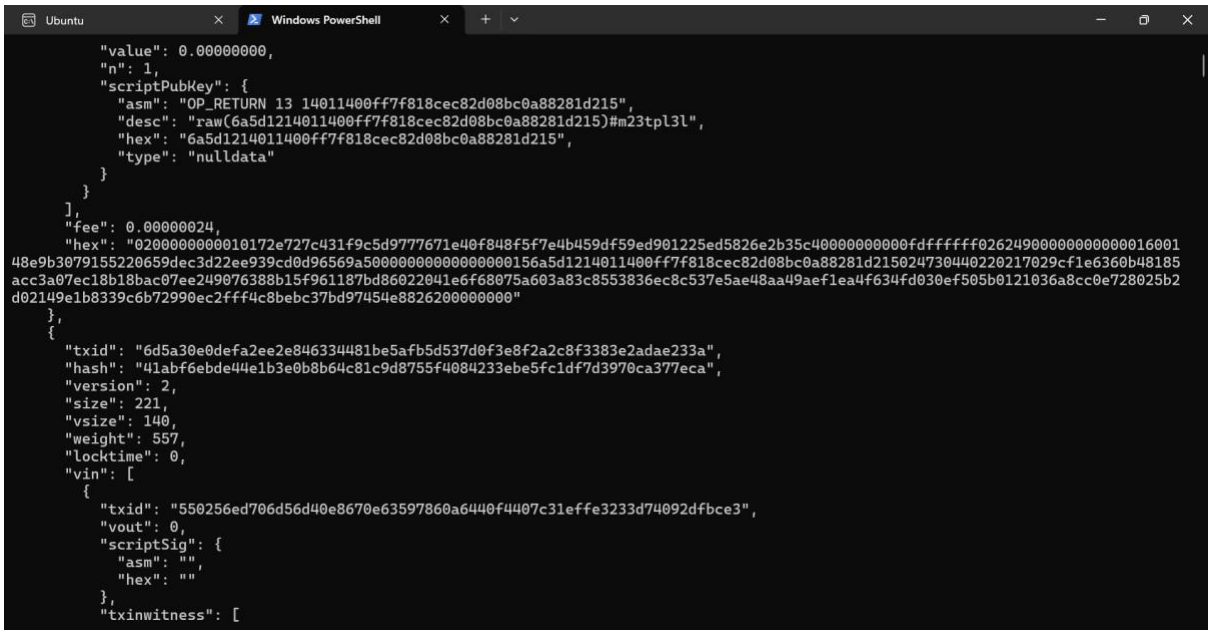
©Joseph Aragbaiye Project, 2026

bitcoin-cli getblock \

"$(bitcoin-cli getblockhash 924800)" \

2





Why?

The 2 asks Bitcoin Core to return the block with decoded transaction information.

This allows us to inspect the individual transactions inside the historical block.

49. Extract a Historical Transaction

Run:

bitcoin-cli getblock \

©Joseph Aragbaiye Project, 2026

```
  "$(bitcoin-cli getblockhash 924800)" \
  2 |
jq '.tx[1] | {
  txid,
  size,
  vsize,
  weight,
  fee
}'
```

You may see something similar to:

```
{
 "txid": "...",
 "size": 249,
 "vsize": 168,
 "weight": 669,
 "fee": 0.00036960
}
```

```
admp@Joseph:~/bitcoin-install$ bitcoin-cli getblock \
    "$(bitcoin-cli getblockhash 924800)" \
    2 |
jq '.tx[1] | {
    txid,
    size,
    vsize,
    weight,
    fee
}'
{
  "txid": "2d4fafde8d77f8ae380e58c9228e9af5f371bdbf177414a27b53a65f2ec297fc",
  "size": 249,
  "vsize": 168,
  "weight": 669,
  "fee": 0.00036960
}
admp@Joseph:~/bitcoin-install$
```

Why?

This demonstrates that historical confirmed transactions can provide:

- TXID
- Size
- Virtual size

©Joseph Aragbaiye Project, 2026

- Weight
- Fee

These are important fields for the future analytics system.

50. Calculate a Historical Transaction Fee Rate

Suppose the transaction has:

fee = 0.00036960 BTC

vsize = 168 vB

Convert BTC to satoshis:

0.00036960 × 100,000,000

Result:

36,960 sat

Then calculate:

36,960 / 168

Result:

220 sat/vB

Why?

This gives us a standardized way to compare transaction fee rates.

51. Fee Estimation

Run:

bitcoin-cli estimatesmartfee 6

```
admp@Joseph:~/bitcoin-install$ bitcoin-cli estimatesmartfee 6
{
  "errors": [
    "Insufficient data or no feerate found"
  ],
  "blocks": 0
}
admp@Joseph:~/bitcoin-install$
```

Why?

Bitcoin Core can use its available transaction history to estimate a fee rate associated with a desired confirmation target.

©Joseph Aragbaiye Project, 2026

During initial synchronization, you may see:

Insufficient data or no feerate found

This can happen because the node does not yet have sufficient usable information.

After synchronization, try again:

bitcoin-cli estimatesmartfee 6

52. Check RPC Security

Our configuration contains:

rpcbind=127.0.0.1

rpcallowip=127.0.0.1

Why?

This keeps the Bitcoin Core RPC interface restricted to the local machine.

The future Python collector can communicate with Bitcoin Core locally.

Do not unnecessarily expose Bitcoin Core RPC to the public Internet.

53. Check the RPC Cookie

Run:

ls -l ~/.bitcoin/.cookie

```
admp@Joseph:~/bitcoin-install$ ls -l ~/.bitcoin/.cookie
-rw------- 1 admp admp 75 Sep 16 20:11 /home/admp/.bitcoin/.cookie
admp@Joseph:~/bitcoin-install$
```

Why?

Bitcoin Core can use a cookie file for RPC authentication.

The cookie is an authentication credential.

Do not run this and share the contents:

cat ~/.bitcoin/.cookie

Never put the cookie in Git repositories, or messages or exposed to third-party platform.

54. Check the Bitcoin Core Debug Log

Run:

tail -100 ~/.bitcoin/debug.log

©Joseph Aragbaiye Project, 2026

```
admp@Joseph:~/bitcoin-install$ tail -100 ~/.bitcoin/debug.log
2026-09-17T00:14:26Z UpdateTip: new best=000000000000000000009960851da3e6211f3261cd888bc3ace35fd32b1b6f61 height=931064 version=0x200
32000 log2_work=96.024924 tx=1294180251 date='2026-01-05T22:53:23Z' progress=0.917475 cache=1186.1MiB(8779362txo)
2026-09-17T00:14:27Z UpdateTip: new best=000000000000000000017539fe9815003c2dc4faf62ad85f77f290de0b2863a6 height=931065 version=0x2a0
00000 log2_work=96.024936 tx=1294183506 date='2026-01-05T23:13:00Z' progress=0.917477 cache=1186.1MiB(8782533txo)
2026-09-17T00:14:29Z UpdateTip: new best=000000000000000000000dfe68333b2e9aad65a25c0a78c4c09d281ee059f7c21 height=931066 version=0x220
b6000 log2_work=96.024947 tx=1294187039 date='2026-01-05T23:37:53Z' progress=0.917480 cache=1186.1MiB(8786757txo)
2026-09-17T00:14:31Z UpdateTip: new best=000000000000000000000435278a81364333e28987a3951f5bceb42614a125cd4 height=931067 version=0x326
0c000 log2_work=96.024958 tx=1294189699 date='2026-01-05T23:42:21Z' progress=0.917482 cache=1186.1MiB(8789176txo)
2026-09-17T00:14:33Z UpdateTip: new best=000000000000000000010790dc8c8c022065380300f037d07dc12e94493dc758 height=931068 version=0x200
40000 log2_work=96.024970 tx=1294193520 date='2026-01-06T00:01:25Z' progress=0.917484 cache=1186.1MiB(8793173txo)
2026-09-17T00:14:33Z UpdateTip: new best=000000000000000000000132afc718aa2212a5035f2c37af8471f85b445d5f7d62 height=931069 version=0x2d3
a6000 log2_work=96.024981 tx=1294195673 date='2026-01-06T00:04:58Z' progress=0.917486 cache=1186.1MiB(8792135txo)
2026-09-17T00:14:34Z UpdateTip: new best=000000000000000000000f570f70f88a30875e693f1f079e480e2e151b179a5dd height=931070 version=0x299
9e000 log2_work=96.024993 tx=1294199957 date='2026-01-06T00:07:43Z' progress=0.917489 cache=1186.1MiB(8795562txo)
2026-09-17T00:14:34Z UpdateTip: new best=000000000000000000010e1c429ddf0a27b19a2c369a9ec8fb1d0ba409b4551b height=931071 version=0x200
28000 log2_work=96.025004 tx=1294201747 date='2026-01-06T00:07:49Z' progress=0.917490 cache=1186.1MiB(8797308txo)
2026-09-17T00:14:35Z UpdateTip: new best=000000000000000000012ae670b389373730019652ff581e762e575029bfe1f1 height=931072 version=0x200
00000 log2_work=96.025015 tx=1294205886 date='2026-01-06T00:12:51Z' progress=0.917493 cache=1186.1MiB(8800663txo)
2026-09-17T00:14:35Z UpdateTip: new best=000000000000000000000d241f72df816840fe8a23ea86cb5a48b831af02e3632 height=931073 version=0x361
3e000 log2_work=96.025027 tx=1294208426 date='2026-01-06T00:13:25Z' progress=0.917495 cache=1186.1MiB(8796416txo)
2026-09-17T00:14:35Z UpdateTip: new best=000000000000000000000169b1e903f65daa09b3c1e078d2f9d077bee4b6209ac6 height=931074 version=0x2ae
30000 log2_work=96.025038 tx=1294212512 date='2026-01-06T00:16:36Z' progress=0.917498 cache=1186.1MiB(8798884txo)
2026-09-17T00:14:36Z UpdateTip: new best=000000000000000000001e0dd58604a1916c7ea23ccd44cfb104236230614fba1 height=931075 version=0x2aa
80000 log2_work=96.025050 tx=1294215397 date='2026-01-06T00:18:36Z' progress=0.917500 cache=1186.1MiB(8800236txo)
2026-09-17T00:14:37Z UpdateTip: new best=000000000000000000001169fd43748154293 4a8f65b62ae05784e136f0bcda8e height=931076 version=0x294
1a000 log2_work=96.025061 tx=1294218827 date='2026-01-06T00:32:02Z' progress=0.917502 cache=1186.1MiB(8802854txo)
2026-09-17T00:14:37Z UpdateTip: new best=000000000000000000000ef0af1e2dbf1c89ce759a37a9f82c2cac2e007b641e8 height=931077 version=0x200
```

Why?

The debug log provides useful information about:

- ✓ Startup
- ✓ Synchronization
- ✓ Peer connections
- ✓ Configuration
- ✓ Database operations
- ✓ Wallet operations
- ✓ Errors

55. Watch the Debug Log Live

Run:

tail -f ~/.bitcoin/debug.log

```
admp@Joseph:~/bitcoin-install$ ls -l ~/.bitcoin/.cookie
-rw------- 1 admp admp 75 Sep 16 20:11 /home/admp/.bitcoin/.cookie
admp@Joseph:~/bitcoin-install$ tail -100 ~/.bitcoin/debug.log
2026-09-17T00:14:26Z UpdateTip: new best=000000000000000000009960851da3e6211f3261cd888bc3ace35fd32b1b6f61 height=931064 version=0x200
32000 log2_work=96.024924 tx=1294180251 date='2026-01-05T22:53:23Z' progress=0.917475 cache=1186.1MiB(8779362txo)
2026-09-17T00:14:27Z UpdateTip: new best=000000000000000000017539fe9815003c2dc4faf62ad85f77f290de0b2863a6 height=931065 version=0x2a0
00000 log2_work=96.024936 tx=1294183506 date='2026-01-05T23:13:00Z' progress=0.917477 cache=1186.1MiB(8782533txo)
2026-09-17T00:14:29Z UpdateTip: new best=000000000000000000000dfe68333b2e9aad65a25c0a78c4c09d281ee059f7c21 height=931066 version=0x220
b6000 log2_work=96.024947 tx=1294187039 date='2026-01-05T23:37:53Z' progress=0.917480 cache=1186.1MiB(8786757txo)
2026-09-17T00:14:31Z UpdateTip: new best=000000000000000000000435278a81364333e28987a3951f5bceb42614a125cd4 height=931067 version=0x326
0c000 log2_work=96.024958 tx=1294189699 date='2026-01-05T23:42:21Z' progress=0.917482 cache=1186.1MiB(8789176txo)
2026-09-17T00:14:33Z UpdateTip: new best=000000000000000000010790dc8c8c022065380300f037d07dc12e94493dc758 height=931068 version=0x200
40000 log2_work=96.024970 tx=1294193520 date='2026-01-06T00:01:25Z' progress=0.917484 cache=1186.1MiB(8793173txo)
2026-09-17T00:14:33Z UpdateTip: new best=000000000000000000000132afc718aa2212a5035f2c37af8471f85b445d5f7d62 height=931069 version=0x2d3
a6000 log2_work=96.024981 tx=1294195673 date='2026-01-06T00:04:58Z' progress=0.917486 cache=1186.1MiB(8792135txo)
2026-09-17T00:14:34Z UpdateTip: new best=000000000000000000000f570f70f88a30875e693f1f079e480e2e151b179a5dd height=931070 version=0x299
9e000 log2_work=96.024993 tx=1294199957 date='2026-01-06T00:07:43Z' progress=0.917489 cache=1186.1MiB(8795562txo)
2026-09-17T00:14:34Z UpdateTip: new best=000000000000000000010e1c429ddf0a27b19a2c369a9ec8fb1d0ba409b4551b height=931071 version=0x200
28000 log2_work=96.025004 tx=1294201747 date='2026-01-06T00:07:49Z' progress=0.917490 cache=1186.1MiB(8797308txo)
2026-09-17T00:14:35Z UpdateTip: new best=000000000000000000012ae670b389373730019652ff581e762e575029bfe1f1 height=931072 version=0x200
00000 log2_work=96.025015 tx=1294205886 date='2026-01-06T00:12:51Z' progress=0.917493 cache=1186.1MiB(8800663txo)
2026-09-17T00:14:35Z UpdateTip: new best=000000000000000000000d241f72df816840fe8a23ea86cb5a48b831af02e3632 height=931073 version=0x361
3e000 log2_work=96.025027 tx=1294208426 date='2026-01-06T00:13:25Z' progress=0.917495 cache=1186.1MiB(8796416txo)
2026-09-17T00:14:35Z UpdateTip: new best=000000000000000000000169b1e903f65daa09b3c1e078d2f9d077bee4b6209ac6 height=931074 version=0x2ae
```

Why?

This continuously displays new log entries.

©Joseph Aragbaiye Project, 2026

Stop watching with:

Ctrl+C

This only stops viewing the log.

It does not stop Bitcoin Core.

56. Check Bitcoin Core Ports

Run:

sudo ss -ltnp | grep -E '8332|8333|28332|28333'

```
admp@Joseph:~/bitcoin-install$ sudo ss -ltnp | grep -E '8332|8333|28332|28333'
[sudo] password for admp:
LISTEN 0      100          127.0.0.1:28332      0.0.0.0:*    users:(("bitcoind",pid=16863,fd=22))
LISTEN 0      100          127.0.0.1:28333      0.0.0.0:*    users:(("bitcoind",pid=16863,fd=20))
LISTEN 0      128          127.0.0.1:8332       0.0.0.0:*    users:(("bitcoind",pid=16863,fd=13))
LISTEN 0      4096           0.0.0.0:8333       0.0.0.0:*    users:(("bitcoind",pid=16863,fd=38))
LISTEN 0      128              [::1]:8332          [::]:*    users:(("bitcoind",pid=16863,fd=12))
LISTEN 0      4096              [::]:8333          [::]:*    users:(("bitcoind",pid=16863,fd=37))
admp@Joseph:~/bitcoin-install$
```

Why?

This checks whether Bitcoin Core is listening on the expected ports.

The ports are:

Port	Purpose
8332	Bitcoin Core RPC
8333	Bitcoin peer-to-peer network
28332	ZMQ raw transactions
28333	ZMQ raw blocks

RPC should remain local:

127.0.0.1:8332

57. Stop the Manual Bitcoin Core Instance

Run:

bitcoin-cli stop

```
admp@Joseph:~/bitcoin-install$ bitcoin-cli stop
Bitcoin Core stopping
admp@Joseph:~/bitcoin-install$
```

Why?

We have finished testing Bitcoin Core manually.

©Joseph Aragbaiye Project, 2026

We want to stop the manually started process before allowing systemd to manage it.

58. Confirm Bitcoin Core Has Stopped

Run:

pgrep -a bitcoind

Why?

This checks whether a bitcoind process is still running.

Normally there should be no running Bitcoin Core process after shutdown has completed.

59. Create the systemd Service

Run:

sudo nano /etc/systemd/system/bitcoind.service

Why?

This creates a Linux service definition for Bitcoin Core.

Instead of manually starting Bitcoin Core every time, systemd can manage it.

Paste:

Bitcoin Core systemd service

Bitcoin Core 31.1

Linux user: admp

[Unit]

Name displayed by systemd.

Description=Bitcoin Core daemon

Start after networking is available.

After=network-online.target

Request the network-online target.

Wants=network-online.target

©Joseph Aragbaiye Project, 2026

```
[Service]

# Bitcoin Core runs in the foreground.
Type=simple

# Run Bitcoin Core as the normal Linux user.
User=admp
Group=admp

# Start Bitcoin Core.
ExecStart=/usr/local/bin/bitcoind \
  -conf=/home/admp/.bitcoin/bitcoin.conf \
  -datadir=/home/admp/.bitcoin

# Shut Bitcoin Core down cleanly.
ExecStop=/usr/local/bin/bitcoin-cli \
  -datadir=/home/admp/.bitcoin \
  stop

# Restart Bitcoin Core if it unexpectedly exits.
Restart=on-failure

# Wait before restarting.
RestartSec=10

# Allow unlimited startup time.
TimeoutStartSec=infinity

# Give Bitcoin Core enough time to shut down.
```

©Joseph Aragbaiye Project, 2026

TimeoutStopSec=600

Prevent the service from gaining new privileges.

NoNewPrivileges=true

Give the service a private temporary directory.

PrivateTmp=true

[Install]

Make the service available during normal system startup.

WantedBy=multi-user.target

Save:

Ctrl+O

Press:

Enter

Exit:

Ctrl+X

60. Why Are We Using systemd?

Without systemd, we would need to manually run:

bitcoind -daemon

With systemd, we can use:

sudo systemctl start bitcoind

and:

sudo systemctl stop bitcoind

We can also check it with:

sudo systemctl status bitcoind

Why?

©Joseph Aragbaiye Project, 2026

This gives us a standard way to start, stop, monitor, and restart Bitcoin Core.

61. Reload systemd

Run:

sudo systemctl daemon-reload

Why?

We just created a new service file.

systemd needs to reload its configuration so it recognizes the new service.

62. Enable Bitcoin Core

Run:

sudo systemctl enable bitcoind

```
admp@Joseph:~/bitcoin-install$ sudo systemctl enable bitcoind
Created symlink /etc/systemd/system/multi-user.target.wants/bitcoind.service → /etc/systemd/system/bitcoind.service.
admp@Joseph:~/bitcoin-install$
```

Why?

This tells systemd that bitcoind is an enabled service.

63. Start Bitcoin Core Through systemd

Run:

sudo systemctl start bitcoind

Why?

This starts Bitcoin Core under systemd's control.

---

64. Check the Bitcoin Core Service

Run:

sudo systemctl status bitcoind --no-pager -l

You want to see:

Active: active (running)

©Joseph Aragbaiye Project, 2026

```
admp@Joseph:~/bitcoin-install$ sudo systemctl status bitcoind --no-pager -l
● bitcoind.service - Bitcoin Core daemon
     Loaded: loaded (/etc/systemd/system/bitcoind.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-09-17 01:26:28 BST; 23s ago
   Main PID: 43190 (bitcoind)
      Tasks: 29 (limit: 9356)
     Memory: 2.2G (peak: 2.2G)
        CPU: 11.320s
     CGroup: /system.slice/bitcoind.service
             └─43190 /usr/local/bin/bitcoind -conf=/home/admp/.bitcoin/bitcoin.conf -datadir=/home/admp/.bitcoin

Sep 17 01:26:31 Joseph bitcoind[43190]: 2026-09-17T00:26:31Z Checking all blk files are present...
Sep 17 01:26:31 Joseph bitcoind[43190]: 2026-09-17T00:26:31Z Loading block index db: Block files have previously been pruned
Sep 17 01:26:32 Joseph bitcoind[43190]: 2026-09-17T00:26:32Z Initializing chainstate Chainstate [ibd] @ height -1 (null)
Sep 17 01:26:32 Joseph bitcoind[43190]: 2026-09-17T00:26:32Z Opening LevelDB in /home/admp/.bitcoin/chainstate
Sep 17 01:26:33 Joseph bitcoind[43190]: 2026-09-17T00:26:33Z Opened LevelDB successfully
Sep 17 01:26:33 Joseph bitcoind[43190]: 2026-09-17T00:26:33Z Using obfuscation key for /home/admp/.bitcoin/chainstate: 91a4fb3037d166
ad
Sep 17 01:26:33 Joseph bitcoind[43190]: 2026-09-17T00:26:33Z Loaded best chain: hashBestChain=000000000000000000132af398485bcde1f87e
c839320de5be5a05ee87906fa height=931258 date=2026-01-07T09:56:58Z progress=0.917914
Sep 17 01:26:34 Joseph bitcoind[43190]: 2026-09-17T00:26:34Z init message: Verifying blocks…
Sep 17 01:26:34 Joseph bitcoind[43190]: 2026-09-17T00:26:34Z Verifying last 6 blocks at level 3
Sep 17 01:26:34 Joseph bitcoind[43190]: 2026-09-17T00:26:34Z Verification progress: 0%
admp@Joseph:~/bitcoin-install$
```

Why?

This confirms that systemd successfully started Bitcoin Core.

65. Check Whether the Service Is Enabled

Run:

systemctl is-enabled bitcoind

Expected:

enabled

Why?

This confirms that the service is enabled in systemd.

```
admp@Joseph:~/bitcoin-install$ systemctl is-enabled bitcoind
enabled
admp@Joseph:~/bitcoin-install$
```

66. Check Whether the Service Is Running

Run:

systemctl is-active bitcoind

Expected:

Active

```
admp@Joseph:~/bitcoin-install$ systemctl is-active bitcoind
active
admp@Joseph:~/bitcoin-install$
```

Why?

This provides a simple confirmation that Bitcoin Core is currently running.

67. Check the Running Bitcoin Process

©Joseph Aragbaiye Project, 2026

Run:

pgrep -a bitcoind

Why?

This gives us another way to confirm that the Bitcoin Core daemon is running.

68. Final Blockchain Check

Run:

```
bitcoin-cli getblockchaininfo | jq '{
  chain,
  blocks,
  headers,
  verificationprogress,
  initialblockdownload,
  pruned
}'
```

Why?

This gives us the final synchronization status.

For the completed unpruned node, we expect:

chain: main

initialblockdownload: false

pruned: false

The block height will continue increasing as new Bitcoin blocks are produced.

69. Final Network Check

Run:

```
bitcoin-cli getnetworkinfo | jq '{
  connections,
  connections_in,
  connections_out,
  networkactive
}'
```

Then:

©Joseph Aragbaiye Project, 2026

bitcoin-cli getpeerinfo | jq 'length'

Why?

These commands confirm that Bitcoin Core is connected to the Bitcoin peer-to-peer network.

70. Final Mempool Check

Run:

```
bitcoin-cli getmempoolinfo | jq '{
  loaded,
  size,
  bytes,
  usage,
  total_fee,
  mempoolminfee
}'
```

Why?

This confirms that the local Bitcoin Core mempool is loaded and shows its current state.

Remember:

size: 0

does not necessarily indicate a problem.

It simply means that your node currently has no unconfirmed transactions in its local mempool.

71. Final Wallet Check

Run:

```
bitcoin-cli listwallets
```

Expected:

```
[
 "main"
]
```

Then:

```
bitcoin-cli -rpcwallet=main getwalletinfo
```

Then:

©Joseph Aragbaiye Project, 2026

bitcoin-cli -rpcwallet=main getbalances

Why?

These commands confirm that the wallet exists, can be accessed, and can report its balances.

72. Final ZMQ Check

Run:

bitcoin-cli getzmqnotifications

Why?

This confirms that the ZMQ endpoints configured in bitcoin.conf are active.

You should see the configured endpoints for:

127.0.0.1:28332

127.0.0.1:28333

73. Final Service Check

Run:

systemctl is-active bitcoind

Expected:

active

Then:

systemctl is-enabled bitcoind

Expected:

enabled

Why?

These two commands confirm that Bitcoin Core is both running and configured as an enabled systemd service.

74. Complete Health Check

Once Bitcoin Core is running, use this command to quickly check the main components:

echo "===== BITCOIN CORE ====="

bitcoind --version | head -1

echo

©Joseph Aragbaiye Project, 2026

```
echo "===== BLOCKCHAIN ====="
bitcoin-cli getblockchaininfo | jq '{
  chain,
  blocks,
  headers,
  verificationprogress,
  initialblockdownload,
  pruned
}'

echo
echo "===== NETWORK ====="
bitcoin-cli getnetworkinfo | jq '{
  connections,
  connections_in,
  connections_out,
  networkactive
}'

echo
echo "===== MEMPOOL ====="
bitcoin-cli getmempoolinfo | jq '{
  loaded,
  size,
  bytes,
  usage,
  total_fee,
  mempoolminfee
}'

echo
```

©Joseph Aragbaiye Project, 2026

```
echo "===== WALLET ====="

bitcoin-cli listwallets


echo

echo "===== ZMQ ====="

bitcoin-cli getzmqnotifications


echo

echo "===== SYSTEMD ====="

systemctl is-active bitcoind
```

Why?

Instead of checking every component separately, this provides a quick overview of the Bitcoin Core installation.

75. Troubleshooting: bitcoind **Command Not Found**

Run:

```
which bitcoind
```

Then:

```
ls -l /usr/local/bin/bitcoind
```

Expected:

```
/usr/local/bin/bitcoind
```

Why?

This checks whether the Bitcoin Core executable was actually installed in the expected location.

76. Troubleshooting: bitcoin-cli **Cannot Connect**

Run:

```
sudo systemctl status bitcoind --no-pager -l
```

Then:

```
tail -100 ~/.bitcoin/debug.log
```

Why?

Bitcoin Core may still be starting, or it may have encountered a configuration or database problem.

The service status and debug log should provide more information.

©Joseph Aragbaiye Project, 2026

77. Troubleshooting: Checksum Failed

If this command:

sha256sum --ignore-missing --check SHA256SUMS

reports:

FAILED

do not install the archive.

Delete it:

rm -f "bitcoin-${VERSION}-${BTC_ARCH}.tar.gz"

Download it again:

curl -fLO "${BASE}/bitcoin-${VERSION}-${BTC_ARCH}.tar.gz"

Then verify again:

sha256sum --ignore-missing --check SHA256SUMS

Why?

We do not want to install an archive that does not match the published checksum.

78. Troubleshooting: No Peers

Run:

bitcoin-cli getnetworkinfo

Then:

bitcoin-cli getpeerinfo | jq 'length'

Check Internet connectivity:

ping -c 3 8.8.8.8

Check DNS:

getent hosts bitcoincore.org

Why?

These commands help determine whether the problem is Bitcoin Core itself, network connectivity, or DNS.

79. Troubleshooting: Mempool Is Empty

Run:

bitcoin-cli getmempoolinfo

©Joseph Aragbaiye Project, 2026

If you see:

size: 0

Why?

This means the local node currently has no unconfirmed transactions.

It does not mean that the entire Bitcoin network has no unconfirmed transactions.

80. Troubleshooting: Fee Estimation Is Unavailable

Run:

```
bitcoin-cli getblockchaininfo | jq '{
  blocks,
  headers,
  verificationprogress,
  initialblockdownload
}'
```

If you see:

initialblockdownload: true

Why?

The node is still synchronizing.

Allow synchronization to continue and try again later:

bitcoin-cli estimatesmartfee 6

81. Troubleshooting: Bitcoin Core Service Keeps Restarting

Run:

sudo journalctl -u bitcoind -n 100 --no-pager

Then:

tail -100 ~/.bitcoin/debug.log

Why?

These logs can reveal problems involving:

- Configuration
- Permissions
- Database files

©Joseph Aragbaiye Project, 2026

- Ports
- Missing files
- Startup errors

82. Historical Data vs. Mempool Data

It is important to understand that these are two different types of transaction data.

Historical Blockchain Data

Historical transactions are transactions that have already been included in blocks.

They can provide:

TXID

Block height

Block time

Size

vsize

Weight

Fee

Inputs

Outputs

Why?

This is the data we can use for historical analytics.

Live Mempool Data

Mempool transactions are currently unconfirmed transactions known to our node.

They can provide:

TXID

Size

vsize

Weight

Fee

Fee rate

Why?

©Joseph Aragbaiye Project, 2026

This is the data we can use for real-time transaction monitoring.

83. What Happens to a Mempool Transaction?

A transaction can initially be unconfirmed.

While it is unconfirmed, it may be visible in the local mempool.

After a miner includes it in a block, it becomes a confirmed blockchain transaction.

Why?

This distinction matters because our future analytics system needs to understand both:

- **Real-time unconfirmed transactions**
- **Confirmed historical transactions**

84. Final Bitcoin Core Configuration

The intended final configuration is:

Bitcoin Core 31.1

Bitcoin Mainnet

Ubuntu 24.04

WSL2

Wallet enabled

txindex=1

Mempool enabled

Unpruned blockchain

Pruned synchronization

RPC localhost only

ZMQ raw transactions

ZMQ raw blocks

systemd service

Why?

This gives the project a Bitcoin Core foundation capable of supplying both historical and real-time data.

85. Future Analytics System

Once Bitcoin Core has completed synchronization, the next stage can be built around:

©Joseph Aragbaiye Project, 2026

Bitcoin Core

Python Collector

Kafka

Database

Analytics

Dashboard

The Python application can use:

RPC

for historical queries and node information.

It can use:

ZMQ

for real-time transaction and block notifications.

Why?

This separates the Bitcoin node from the analytics layer.

Bitcoin Core remains responsible for Bitcoin network connectivity, validation, blockchain storage, mempool management, wallet functionality, RPC, and ZMQ notifications.

The Python/Kafka layer can then focus on collecting, processing, storing, and analysing the data.

©Joseph Aragbaiye Project, 2026

Apache Kafka 4.3.1 on WSL2

Step-by-Step Installation, Configuration, Topic Creation, Real-Time Bitcoin Streaming and ML.

Platform: Windows 11
Linux Environment: Ubuntu 24.04 on WSL2
Linux User: admp
Kafka Version: Apache Kafka 4.3.1
Java Version: OpenJDK 21
Kafka Mode: KRaft
Kafka Broker: localhost:9092
Kafka Topic: btc-tx
Bitcoin ZMQ: tcp://127.0.0.1:28332
Bitcoin RPC: http://127.0.0.1:8332
Streaming Program: /home/admp/industrial_stream.py, industrial_ml.py

1. What Are We Building?

We are installing **Apache Kafka 4.3.1** inside Ubuntu on WSL2 and connecting it to the Bitcoin Core node already running on the same Linux environment.

Bitcoin Core remains responsible for Bitcoin network connectivity, validation, mempool management, RPC and ZMQ. Kafka provides the streaming boundary between Bitcoin data collection and the later analytics/ML layer. This follows the architecture already established in the Bitcoin Core documentation.

2. Why Kafka?

Kafka is used to receive and buffer Bitcoin transaction events produced by industrial_stream.py.

Instead of connecting the machine-learning process directly to Bitcoin Core, Kafka separates the two processes.

This means: Bitcoin Core – Producer – Kafka - Consumer / ML

If the ML process temporarily stops, the acquisition layer does not need to be tightly coupled to it.

3. Enter Ubuntu WSL2

From Windows PowerShell:

wsl

You should enter Ubuntu and see something like:

admp@Joseph:~$

Why?

©Joseph Aragbaiye Project, 2026

Kafka, Java, Bitcoin Core and the Python streaming programs are all being run inside the Ubuntu WSL2 environment.

4. Confirm the Linux User

Run:

whoami

Expected:

admp

Why?

The Kafka installation paths in this guide use:

/home/admp/

If another Linux username is being used, the paths must be adjusted accordingly.

5. Update Ubuntu

Run:

sudo apt update

sudo apt upgrade -y

Why?

This refreshes Ubuntu's package information and applies available package updates before Java and Kafka are installed.

6. Install Required Utilities

Run:

```
sudo apt install -y \
  wget \
  curl \
  tar \
  python3 \
  python3-pip \
  python3-venv
```

Why?

These provide the tools required to download Kafka, extract the archive and prepare the Python streaming environment.

©Joseph Aragbaiye Project, 2026

7. Install Java 21

Run:

sudo apt install -y openjdk-21-jdk

Why?

Kafka 4.3.1 requires Java 17 or newer. Java 21 is used for this implementation.

8. Select Java 21

Run:

sudo update-alternatives --config java

Select:

/usr/lib/jvm/java-21-openjdk-amd64/bin/java

Then configure the Java compiler:

sudo update-alternatives --config javac

Select:

/usr/lib/jvm/java-21-openjdk-amd64/bin/javac

9. Verify Java

Run:

java -version

Expected:

openjdk version "21..."

Then:

javac -version

Expected:

javac 21...

©Joseph Aragbaiye Project, 2026

Why?

Kafka will fail with an UnsupportedClassVersionError if an older Java runtime such as Java 11 is selected.

10. Configure JAVA_HOME

Run:

nano ~/.bashrc

Add near the bottom:

Java 21

export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64

export PATH="$JAVA_HOME/bin:$PATH"

Save:

Ctrl+O

Enter

Ctrl+X

Reload:

source ~/.bashrc

Check:

echo "$JAVA_HOME"

Expected:

/usr/lib/jvm/java-21-openjdk-amd64

Why?

This makes new terminal sessions use Java 21 automatically instead of returning to an older Java installation.

11. Create the Kafka Download Directory

Run:

©Joseph Aragbaiye Project, 2026

mkdir -p ~/kafka-install

cd ~/kafka-install

Check:

Pwd

Expected:

/home/admp/kafka-install

Why?

This separates Kafka installation files from the actual Kafka program and its stored data.

12. Select Kafka 4.3.1

Run:

KAFKA_VERSION=4.3.1

SCALA_VERSION=2.13

Check:

echo "$KAFKA_VERSION"

Expected:

4.3.1

13. Download Kafka 4.3.1

Run:

wget https://dlcdn.apache.org/kafka/4.3.1/kafka_2.13-4.3.1.tgz

Check:

ls -lh kafka_2.13-4.3.1.tgz

Why?

©Joseph Aragbaiye Project, 2026

kafka_2.13-4.3.1.tgz is the Apache Kafka 4.3.1 binary distribution.

14. Check the Kafka Archive

Run:

tar -tzf kafka_2.13-4.3.1.tgz | head

You should see:

kafka_2.13-4.3.1/

kafka_2.13-4.3.1/LICENSE

kafka_2.13-4.3.1/NOTICE

kafka_2.13-4.3.1/bin/

...

Why?

This confirms that the downloaded archive can be read before it is extracted.

15. Create the Kafka Installation Directory

Run:

mkdir -p ~/kafka-4.3.1

Why?

Kafka is installed separately from project code and Kafka data.

The structure will eventually be:

/home/admp/

kafka-4.3.1/ Kafka software

kafka-data-4.3.1/ Kafka data

industrial_stream.py Bitcoin streaming producer

industrial_ml.py ML consumer

16. Extract Kafka

Run:

tar -xzf ~/kafka-install/kafka_2.13-4.3.1.tgz \

©Joseph Aragbaiye Project, 2026

-C ~/kafka-4.3.1 \

--strip-components=1

Enter Kafka:

cd ~/kafka-4.3.1

Check:

Ls

Expected directories include:

bin

config

libs

licenses

site-docs

LICENSE

NOTICE

17. Verify Kafka Version

Run:

bin/kafka-topics.sh –version

Expected:

4.3.1

Why?

This proves that the commands being executed belong to Kafka 4.3.1 rather than an older Kafka installation.

18. Set KAFKA_HOME

Run:

nano ~/.bashrc

©Joseph Aragbaiye Project, 2026

Add:

Apache Kafka 4.3.1

export KAFKA_HOME=$HOME/kafka-4.3.1

export PATH="$KAFKA_HOME/bin:$PATH"

Save and reload:

source ~/.bashrc

Check:

echo "$KAFKA_HOME"

Expected:

/home/admp/kafka-4.3.1

Check:

kafka-topics.sh –version

Expected:

4.3.1

19. Kafka 4.3.1 Uses KRaft

Kafka 4.3.1 does not require a ZooKeeper installation for this setup.

The server is initialized using **KRaft**.

Therefore, the configuration used is:

config/server.properties

and not a separate ZooKeeper server.

20. Create a Persistent Kafka Data Directory

Run:

mkdir -p ~/kafka-data-4.3.1

©Joseph Aragbaiye Project, 2026

Check:

ls -ld ~/kafka-data-4.3.1

Why?

Kafka requires a directory in which its topic partitions, metadata and log segments are stored.

Using:

/home/admp/kafka-data-4.3.1

is preferable to keeping important project Kafka data inside /tmp.

21. Configure Kafka Storage

Open:

nano ~/kafka-4.3.1/config/server.properties

Find:

log.dirs=

Set it to:

log.dirs=/home/admp/kafka-data-4.3.1

Save:

Ctrl+O

Enter

Ctrl+X

22. Check the Important Kafka Configuration

Run:

grep -E \

'^(process.roles|node.id|listeners|advertised.listeners|controller|log.dirs)' \

~/kafka-4.3.1/config/server.properties

©Joseph Aragbaiye Project, 2026

Why?

This checks the important single-node KRaft and storage settings before the Kafka data directory is initialized.

23. Generate the Kafka Cluster ID

Run:

cd ~/kafka-4.3.1

Then:

KAFKA_CLUSTER_ID="$(bin/kafka-storage.sh random-uuid)"

Display it:

echo "$KAFKA_CLUSTER_ID"

You should receive a unique identifier.

Why?

A KRaft Kafka cluster requires its own cluster ID.

24. Format Kafka Storage

Run:

```
bin/kafka-storage.sh format \
  --standalone \
  -t "$KAFKA_CLUSTER_ID" \
  -c config/server.properties
```

Why?

This initializes the Kafka KRaft metadata and storage directories.

Important

Run the formatting stage when creating the new Kafka storage.

Do **not** format the Kafka storage every time Kafka is started.

After the storage has been initialized, normal operation begins by starting the Kafka server.

©Joseph Aragbaiye Project, 2026

25. Start Kafka

Run:

cd ~/kafka-4.3.1

Then:

bin/kafka-server-start.sh config/server.properties

Leave this terminal open.

Why?

This starts the Kafka broker.

Kafka will listen for producers and consumers through:

localhost:9092

26. Check Kafka Port 9092

Open another Ubuntu terminal.

Run:

ss -ltnp | grep 9092

You should see Kafka listening on port:

9092

Why?

Port 9092 is the broker endpoint used by industrial_stream.py and industrial_ml.py.

27. Create the Bitcoin Kafka Topic

Enter Kafka:

cd ~/kafka-4.3.1

Create:

```
bin/kafka-topics.sh \
  --create \
  --topic btc-tx \
  --bootstrap-server localhost:9092 \
```

©Joseph Aragbaiye Project, 2026

--partitions 3 \

--replication-factor 1

Why?

btc-tx is the Kafka topic used for live Bitcoin transaction events.

The architecture becomes:

industrial_stream.py - industrial_ml.py

28. List Kafka Topics

Run:

bin/kafka-topics.sh \

--bootstrap-server localhost:9092 \

--list

Expected:

btc-tx

Why?

This confirms that the topic exists.

29. Describe the btc-tx Topic

Run:

bin/kafka-topics.sh \

--describe \

--topic btc-tx \

--bootstrap-server localhost:9092

The project configuration should show:

Topic: btc-tx

PartitionCount: 3

ReplicationFactor: 1

©Joseph Aragbaiye Project, 2026

and three partitions:

Partition: 0

Partition: 1

Partition: 2

Why?

This verifies the topic structure and confirms that each partition has an active leader.

30. Test Kafka Before Connecting Bitcoin Core

Start a consumer in one terminal:

cd ~/kafka-4.3.1

Run:

```
bin/kafka-console-consumer.sh \
  --topic btc-tx \
  --bootstrap-server localhost:9092
```

Leave it open.

31. Send a Manual Kafka Message

Open another terminal:

cd ~/kafka-4.3.1

Run:

```
echo '{"test":"kafka-ok","source":"manual"}' | \
bin/kafka-console-producer.sh \
  --topic btc-tx \
  --bootstrap-server localhost:9092
```

The consumer terminal should display:

{"test":"kafka-ok","source":"manual"}

Why?

This verifies the complete Kafka producer-to-consumer path before Bitcoin Core is introduced.

©Joseph Aragbaiye Project, 2026

At this point:

Test Producer - Kafka :9092 - btc-tx - Test Consumer

is working.

32. Check Bitcoin Core Before Streaming

Kafka is now ready.

The next stage is to connect it to the Bitcoin Core node.

Check Bitcoin Core:

bitcoin-cli getblockchaininfo

Why?

industrial_stream.py depends on the running Bitcoin Core node.

33. Check Bitcoin Core ZMQ

Run:

bitcoin-cli getzmqnotifications

The configuration should contain the raw transaction endpoint:

tcp://127.0.0.1:28332

Why?

ZMQ sends new Bitcoin raw transaction notifications to industrial_stream.py as they arrive.

34. Check ZMQ Port

Run:

ss -ltnp | grep 28332

Why?

This confirms the Bitcoin Core raw-transaction ZMQ endpoint is active.

35. Check Bitcoin RPC

Run:

©Joseph Aragbaiye Project, 2026

bitcoin-cli getblockchaininfo | head

Bitcoin RPC operates locally through:

127.0.0.1:8332

Why?

ZMQ tells the Python program that a transaction has arrived.

RPC provides additional information required to enrich that transaction.

Therefore:

ZMQ = live notification

RPC = transaction enrichment

36. Create the Python Streaming Environment

Run:

python3 -m venv ~/kafka_venv

Activate it:

source ~/kafka_venv/bin/activate

The prompt should change to something similar to:

(kafka_venv) admp@Joseph:~$

Why?

The virtual environment keeps the Python packages used by the streaming application separate from Ubuntu's system Python.

37. Upgrade pip

Run:

python -m pip install --upgrade pip

38. Install the Streaming Dependencies

Run:

python -m pip install \

©Joseph Aragbaiye Project, 2026

pyzmq \

requests \

kafka-python

Why?

The streaming program requires:

pyzmq

for Bitcoin Core ZMQ communication.

requests

for Bitcoin Core RPC requests.

kafka-python

for sending transaction records to Kafka.

39. Verify the Python Dependencies

Run:

python -c "import zmq; print('ZMQ OK:', zmq.__version__)"

Then:

python -c "import requests; print('Requests OK:', requests.__version__)"

Then:

python -c "import kafka; print('Kafka OK:', kafka.__version__)"

All three commands should run without an import error.

40. Create the Streaming Program Location

The working program is stored at:

/home/admp/industrial_stream.py

Check:

ls -lh ~/industrial_stream.py

Why?

The Python project does not need to be stored inside the Kafka installation directory.

Keep:

©Joseph Aragbaiye Project, 2026

/home/admp/kafka-4.3.1

for Kafka itself.

Keep:

/home/admp/industrial_stream.py

for project code.

41. Check the Streaming Program Syntax

Run:

source ~/kafka_venv/bin/activate

Then:

python -m py_compile ~/industrial_stream.py

Check:

echo $?

Expected:

0

Why?

A return code of 0 confirms that Python found no syntax error before the live streaming program is started.

42. Understand the Required Streaming Configuration

industrial_stream.py connects the following components:

Bitcoin Core ZMQ

tcp://127.0.0.1:28332

industrial_stream.py

Bitcoin RPC

http://127.0.0.1:8332

Kafka localhost:9092

btc-tx

The important configuration values are:

Kafka broker:

©Joseph Aragbaiye Project, 2026

localhost:9092

Kafka topic:

btc-tx

Bitcoin ZMQ:

tcp://127.0.0.1:28332

Bitcoin RPC:

http://127.0.0.1:8332

43. Configuration in industrial_stream.py

Run:

Nano industrial_stream.py

Paste this below in the text file:

```
import hashlib
import json
import queue
import subprocess
import threading
from collections import OrderedDict
from datetime import datetime, timezone
import zmq
from kafka import KafkaProducer

# Local setup used for the dissertation stream.
BTC_TOPIC = "btc-tx"
KAFKA = ["localhost:9092"]
RAWTX_ZMQ = "tcp://127.0.0.1:28332"
```

©Joseph Aragbaiye Project, 2026

```
# bitcoin-cli is the slow part here, not ZMQ reception. I use several workers so
# rawtx notifications can keep entering the queue while other rows are decoded.
WORKERS = 4
QUEUE_LIMIT = 50_000
SEEN_LIMIT = 50_000
RPC_TIMEOUT = 3
CHAIN_REFRESH = 30
PRINT_RECORDS = True


def encode_for_kafka(value):
   return json.dumps(value, separators=(",", ":")).encode("utf-8")


producer = KafkaProducer(
   bootstrap_servers=KAFKA,
   value_serializer=encode_for_kafka,
   acks=1,
   retries=5,
   linger_ms=10,
   batch_size=65_536,
   max_block_ms=5_000,
)


jobs = queue.Queue(maxsize=QUEUE_LIMIT)
seen_rawtx = OrderedDict()
stopping = threading.Event()


chain_lock = threading.Lock()
print_lock = threading.Lock()
chain_state = {"block_height": None, "blockchain_mediantime": None}
```

©Joseph Aragbaiye Project, 2026

```
def cli_json(args, timeout=RPC_TIMEOUT):
    result = subprocess.run(
        ["bitcoin-cli", *args],
        text=True,
        capture_output=True,
        timeout=timeout,
    )

    if result.returncode:
        message = result.stderr.strip() or result.stdout.strip() or "bitcoin-cli failed"
        message = message.replace("\n", " ")
        if len(message) > 250:
            message = message[:250] + "..."
        raise RuntimeError(message)

    text = result.stdout.strip()
    if not text:
        return None
    return json.loads(text)

def rawtx_key(raw_tx):
    # This is only the early ZMQ duplicate key. After decoding I keep the txid
    # returned by Bitcoin Core in the Kafka row.
    digest = hashlib.sha256(hashlib.sha256(raw_tx).digest()).digest()
    return digest[::-1].hex()



def update_chain_state():
    while not stopping.is_set():
        try:
```

©Joseph Aragbaiye Project, 2026

```
            info = cli_json(["getblockchaininfo"])
            if info:
                median = info.get("mediantime")
                median_iso = None
                if median is not None:
                    median_iso = datetime.fromtimestamp(median, timezone.utc).isoformat()

                with chain_lock:
                    chain_state["block_height"] = info.get("blocks")
                    chain_state["blockchain_mediantime"] = median_iso

        except Exception as exc:
            with print_lock:
                print(f"[chain] {type(exc).__name__}: {exc}", flush=True)

        stopping.wait(CHAIN_REFRESH)

def mempool_fee(txid):
    # The tx may already have left the mempool by the time a worker reaches this.
    # Missing fee therefore does not make the complete transaction unusable.
    try:
        entry = cli_json(["getmempoolentry", txid], timeout=2)
        if entry:
            return entry.get("fees", {}).get("base")
    except Exception:
        pass
    return None


def output_details(vout):
    receivers = []
```

©Joseph Aragbaiye Project, 2026

```
    total = 0.0

    for out in vout:
        btc = float(out.get("value", 0.0))
        script = out.get("scriptPubKey", {})

        address = script.get("address")
        if address is None:
            # Kept because older Core output can expose an addresses list.
            address = script.get("addresses") or "unknown"

        receivers.append({"address": address, "btc": btc})
        total += btc

    return receivers, round(total, 8)



def previous_txids(vin):
    result = []
    for item in vin:
        if "coinbase" in item:
            result.append("coinbase")
        elif item.get("txid"):
            result.append(item["txid"])
    return result



def enrich_tx(key, raw_tx):
    try:
        decoded = cli_json(["decoderawtransaction", raw_tx.hex()])
        if not decoded:
```

©Joseph Aragbaiye Project, 2026

```
    return None

vin = decoded.get("vin", [])
vout = decoded.get("vout", [])
receivers, total_btc = output_details(vout)

# For segwit transactions `hash` may differ from txid, so both are kept.
txid = decoded.get("txid") or key
wtxid = decoded.get("hash") or txid
now = datetime.now(timezone.utc)

with chain_lock:
    height = chain_state["block_height"]
    median_time = chain_state["blockchain_mediantime"]

row = {
    "txid": txid,
    "hash": wtxid,
    "version": decoded.get("version"),
    "size": decoded.get("size"),
    "vsize": decoded.get("vsize"),
    "weight": decoded.get("weight"),
    "locktime": decoded.get("locktime"),
    "input_txids": previous_txids(vin),
    "input_count": len(vin),
    "receivers": receivers,
    "output_count": len(vout),
    "total_output_btc": total_btc,
    "fee_btc": mempool_fee(txid),
    "year": now.strftime("%Y"),
    "month": now.strftime("%m"),
```

©Joseph Aragbaiye Project, 2026

```
            "time_recorded": now.strftime("%H:%M:%S"),
            "timestamp": now.isoformat(),
            "block_height": height,
            "blockchain_mediantime": median_time,
        }
        return row

    except subprocess.TimeoutExpired:
        with print_lock:
            print(
                json.dumps({"event": "rpc_timeout", "txid": key}, separators=(",", ":")),
                flush=True,
            )
    except Exception as exc:
        with print_lock:
            print(
                json.dumps(
                    {"event": "enrichment_error", "txid": key, "error": str(exc)},
                    separators=(",", ":"),
                ),
                flush=True,
            )
    return None


def delivered(_metadata, row):
    if PRINT_RECORDS:
        with print_lock:
            print(json.dumps(row, separators=(",", ":")), flush=True)

def delivery_failed(error):
    with print_lock:
        print(
```

©Joseph Aragbaiye Project, 2026

```
            json.dumps({"event": "kafka_error", "error": str(error)}, separators=(",", ":")),
            flush=True,
        )


def worker_loop():
    while not stopping.is_set() or not jobs.empty():
        try:
            key, raw_tx = jobs.get(timeout=0.5)
        except queue.Empty:
            continue


        try:
            row = enrich_tx(key, raw_tx)
            if row is None:
                continue


            try:
                pending = producer.send(BTC_TOPIC, row)
                pending.add_callback(lambda meta, data=row: delivered(meta, data))
                pending.add_errback(delivery_failed)
            except Exception as exc:
                # This catches failures raised directly by send(). Async delivery
                # failures still go through delivery_failed above.
                with print_lock:
                    print(
                        json.dumps(
                            {
                                "event": "kafka_send_error",
                                "txid": row.get("txid"),
                                "error": str(exc),
                            },
                            separators=(",", ":"),
```

©Joseph Aragbaiye Project, 2026

```
                ),
                flush=True,
            )
        finally:
            jobs.task_done()


def first_time_seen(key):
    if key in seen_rawtx:
        return False

    seen_rawtx[key] = None
    if len(seen_rawtx) > SEEN_LIMIT:
        seen_rawtx.popitem(last=False)
    return True

def main():
    context = zmq.Context()
    socket = context.socket(zmq.SUB)
    socket.setsockopt(zmq.RCVHWM, 100_000)
    socket.connect(RAWTX_ZMQ)
    socket.setsockopt_string(zmq.SUBSCRIBE, "rawtx")
    socket.RCVTIMEO = 1000

    chain_thread = threading.Thread(
        target=update_chain_state,
        name="chain-state",
        daemon=True,
    )
    chain_thread.start()

    worker_threads = []
```

©Joseph Aragbaiye Project, 2026

```
for number in range(WORKERS):
    thread = threading.Thread(
        target=worker_loop,
        name=f"rpc-worker-{number + 1}",
        daemon=True,
    )
    thread.start()
    worker_threads.append(thread)

print(
    f"Bitcoin rawtx stream -> Kafka | {RAWTX_ZMQ} | {KAFKA[0]} | "
    f"topic={BTC_TOPIC} | workers={WORKERS}",
    flush=True,
)

try:
    while True:
        try:
            _topic, raw_tx, _sequence = socket.recv_multipart()
            key = rawtx_key(raw_tx)

            if not first_time_seen(key):
                continue

            try:
                jobs.put((key, raw_tx), timeout=1)
            except queue.Full:
                # It never entered the worker queue, so do not leave it marked
                # as processed. A later ZMQ announcement can then try again.
                seen_rawtx.pop(key, None)
                with print_lock:
```

©Joseph Aragbaiye Project, 2026

```
                print(
                    json.dumps(
                        {
                            "event": "queue_full",
                            "txid": key,
                            "queue_size": jobs.qsize(),
                        },
                        separators=(",", ":"),
                    ),
                    flush=True,
                )

            except zmq.Again:
                continue

    except KeyboardInterrupt:
        print("\nStopping Bitcoin Kafka stream...", flush=True)

    finally:
        stopping.set()
        jobs.join()

        for thread in worker_threads:
            thread.join(timeout=2)

        try:
            producer.flush(timeout=10)
        finally:
            producer.close(timeout=10)
            socket.close(linger=0)
            context.term()
```

©Joseph Aragbaiye Project, 2026

```
    print("Bitcoin Kafka stream stopped.", flush=True)


if __name__ == "__main__":
  main()
```

hold down ctrl + x

when the prompt comes up... press y + enter.

Why?

This configures the producer to connect to the same Bitcoin Core and Kafka services that has been configured.

44. Check All Required Ports Together

Run:

ss -ltnp | grep -E '8332|9092|28332'

The important ports are:

Port	Service
8332	Bitcoin Core RPC
28332	Bitcoin Core ZMQ raw transactions
9092	Apache Kafka broker

Why?

All three are required for the complete real-time transaction streaming path.

45. Start industrial_stream.py

Make sure Kafka is already running.

Activate the Python environment:

source ~/kafka_venv/bin/activate

©Joseph Aragbaiye Project, 2026

Move to the home directory:

cd ~

Run:

python -u industrial_stream.py

Why -u?

The -u option makes Python output appear immediately in the terminal.

This is useful for observing live transaction records and errors.

46. What industrial_stream.py Does

Once running, the program performs this sequence:

1. Connects to Bitcoin Core ZMQ
2. Receives a live raw transaction
3. Decodes/extracts transaction information
4. Uses Bitcoin Core RPC where additional
   transaction information is required
5. Builds a structured transaction record
6. Sends the record to Kafka
7. Kafka stores it in btc-tx

The resulting transaction records can contain information such as:

txid

hash

version

size

vsize

weight

input_txids

input_count

receivers

output_count

©Joseph Aragbaiye Project, 2026

total_output_btc

fee_btc

timestamp

block_height

47. Prevent Excessive Bitcoin RPC Requests

For this implementation, avoid using very high worker concurrency.

If the program supports the worker environment setting, start conservatively:

BTC_STREAM_WORKERS=4 python -u ~/industrial_stream.py

Why?

Very high RPC concurrency can cause Bitcoin Core to report:

Work queue depth exceeded

The producer should therefore process RPC enrichment at a controlled rate.

48. Verify That industrial_stream.py Is Running

Open another terminal:

pgrep -af industrial_stream.py

You should see a running Python process.

Why?

This confirms the Bitcoin-to-Kafka producer is active.

49. Watch the Live Bitcoin Transactions in Kafka

Open another terminal.

Make sure Java 21 is active:

export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64

export PATH="$JAVA_HOME/bin:$PATH"

Enter Kafka:

cd ~/kafka-4.3.1

Start the consumer:

©Joseph Aragbaiye Project, 2026

```
bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic btc-tx
```

Expected

As Bitcoin transactions arrive, JSON transaction records should begin appearing.

Why?

This proves that industrial_stream.py is successfully writing Bitcoin transactions into Kafka.

50. Completed Streaming Connection

The system is now connected as:

BITCOIN NETWORK - Bitcoin Core - ZMQ :28332 - RPC :8332 - industrial_stream.py

51. Check the Topic During Live Streaming

Run:

```
cd ~/kafka-4.3.1
```

Then:

```
bin/kafka-topics.sh \
  --describe \
  --topic btc-tx \
  --bootstrap-server localhost:9092
```

The three partitions should remain available.

52. Check Kafka Consumer Groups

Run:

```
bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --list
```

Why?

When industrial_ml.py is connected using a Kafka consumer group, the group can be monitored from Kafka.

©Joseph Aragbaiye Project, 2026

53. Normal Project Startup Order

Once installation and configuration are complete, **do not reinstall or reformat Kafka each time**.

The normal startup procedure is now much shorter.

Terminal 1 : Bitcoin Core

If Bitcoin Core is managed by systemd:

sudo systemctl start bitcoind

Check:

systemctl is-active bitcoind

Expected:

active

Check ZMQ:

bitcoin-cli getzmqnotifications

Terminal 2 : Kafka

Run:

export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64

export PATH="$JAVA_HOME/bin:$PATH"

cd ~/kafka-4.3.1

bin/kafka-server-start.sh config/server.properties

Leave it running.

Terminal 3 : Bitcoin Streaming Producer

Run:

source ~/kafka_venv/bin/activate

Then:

cd ~

python -u industrial_stream.py

Leave it running.

Terminal 4 : Machine-Learning Consumer

©Joseph Aragbaiye Project, 2026

Run:

source ~/kafka_venv/bin/activate

Then:

cd ~

python -u industrial_ml.py

Leave it running.

54. Quick Full Health Check

Run:

```
echo "===== BITCOIN CORE ====="
systemctl is-active bitcoind

echo
echo "===== ZMQ ====="
bitcoin-cli getzmqnotifications

echo
echo "===== KAFKA ====="
ss -ltnp | grep 9092

echo
echo "===== TOPIC ====="
~/kafka-4.3.1/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --describe \
  --topic btc-tx

echo
echo "===== STREAMING PRODUCER ====="
pgrep -af industrial_stream.py
```

©Joseph Aragbaiye Project, 2026

echo

echo "===== ML CONSUMER ====="

pgrep -af industrial_ml.py

Why?

This provides a quick check of the complete pipeline without testing each component separately.

Common issues likely face and solution to them:

55. Troubleshooting : Kafka Does Not Start

Check Java:

java -version

Then:

echo "$JAVA_HOME"

Expected:

openjdk version "21..."

and:

/usr/lib/jvm/java-21-openjdk-amd64

If required:

export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64

export PATH="$JAVA_HOME/bin:$PATH"

56. Troubleshooting: UnsupportedClassVersionError

If Kafka reports:

UnsupportedClassVersionError

run:

java -version

Why?

This normally means Kafka is being started using an older Java runtime.

Reset:

export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64

©Joseph Aragbaiye Project, 2026

export PATH="$JAVA_HOME/bin:$PATH"

Then retry Kafka.

57. Troubleshooting: Kafka Port 9092 Is Not Listening

Run:

ss -ltnp | grep 9092

If nothing is returned, start Kafka:

cd ~/kafka-4.3.1

bin/kafka-server-start.sh config/server.properties

58. Troubleshooting : btc-tx Does Not Exist

Check:

cd ~/kafka-4.3.1

Then:

```
bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --list
```

If btc-tx is missing:

```
bin/kafka-topics.sh \
  --create \
  --topic btc-tx \
  --bootstrap-server localhost:9092 \
  --partitions 3 \
  --replication-factor 1
```

59. Troubleshooting : Python Cannot Import zmq

If:

ModuleNotFoundError: No module named 'zmq'

activate the correct environment:

source ~/kafka_venv/bin/activate

Then:

©Joseph Aragbaiye Project, 2026

python -m pip install pyzmq

60. Troubleshooting : Python Cannot Import Kafka

If:

ModuleNotFoundError: No module named 'kafka'

run:

source ~/kafka_venv/bin/activate

Then:

python -m pip install kafka-python

61. Troubleshooting: Work Queue Depth Exceeded

If industrial_stream.py reports:

Work queue depth exceeded

reduce producer RPC concurrency.

For example:

BTC_STREAM_WORKERS=4 python -u ~/industrial_stream.py

Also check the Bitcoin Core RPC configuration:

```
grep -E '^(rpcthreads|rpcworkqueue)' \
  ~/.bitcoin/bitcoin.conf
```

The project configuration uses controlled RPC capacity rather than allowing an excessive number of concurrent enrichment requests.

62. Troubleshooting : No Transactions Appear

Check Bitcoin Core:

bitcoin-cli getblockchaininfo

Check ZMQ:

bitcoin-cli getzmqnotifications

Check the ZMQ port:

ss -ltnp | grep 28332

Check Kafka:

ss -ltnp | grep 9092

Check the producer:

©Joseph Aragbaiye Project, 2026

pgrep -af industrial_stream.py

Why?

The complete flow requires all four components:

Bitcoin Core

+

ZMQ

+

industrial_stream.py

+

Kafka

If one is unavailable, the end-to-end stream will not operate correctly.

MODELS CONFIGURATION:

Install all libraries to be used by our models....

Step 1. Go to your home folder

```
cd ~
```

Why: This takes you to your normal Linux home folder. It gives us a simple place to create the project folder.

Step 2. Create the project folder

```
mkdir -p bitcoin-fraud
cd bitcoin-fraud
```

Why: This folder will hold the Python machine learning code, the virtual environment, the requirements file, and other project files.

What to expect: Your terminal should now show that you are inside the bitcoin-fraud folder.

Step 3. Install Python tools

```
sudo apt update
sudo apt install -y python3 python3-pip python3-venv
```

Why? Python runs the machine learning program. pip installs Python packages. venv creates a separate Python environment for this project.

©Joseph Aragbaiye Project, 2026

Step 4. Check the Python version

```
python3 --version
```

Why: This confirms that Python is installed.

What to expect: You should see a Python version number. Python 3.11 or Python 3.12 is suitable for this setup.

Step 5. Create the Python environment

```
python3 -m venv kafka_venv
```

Why: The virtual environment keeps the project packages separate from the main Ubuntu Python installation.

Step 6. Activate the Python environment

```
source kafka_venv/bin/activate
```

Why: The machine learning packages must be installed inside this environment.

What to expect: The beginning of the terminal line should show (kafka_venv).

Step 7. Update pip

```
python -m pip install --upgrade pip
```

Why: A current version of pip reduces package installation problems.

Step 8. Know what does not need to be installed

The following items are already part of Python. Do not install them with pip.

```
hashlib
json
math
os
smtplib
sqlite3
threading
time
collections
datetime
email
typing
```

Why: Python already provides these modules. They can be imported directly in the program.

Step 9. Install NumPy

```
pip install numpy
```

Why: NumPy is used for numbers, arrays, and calculations in the machine learning code.

©Joseph Aragbaiye Project, 2026

Step 10. Install scikit learn

pip install scikit-learn

Why: This package provides MiniBatchKMeans and IsolationForest, which are used by the project.

Step 11. Install the Python Kafka library

pip install kafka-python

Why: The Python program needs this package to read messages from Kafka. It provides KafkaConsumer.

Step 12. Install River

pip install river

Why: River provides tools for streaming machine learning. It is used for online anomaly detection and drift detection.

Step 13. Check Java before CapyMOA

java -version

Why: CapyMOA uses Java. This check confirms that Java is available.

What to expect: You should see Java 21 if Java 21 was selected earlier.

Step 14. Set JAVA_HOME

readlink -f "$(which java)"

export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64

echo $JAVA_HOME

Why: JAVA_HOME tells CapyMOA where Java is installed.

What to expect: The last command should show /usr/lib/jvm/java-21-openjdk-amd64. If your Java path is different, use the path shown by the first command.

Step 15. Install CapyMOA

pip install capymoa

Why: CapyMOA provides the OnlineIsolationForest model used by the machine learning code.

Step 16. Install SHAP

pip install shap

Why: SHAP helps explain which transaction features had an effect on a model result.

Step 17. Simple installation command

If you prefer to install the main external packages together, use this command while the virtual environment is active.

©Joseph Aragbaiye Project, 2026

pip install numpy scikit-learn kafka-python river capymoa shap

Why: This installs the main packages used by the machine learning program in one command.

Step 18. Test the Kafka Python package

python -c "from kafka import KafkaConsumer; print('KafkaConsumer OK')"

Why: This checks that Python can use KafkaConsumer.

What to expect: The terminal should print KafkaConsumer OK.

Step 19. Test River

python -c "from river import anomaly, drift; print('River OK')"

Why: This checks that the online anomaly and drift tools can be imported.

What to expect: The terminal should print River OK.

Step 20. Test scikit learn

python -c "from sklearn.cluster import MiniBatchKMeans; from sklearn.ensemble import IsolationForest; print('Scikit learn OK')"

Why: This checks that the two scikit learn models can be imported.

What to expect: The terminal should print Scikit learn OK.

Step 21. Test CapyMOA

python -c "from capymoa.anomaly import OnlineIsolationForest; print('CapyMOA OK')"

Why: This checks that the online isolation model is ready.

What to expect: The terminal should print CapyMOA OK.

Step 22. Test SHAP

python -c "import shap; print('SHAP OK')"

Why: This checks that SHAP is installed.

What to expect: The terminal should print SHAP OK.

Step 23. Test all imports together

Run this final test after all packages have been installed.

python -c "import numpy, shap; from kafka import KafkaConsumer; from river import anomaly, drift; from sklearn.cluster import MiniBatchKMeans; from sklearn.ensemble import IsolationForest; from capymoa.anomaly import OnlineIsolationForest; print('All main packages are working')"

Why: This checks the imports used at the top of the machine learning program in one test.

©Joseph Aragbaiye Project, 2026

Step 24. Save the package list

```
pip freeze > requirements.txt
```

Why: This saves the installed package names and versions. It makes the project easier to rebuild on another machine.

```
cat requirements.txt
```

What to expect: You should see a list of installed Python packages and their versions.

Step 25. Start the environment again later

When you close the terminal, the virtual environment stops being active. Use these commands the next time you want to run the machine learning program.

```
cd ~/bitcoin-fraud
source kafka_venv/bin/activate
```

Why: This returns to the project folder and activates the same Python environment again.

Step 26. Keep the programs simple and separate

Keep the Kafka installation in its own folder. Keep the Python project in the bitcoin-fraud folder.

```
/home/admp/kafka
/home/admp/bitcoin-fraud
```

The streaming producer can be saved as industry_streaming.py. The machine learning consumer can be saved as machine_learning.py.

Step 27. What each Python program does

industry_streaming.py: Reads live Bitcoin transaction information and sends it to the Kafka topic called btc-tx.

Industrial_ml.py: Reads transactions from the btc-tx topic, runs the machine learning models, checks for unusual behaviour, and saves the result.

Step 28. Connect the machine learning program to Kafka

The machine learning program uses KafkaConsumer to listen to the btc-tx topic.

```
from kafka import KafkaConsumer

consumer = KafkaConsumer(
  "btc-tx",
  bootstrap_servers=["localhost:9092"]
)
```

Why: btc-tx is the topic containing the Bitcoin transactions. localhost:9092 is the local Kafka server.

Now:

Open your nano file:

admp@Joseph:~$ nano industrial_ml.py

©Joseph Aragbaiye Project, 2026

Paste all the codes below in the nano file:

```
"""
Real-time Bitcoin mempool anomaly detector.

Decoded transactions arrive through Kafka (``btc-tx``).  Each unique row is
scored by Half-Space Trees, CapyMOA Online Isolation Forest and a rolling
scikit-learn Isolation Forest.  A transaction is treated as a detection only
when at least two of the three models agree.  ADWIN watches feature drift, the
controller adjusts operating sensitivity, and completed verdicts are stored in
SQLite for Grafana and optional email alerts.

The scoring order is deliberate: score first, then learn.  This prevents the
current transaction from training an online detector before that same detector
has judged it.
"""
import hashlib
import json
import math
import os
import smtplib
import sqlite3
import threading
import time
from collections import deque
from datetime import datetime, timezone
from email.mime.text import MIMEText
from typing import Any, Iterable

import numpy as np
import shap
from capymoa.anomaly import OnlineIsolationForest as CapyOnlineIsolationForest
```

©Joseph Aragbaiye Project, 2026

```
from capymoa.instance import Instance
from capymoa.stream import Schema
from kafka import KafkaConsumer
from river import anomaly, drift
from sklearn.cluster import MiniBatchKMeans
from sklearn.ensemble import IsolationForest

try:
    from dotenv import load_dotenv

    load_dotenv()
except ImportError:
    # A .env file is optional. Normal environment variables still work.
    pass

# Runtime settings for the Bitcoin stream

def read_int_setting(name: str, default: int) -> int:
    """Read an integer override used by this detector, otherwise keep the project default."""

    return int(os.getenv(name, str(default)))

def read_float_setting(name: str, default: float) -> float:
    """Read a decimal detector setting from the environment."""

    return float(os.getenv(name, str(default)))

# Kafka and SQLite
DB_FILE = os.getenv("BTC_ANALYTICS_DB", "bitcoin_analytics.db")
KAFKA_TOPIC = os.getenv("BTC_KAFKA_TOPIC", "btc-tx")
KAFKA_SERVERS = os.getenv(
```

©Joseph Aragbaiye Project, 2026

```
    "BTC_KAFKA_BOOTSTRAP_SERVERS",
    "localhost:9092",
).split(",")
KAFKA_GROUP_ID = os.getenv(
    "BTC_KAFKA_GROUP_ID",
    "bitcoin-anomaly-capymoa-v1",
)

# Five separate transaction features are used by every global model.
# Inputs and outputs are deliberately separate so consolidation and fan-out
# transactions do not collapse into the same combined complexity value.
FEATURE_NAMES = [
    "amount",
    "vsize",
    "input_count",
    "output_count",
    "io_ratio",
]

RANDOM_SEED = read_int_setting("BTC_RANDOM_SEED", 42)

# Operational review rule. This does not prove fraud.
POLICY_BTC_THRESHOLD = read_float_setting("BTC_POLICY_THRESHOLD", 50.0)

# Warm-up and history sizes
# The first 400 unique rows train/calibrate the models. Transaction 401 is the
# first transaction eligible for voting. At least two models must flag before
# the transaction is treated as a confirmed anomaly detection.
GLOBAL_WARMUP_SIZE = read_int_setting("BTC_GLOBAL_WARMUP_SIZE", 400)
INITIAL_BATCH_TRAINING_SIZE = GLOBAL_WARMUP_SIZE
MIN_SCORE_HISTORY = GLOBAL_WARMUP_SIZE
```

©Joseph Aragbaiye Project, 2026

```
ONLINE_IF_MIN_SCORE_HISTORY = max(1, GLOBAL_WARMUP_SIZE - 1)
HST_SCORE_HISTORY_SIZE = read_int_setting("BTC_HST_SCORE_HISTORY_SIZE", 1_000)
ONLINE_IF_SCORE_HISTORY_SIZE = read_int_setting(
    "BTC_ONLINE_IF_SCORE_HISTORY_SIZE",
    1_000,
)
LATENCY_HISTORY_SIZE = read_int_setting("BTC_LATENCY_HISTORY_SIZE", 500)
DUPLICATE_CACHE_SIZE = read_int_setting("BTC_DUPLICATE_CACHE_SIZE", 50_000)
SQLITE_TIMEOUT_SECONDS = read_float_setting("BTC_SQLITE_TIMEOUT_SECONDS", 15.0)
SMTP_TIMEOUT_SECONDS = read_float_setting("BTC_SMTP_TIMEOUT_SECONDS", 10.0)
STRUCTURE_MIN_COUNT = read_int_setting("BTC_STRUCTURE_MIN_COUNT", 10)
STRUCTURE_DOMINANCE = read_float_setting("BTC_STRUCTURE_DOMINANCE", 3.0)
COINJOIN_MIN_COUNT = read_int_setting("BTC_COINJOIN_MIN_COUNT", 5)
COINJOIN_RATIO_LOW = read_float_setting("BTC_COINJOIN_RATIO_LOW", 0.5)
COINJOIN_RATIO_HIGH = read_float_setting("BTC_COINJOIN_RATIO_HIGH", 2.0)
THREE_MODEL_CONFIDENCE_BONUS = read_float_setting(
    "BTC_THREE_MODEL_CONFIDENCE_BONUS",
    0.10,
)
TWO_MODEL_CONFIDENCE_BONUS = read_float_setting(
    "BTC_TWO_MODEL_CONFIDENCE_BONUS",
    0.05,
)
HST_SCORE_ROUND_DECIMALS = read_int_setting("BTC_HST_SCORE_ROUND_DECIMALS", 12)

# HST
HST_WINDOW_SIZE = read_int_setting("BTC_HST_WINDOW_SIZE", 250)
HST_TREES = read_int_setting("BTC_HST_TREES", 30)
HST_HEIGHT = read_int_setting("BTC_HST_HEIGHT", 10)
HST_HEALTH_WINDOW = read_int_setting("BTC_HST_HEALTH_WINDOW", 100)
HST_MIN_HEALTH_SCORES = read_int_setting("BTC_HST_MIN_HEALTH_SCORES", 50)
```

©Joseph Aragbaiye Project, 2026

```
HST_FLATLINE_SPREAD = read_float_setting("BTC_HST_FLATLINE_SPREAD", 1e-12)
HST_FLATLINE_UNIQUE = read_int_setting("BTC_HST_FLATLINE_UNIQUE", 2)

# CapyMOA Online Isolation Forest
ONLINE_IF_WARMUP_SIZE = GLOBAL_WARMUP_SIZE
ONLINE_IF_WINDOW_SIZE = read_int_setting("BTC_ONLINE_IF_WINDOW_SIZE", 2_048)
ONLINE_IF_TREES = read_int_setting("BTC_ONLINE_IF_TREES", 32)
ONLINE_IF_MAX_LEAF_SAMPLES = read_int_setting(
  "BTC_ONLINE_IF_MAX_LEAF_SAMPLES",
  32,
)
ONLINE_IF_GROWTH_CRITERION = os.getenv(
  "BTC_ONLINE_IF_GROWTH_CRITERION",
  "adaptive",
)
ONLINE_IF_SUBSAMPLE = read_float_setting("BTC_ONLINE_IF_SUBSAMPLE", 1.0)
ONLINE_IF_BRANCHING_FACTOR = read_int_setting(
  "BTC_ONLINE_IF_BRANCHING_FACTOR",
  2,
)
ONLINE_IF_MODEL_NAME = "CAPYMOA_ONLINE_IFOREST_ICML_2024"

# Batch Isolation Forest
BATCH_IF_WINDOW = read_int_setting("BTC_BATCH_IF_WINDOW", 1_000)
BATCH_RETRAIN_INTERVAL = read_int_setting("BTC_BATCH_RETRAIN_INTERVAL", 1_000)
BATCH_IF_TREES = read_int_setting("BTC_BATCH_IF_TREES", 200)
BATCH_IF_CONTAMINATION = read_float_setting("BTC_BATCH_IF_CONTAMINATION", 0.02)

# Adaptive model controller
# The system reviews recent behaviour every 100 transactions after warm-up.
# It changes sensitivity only when the stream provides evidence that a change
```

©Joseph Aragbaiye Project, 2026

```
# is needed. The two-model confirmation rule is never relaxed.
ADAPTIVE_HISTORY_SIZE = read_int_setting("BTC_ADAPTIVE_HISTORY_SIZE", 500)
ADAPTIVE_REVIEW_INTERVAL = read_int_setting("BTC_ADAPTIVE_REVIEW_INTERVAL", 100)
ADAPTIVE_MIN_SAMPLES = read_int_setting("BTC_ADAPTIVE_MIN_SAMPLES", 100)

# Sensitivity modes. Lower percentile = more sensitive; higher = stricter.
SENSITIVE_ANOMALY_PERCENTILE = read_float_setting(
    "BTC_SENSITIVE_ANOMALY_PERCENTILE",
    0.90,
)
STABLE_ANOMALY_PERCENTILE = read_float_setting(
    "BTC_STABLE_ANOMALY_PERCENTILE",
    0.95,
)
DRIFT_ANOMALY_PERCENTILE = read_float_setting(
    "BTC_DRIFT_ANOMALY_PERCENTILE",
    0.93,
)
STRICT_ANOMALY_PERCENTILE = read_float_setting(
    "BTC_STRICT_ANOMALY_PERCENTILE",
    0.98,
)

# Evidence needed before changing sensitivity.
HIGH_RISK_UNCONFIRMED_TRIGGER = read_float_setting(
    "BTC_HIGH_RISK_UNCONFIRMED_TRIGGER",
    0.05,
)
SINGLE_MODEL_DISAGREEMENT_TRIGGER = read_float_setting(
    "BTC_SINGLE_MODEL_DISAGREEMENT_TRIGGER",
    0.15,
)
```

©Joseph Aragbaiye Project, 2026

```
HIGH_CONFIRMED_DETECTION_TRIGGER = read_float_setting(
    "BTC_HIGH_CONFIRMED_DETECTION_TRIGGER",
    0.08,
)

# Drift mode lasts for this many transactions after ADWIN detects a change.
DRIFT_ADAPTATION_TRANSACTIONS = read_int_setting(
    "BTC_DRIFT_ADAPTATION_TRANSACTIONS",
    300,
)

# K-Means behaviour groups
KMEANS_BATCH_SIZE = read_int_setting("BTC_KMEANS_BATCH_SIZE", 32)
KMEANS_CLUSTERS = read_int_setting("BTC_KMEANS_CLUSTERS", 4)

# Risk thresholds attached to each adaptive mode.
SENSITIVE_MEDIUM_RISK = read_float_setting("BTC_SENSITIVE_MEDIUM_RISK", 0.65)
SENSITIVE_CRITICAL_RISK = read_float_setting("BTC_SENSITIVE_CRITICAL_RISK", 0.85)
STABLE_MEDIUM_RISK = read_float_setting("BTC_STABLE_MEDIUM_RISK", 0.72)
STABLE_CRITICAL_RISK = read_float_setting("BTC_STABLE_CRITICAL_RISK", 0.88)
DRIFT_MEDIUM_RISK = read_float_setting("BTC_DRIFT_MEDIUM_RISK", 0.70)
DRIFT_CRITICAL_RISK = read_float_setting("BTC_DRIFT_CRITICAL_RISK", 0.90)
STRICT_MEDIUM_RISK = read_float_setting("BTC_STRICT_MEDIUM_RISK", 0.78)
STRICT_CRITICAL_RISK = read_float_setting("BTC_STRICT_CRITICAL_RISK", 0.94)

# Equal weights are the starting point. The controller can later make small
# reliability-based changes, but no weight can create a detection by itself.
MODEL_WEIGHT = 1.0 / 3.0
MIN_MODEL_WEIGHT = read_float_setting("BTC_MIN_MODEL_WEIGHT", 0.20)
MAX_MODEL_WEIGHT = read_float_setting("BTC_MAX_MODEL_WEIGHT", 0.50)
```

©Joseph Aragbaiye Project, 2026

```
# Print and email a summary after each block of 1,000 transactions.
MODEL_COMPARISON_INTERVAL = read_int_setting("BTC_MODEL_COMPARISON_INTERVAL", 1_000)


# Email settings
EMAIL_ALERTS_ENABLED = os.getenv("BTC_EMAIL_ALERTS_ENABLED", "true").lower() == "true"
EMAIL_COMPARISON_REPORTS_ENABLED = (
  os.getenv("BTC_EMAIL_COMPARISON_REPORTS_ENABLED", "true").lower()
  == "true"
)
EMAIL_SENDER = os.getenv("BTC_EMAIL_SENDER", "aragbaiyejoseph@gmail.com")
EMAIL_PASSWORD = os.getenv("BTC_EMAIL_APP_PASSWORD", "")
EMAIL_RECEIVER = os.getenv("BTC_EMAIL_RECEIVER", "josephoaam@gmail.com")
SMTP_HOST = os.getenv("BTC_SMTP_HOST", "smtp.gmail.com")
SMTP_PORT = read_int_setting("BTC_SMTP_PORT", 587)
SMTP_USE_TLS = os.getenv("BTC_SMTP_USE_TLS", "true").lower() == "true"
EMAIL_ALERT_VERDICTS = {
  "CRITICAL",
  "MEDIUM_RISK",
  "LESS_CRITICAL",
}


# Small helpers used while processing Bitcoin transactions

def clean_bitcoin_metric(value: Any, default: float = 0.0) -> float:
  """Clean a transaction metric before it is passed to the anomaly models."""

  try:
    number = float(value)
  except (TypeError, ValueError):
    return default

  if not math.isfinite(number):
```

©Joseph Aragbaiye Project, 2026

```
        return default
    return max(0.0, number)


def rolling_score_percentile(reference: Iterable[float], value: float | None) -> float:
    """Place the current anomaly score inside its recent Bitcoin score history."""

    values = np.asarray(list(reference), dtype=np.float64)

    if value is None or values.size == 0 or not np.isfinite(value):
        return 0.0

    lower = np.count_nonzero(values < value)
    equal = np.count_nonzero(values == value)
    return float((lower + 0.5 * equal) / values.size)


def rolling_mean(values: deque[float]) -> float | None:
    """Average a rolling metric only after at least one value has been recorded."""

    if not values:
        return None

    return float(np.mean(values))


def display_metric(value: float | None, decimals: int = 4) -> str:
    """Keep terminal output readable while a model is still warming up."""

    if value is None:
        return "Calibrating"

    return f"{value:.{decimals}f}"
```

©Joseph Aragbaiye Project, 2026

```
# Clock fields are excluded so the same blockchain row cannot evade the
# second duplicate check merely because it was received at another time.
VOLATILE_FINGERPRINT_FIELDS = {
    "timestamp",
    "time_recorded",
    "year",
    "month",
    "processed_at",
    "sequence",
}


def transaction_fingerprint(transaction: dict[str, Any]) -> str:
    """Fingerprint the stable transaction fields so replayed Kafka rows are not learned twice."""

    stable_row = {
        key: value
        for key, value in transaction.items()
        if key not in VOLATILE_FINGERPRINT_FIELDS
    }
    canonical = json.dumps(
        stable_row,
        sort_keys=True,
        separators=(",", ":"),
        ensure_ascii=False,
        default=str,
    )
    return hashlib.sha256(canonical.encode("utf-8")).hexdigest()



class TransactionDuplicateGuard:
    """Keep a bounded memory of Bitcoin rows already seen by this process."""
```

©Joseph Aragbaiye Project, 2026

```
    def __init__(self, capacity: int = DUPLICATE_CACHE_SIZE) -> None:
        self.capacity = capacity
        self.txids: set[str] = set()
        self.fingerprints: set[str] = set()
        self.txid_order: deque[str] = deque()
        self.fingerprint_order: deque[str] = deque()

    def duplicate_reason(self, txid: str, fingerprint: str) -> str | None:
        if txid in self.txids:
            return "MEMORY_TXID_DUPLICATE"
        if fingerprint in self.fingerprints:
            return "MEMORY_ROW_DUPLICATE"
        return None

    def remember(self, txid: str, fingerprint: str) -> None:
        self.txids.add(txid)
        self.fingerprints.add(fingerprint)
        self.txid_order.append(txid)
        self.fingerprint_order.append(fingerprint)

        while len(self.txid_order) > self.capacity:
            self.txids.discard(self.txid_order.popleft())
        while len(self.fingerprint_order) > self.capacity:
            self.fingerprints.discard(self.fingerprint_order.popleft())

# Email notifications for noteworthy verdicts

def missing_email_settings() -> list[str]:
    """Report the alert settings that are missing before opening an SMTP connection."""

    errors: list[str] = []
```

©Joseph Aragbaiye Project, 2026

```
    if not EMAIL_SENDER:
        errors.append("EMAIL_SENDER is empty.")

    if not EMAIL_PASSWORD:
        errors.append("BTC_EMAIL_APP_PASSWORD is empty.")

    if not EMAIL_RECEIVER:
        errors.append("EMAIL_RECEIVER is empty.")

    return errors


def send_alert_email(subject: str, body: str) -> bool:
    """Send one detector notification and report whether SMTP accepted it."""

    if not EMAIL_ALERTS_ENABLED:
        return False

    errors = missing_email_settings()
    if errors:
        print("[!] Email skipped:\n    - " + "\n    - ".join(errors))
        return False

    message = MIMEText(body, "plain", "utf-8")
    message["Subject"] = subject
    message["From"] = EMAIL_SENDER
    message["To"] = EMAIL_RECEIVER

    smtp: smtplib.SMTP | None = None
```

©Joseph Aragbaiye Project, 2026

```
    try:
        smtp = smtplib.SMTP(SMTP_HOST, SMTP_PORT, timeout=SMTP_TIMEOUT_SECONDS)
        smtp.ehlo()

        if SMTP_USE_TLS:
            smtp.starttls()
            smtp.ehlo()

        smtp.login(EMAIL_SENDER, EMAIL_PASSWORD)
        smtp.sendmail(EMAIL_SENDER, [EMAIL_RECEIVER], message.as_string())
        print("EMAIL ALERT SENT")
        return True

    except (OSError, smtplib.SMTPException) as error:
        print("EMAIL ERROR:", error)
        return False

    finally:
        if smtp is not None:
            try:
                smtp.quit()
            except (OSError, smtplib.SMTPException):
                pass


def email_transaction_alert(result: dict[str, Any]) -> bool:
    """Email only the verdicts configured as operationally important."""

    verdict = str(result["verdict"]).upper()
    if verdict not in EMAIL_ALERT_VERDICTS:
        return False
```

©Joseph Aragbaiye Project, 2026

```
    cluster = result["cluster_id"]
    cluster_text = str(cluster) if cluster is not None else "Calibrating"

    body = f"""
BTC TRANSACTION ALERT

Transaction ID: {result['txid']}
Verdict: {verdict}
Detection Flag: {result['detection_flag']}
Risk Score: {result['risk_score']:.4f}
Consensus: {result['consensus']}
Anomaly Votes: {result['anomaly_votes']}
Normal Votes: {result['normal_votes']}

HST Score: {display_metric(result['hst_score'], decimals=12)}
HST Percentile: {result['hst_percentile']:.4f}
HST Flag: {result['hst_flag']}
HST Health: {result['hst_health']}
HST Learning: {result['hst_learning_state']}

Online IF Model: {result['online_if_model']}
Online IF Score: {display_metric(result['online_if_score'])}
Online IF Percentile: {result['online_if_percentile']:.4f}
Online IF Flag: {result['online_if_flag']}

Batch IF Score: {display_metric(result['if_score'])}
Batch IF Percentile: {result['if_percentile']:.4f}
Batch IF Flag: {result['if_flag']}
Batch IF Retraining: {result['batch_if_retraining']}
```

©Joseph Aragbaiye Project, 2026

```
Cluster: {cluster_text}

Behaviour: {result['profile']}

Drift: {result['drift']}

Adaptation: {result['adaptation_mode']}


Explanation: {result['shap_reason']}

""".strip()


    return send_alert_email("BTC Transaction Alert", body)



def email_model_report(report: str, end_row: int) -> bool:

    """Send the periodic comparison report produced after each 1,000-row block."""


    if not EMAIL_COMPARISON_REPORTS_ENABLED:

        return False


    start_row = max(1, end_row - MODEL_COMPARISON_INTERVAL + 1)

    subject = f"BTC model statistics: transactions {start_row}-{end_row}"

    return send_alert_email(subject, report)



# SQLite result store consumed by Grafana


# Each item is: database column, SQLite type, result-dictionary key.

DATABASE_FIELDS: tuple[tuple[str, str, str], ...] = (

    ("txid", "TEXT PRIMARY KEY", "txid"),

    ("transaction_fingerprint", "TEXT UNIQUE", "transaction_fingerprint"),

    ("timestamp", "TEXT", "timestamp"),

    ("verdict", "TEXT", "verdict"),

    ("detection_flag", "INTEGER", "detection_flag"),
```

©Joseph Aragbaiye Project, 2026

("risk_score", "REAL", "risk_score"),

("hst_score", "REAL", "hst_score"),

("hst_percentile", "REAL", "hst_percentile"),

("hst_threshold", "REAL", "hst_threshold"),

("hst_flag", "INTEGER", "hst_flag"),

("hst_available", "INTEGER", "hst_available"),

("hst_health", "TEXT", "hst_health"),

("hst_error", "TEXT", "hst_error"),

("online_if_score", "REAL", "online_if_score"),

("online_if_percentile", "REAL", "online_if_percentile"),

("online_if_flag", "INTEGER", "online_if_flag"),

("online_if_available", "INTEGER", "online_if_available"),

("online_if_model", "TEXT", "online_if_model"),

("online_if_health", "TEXT", "online_if_health"),

("online_if_error", "TEXT", "online_if_error"),

("if_score", "REAL", "if_score"),

("if_percentile", "REAL", "if_percentile"),

("if_flag", "INTEGER", "if_flag"),

("batch_if_available", "INTEGER", "batch_if_available"),

("batch_if_retraining", "INTEGER", "batch_if_retraining"),

("network_drift", "TEXT", "drift"),

("adaptation_mode", "TEXT", "adaptation_mode"),

("adaptive_reason", "TEXT", "adaptive_reason"),

("anomaly_cutoff", "REAL", "anomaly_cutoff"),

("model_change_count", "INTEGER", "model_change_count"),

("confirmed_detection_rate", "REAL", "confirmed_detection_rate"),

("single_model_rate", "REAL", "single_model_rate"),

("high_risk_unconfirmed_rate", "REAL", "high_risk_unconfirmed_rate"),

("consensus", "TEXT", "consensus"),

("agreement_pattern", "TEXT", "agreement_pattern"),

("flagged_models", "TEXT", "flagged_models"),

©Joseph Aragbaiye Project, 2026

```
    ("normal_models", "TEXT", "normal_models"),
    ("anomaly_votes", "INTEGER", "anomaly_votes"),
    ("normal_votes", "INTEGER", "normal_votes"),
    ("available_models", "INTEGER", "available_models"),
    ("policy_review", "INTEGER", "policy_review"),
    ("hst_weight", "REAL", "hst_weight"),
    ("online_if_weight", "REAL", "online_if_weight"),
    ("if_weight", "REAL", "if_weight"),
    ("critical_threshold", "REAL", "critical_threshold"),
    ("medium_threshold", "REAL", "medium_threshold"),
    ("if_window", "INTEGER", "if_window"),
    ("retrain_interval", "INTEGER", "retrain_interval"),
    ("behavior_profile", "TEXT", "profile"),
    ("classify_transaction_shape", "TEXT", "classify_transaction_shape"),
    ("shap_reason", "TEXT", "shap_reason"),
    ("hst_latency_ms", "REAL", "hst_latency_ms"),
    ("online_if_latency_ms", "REAL", "online_if_latency_ms"),
    ("if_latency_ms", "REAL", "if_latency_ms"),
    ("faster_model", "TEXT", "faster_model"),
    ("latency_ratio", "REAL", "latency_ratio"),
    ("amount_btc", "REAL", "amount_btc"),
    ("vsize_raw", "REAL", "vsize_raw"),
    ("input_count_raw", "REAL", "input_count_raw"),
    ("output_count_raw", "REAL", "output_count_raw"),
    ("output_input_ratio", "REAL", "output_input_ratio"),
    ("complexity_raw", "REAL", "complexity_raw"),
)

BOOLEAN_DATABASE_COLUMNS = {
    "detection_flag",
    "hst_flag",
```

©Joseph Aragbaiye Project, 2026

```
    "hst_available",
    "online_if_flag",
    "online_if_available",
    "if_flag",
    "batch_if_available",
    "batch_if_retraining",
    "policy_review",
}



def create_verdict_table(connection: sqlite3.Connection, table_name: str) -> None:
    """Create the transaction_verdicts schema used by the Grafana dashboard."""

    definitions = ",\n    ".join(
        f"{column_name} {column_type}"
        for column_name, column_type, _ in DATABASE_FIELDS
    )

    connection.execute(
        f"""
        CREATE TABLE {table_name} (
            {definitions}
        )
        """
    )



def prepare_results_database() -> None:
    """
    Create the result table.
```

©Joseph Aragbaiye Project, 2026

```
When an older table contains a different schema, preserve all compatible
columns and rebuild the table with the current supported fields.
"""

with sqlite3.connect(DB_FILE, timeout=SQLITE_TIMEOUT_SECONDS) as connection:
    table_exists = connection.execute(
        """
        SELECT 1
        FROM sqlite_master
        WHERE type = 'table' AND name = 'transaction_verdicts'
        """
    ).fetchone()

    if table_exists is None:
        create_verdict_table(connection, "transaction_verdicts")
        return

    old_columns = [
        row[1]
        for row in connection.execute(
            "PRAGMA table_info(transaction_verdicts)"
        )
    ]
    new_columns = [column for column, _, _ in DATABASE_FIELDS]

    if old_columns == new_columns:
        return

    temporary_table = "transaction_verdicts_new"
    connection.execute(f"DROP TABLE IF EXISTS {temporary_table}")
    create_verdict_table(connection, temporary_table)
```

©Joseph Aragbaiye Project, 2026

```
    compatible_columns = [
        column for column in new_columns if column in old_columns
    ]

    if "txid" in compatible_columns:
        column_text = ", ".join(compatible_columns)
        connection.execute(
            f"""
            INSERT OR IGNORE INTO {temporary_table} ({column_text})
            SELECT {column_text}
            FROM transaction_verdicts
            """
        )

    connection.execute("DROP TABLE transaction_verdicts")
    connection.execute(
        f"ALTER TABLE {temporary_table} RENAME TO transaction_verdicts"
    )



def txid_already_saved(txid: str) -> bool:
    """Check persistent storage before allowing a txid into the models again."""

    if not txid or txid == "UNKNOWN":
        return False

    with sqlite3.connect(DB_FILE, timeout=SQLITE_TIMEOUT_SECONDS) as connection:
        row = connection.execute(
            "SELECT 1 FROM transaction_verdicts WHERE txid = ? LIMIT 1",
            (txid,),
```

©Joseph Aragbaiye Project, 2026

```
        ).fetchone()

    return row is not None


def transaction_body_already_saved(fingerprint: str) -> bool:
    """Catch duplicate transaction bodies even when the incoming txid check is not enough."""

    if not fingerprint:
        return False

    with sqlite3.connect(DB_FILE, timeout=SQLITE_TIMEOUT_SECONDS) as connection:
        row = connection.execute(
            """
            SELECT 1
            FROM transaction_verdicts
            WHERE transaction_fingerprint = ?
            LIMIT 1
            """,
            (fingerprint,),
        ).fetchone()

    return row is not None


def save_transaction_result(result: dict[str, Any]) -> bool:
    """Persist one completed detector verdict; duplicates are rejected by SQLite."""

    columns: list[str] = []
    values: list[Any] = []
```

©Joseph Aragbaiye Project, 2026

```
    for column_name, _, result_key in DATABASE_FIELDS:
        value = result[result_key]

        if column_name in BOOLEAN_DATABASE_COLUMNS:
            value = int(bool(value))

        columns.append(column_name)
        values.append(value)

    column_text = ", ".join(columns)
    placeholders = ", ".join("?" for _ in columns)

    with sqlite3.connect(DB_FILE, timeout=SQLITE_TIMEOUT_SECONDS) as connection:
        try:
            connection.execute(
                f"""
                INSERT INTO transaction_verdicts ({column_text})
                VALUES ({placeholders})
                """,
                values,
            )
        except sqlite3.IntegrityError:
            return False

    return True


# CapyMOA Online Isolation Forest adapter


class BitcoinOnlineIsolationForest:
```

©Joseph Aragbaiye Project, 2026

```
    """
    A small, clear wrapper around CapyMOA BitcoinOnlineIsolationForest.

    CapyMOA owns the genuine streaming forest. This wrapper keeps the same
    score-first, learn-second interface used by the rest of the pipeline.
    """

    def __init__(
        self,
        warmup_size: int = ONLINE_IF_WARMUP_SIZE,
        window_size: int = ONLINE_IF_WINDOW_SIZE,
    ) -> None:
        if warmup_size < 2:
            raise ValueError("Online IF warm-up must be at least 2.")

        if window_size < warmup_size:
            raise ValueError(
                "Online IF window must be at least as large as its warm-up."
            )

        self.warmup_size = warmup_size
        self.window_size = window_size
        self.learned_count = 0
        self.last_error: str | None = None

        schema_columns = [*FEATURE_NAMES, "target"]
        self.schema = Schema.from_custom(
            features=schema_columns,
            target="target",
            name="bitcoin-online-iforest",
        )
```

©Joseph Aragbaiye Project, 2026

```
        self.model = CapyOnlineIsolationForest(
            schema=self.schema,
            random_seed=RANDOM_SEED,
            num_trees=ONLINE_IF_TREES,
            max_leaf_samples=ONLINE_IF_MAX_LEAF_SAMPLES,
            growth_criterion=ONLINE_IF_GROWTH_CRITERION,
            subsample=ONLINE_IF_SUBSAMPLE,
            window_size=ONLINE_IF_WINDOW_SIZE,
            branching_factor=ONLINE_IF_BRANCHING_FACTOR,
            split="axisparallel",
            n_jobs=1,
        )

    @property
    def is_ready(self) -> bool:
        return self.learned_count >= self.warmup_size

    @property
    def health(self) -> str:
        if self.last_error is not None:
            return "ERROR"
        if not self.is_ready:
            return "WARMING_UP"
        return "LEARNING_AND_FORGETTING_CONTINUOUSLY"

    def make_instance(self, vector: list[float]) -> Instance:
        """Convert one NumPy-compatible vector to a CapyMOA instance."""

        point = np.asarray(vector, dtype=np.float64)
        return Instance.from_array(self.schema, point)
```

©Joseph Aragbaiye Project, 2026

```
  def score(self, vector: list[float]) -> float | None:
    """Score without learning. None means no valid score is available."""

    if self.learned_count == 0:
      return None

    try:
      score = float(self.model.score_instance(self.make_instance(vector)))
      if not np.isfinite(score):
        raise ValueError(f"Online-iForest returned invalid score {score}")
      self.last_error = None
      return score
    except Exception as error:
      self.last_error = f"{type(error).__name__}: {error}"
      print(f"[!] CapyMOA Online-iForest scoring error: {self.last_error}")
      return None

  def learn(self, vector: list[float]) -> None:
    """Learn one transaction after it has been scored."""

    try:
      self.model.train(self.make_instance(vector))
      self.learned_count += 1
      self.last_error = None
    except Exception as error:
      self.last_error = f"{type(error).__name__}: {error}"
      print(f"[!] CapyMOA Online-iForest learning error: {self.last_error}")


# Three-model Bitcoin anomaly engine
```

©Joseph Aragbaiye Project, 2026

```
class BitcoinAnomalyEngine:
    """Score, learn, combine, explain, and report one transaction at a time."""

    def __init__(self) -> None:
        self.rows_seen = 0

        # HST is created once. There is no code anywhere that replaces it.
        self.hst = anomaly.HalfSpaceTrees(
            n_trees=HST_TREES,
            height=HST_HEIGHT,
            window_size=HST_WINDOW_SIZE,
            seed=RANDOM_SEED,
        )
        self.hst_scores: deque[float] = deque(
            maxlen=HST_SCORE_HISTORY_SIZE
        )
        self.hst_recent_scores: deque[float] = deque(
            maxlen=HST_HEALTH_WINDOW
        )
        self.hst_learning_count = 0
        self.hst_last_error: str | None = None

        # Online Isolation Forest
        self.online_if = BitcoinOnlineIsolationForest()
        self.online_if_scores: deque[float] = deque(
            maxlen=ONLINE_IF_SCORE_HISTORY_SIZE
        )

        # Batch Isolation Forest and SHAP
        self.batch_if: IsolationForest | None = None
```

©Joseph Aragbaiye Project, 2026

```
self.batch_explainer: shap.TreeExplainer | None = None
self.batch_reference_scores = np.asarray([], dtype=float)
self.training_window: deque[list[float]] = deque(
    maxlen=BATCH_IF_WINDOW
)
self.last_batch_training_row = 0

# Locks keep scoring safe while a background model is being replaced.
self.model_lock = threading.Lock()
self.training_lock = threading.Lock()
self.batch_retraining = False
self.retrain_queued = False

# Drift detectors and K-Means
self.adwin = {
    feature: drift.ADWIN()
    for feature in FEATURE_NAMES
}
self.drift_transactions_remaining = 0
self.last_drift_features: list[str] = []

# Adaptive controller state. It changes only after enough recent
# evidence is available, or immediately when ADWIN detects drift.
self.adaptive_mode = "CALIBRATING"
self.adaptive_reason = "Waiting for the 400-transaction warm-up."
self.current_anomaly_cutoff = STABLE_ANOMALY_PERCENTILE
self.current_medium_threshold = STABLE_MEDIUM_RISK
self.current_critical_threshold = STABLE_CRITICAL_RISK
self.current_model_weights = {
    "HST": MODEL_WEIGHT,
    "ONLINE_IF": MODEL_WEIGHT,
```

©Joseph Aragbaiye Project, 2026

```
    "BATCH_IF": MODEL_WEIGHT,
}
self.model_change_count = 0
self.last_adaptive_review_row = 0
self.adaptive_history: deque[dict[str, Any]] = deque(
    maxlen=ADAPTIVE_HISTORY_SIZE
)
self.confirmed_detection_rate = 0.0
self.single_model_rate = 0.0
self.high_risk_unconfirmed_rate = 0.0

self.kmeans = MiniBatchKMeans(
    n_clusters=KMEANS_CLUSTERS,
    batch_size=KMEANS_BATCH_SIZE,
    random_state=RANDOM_SEED,
    n_init=10,
)
self.kmeans_ready = False

# Rolling latency records
self.hst_latency: deque[float] = deque(maxlen=LATENCY_HISTORY_SIZE)
self.online_if_latency: deque[float] = deque(
    maxlen=LATENCY_HISTORY_SIZE
)
self.batch_if_latency: deque[float] = deque(
    maxlen=LATENCY_HISTORY_SIZE
)

# 1,000-transaction report counters
self.report_start_row = 1
self.reset_report_counters()
```

©Joseph Aragbaiye Project, 2026

```
# Feature preparation

@staticmethod
def build_transaction_features(tx: dict[str, Any]) -> dict[str, Any]:
    """Turn one decoded Bitcoin transaction into the five features shared by the detectors."""

    raw_values = np.asarray(
        [
            clean_bitcoin_metric(tx.get("total_output_btc")),
            clean_bitcoin_metric(tx.get("vsize")),
            clean_bitcoin_metric(tx.get("input_count")),
            clean_bitcoin_metric(tx.get("output_count")),
        ],
        dtype=np.float64,
    )
    raw_values = np.nan_to_num(
        raw_values,
        nan=0.0,
        posinf=0.0,
        neginf=0.0,
    )
    raw_values = np.maximum(raw_values, 0.0)

    amount, vsize, inputs, outputs = raw_values
    output_input_ratio = (outputs + 1.0) / (inputs + 1.0)

    model_vector = np.concatenate(
        [
            np.log1p(raw_values),
            np.asarray([np.log(output_input_ratio)], dtype=np.float64),
```

©Joseph Aragbaiye Project, 2026

```
    ]
)
scaled = {
    name: float(value)
    for name, value in zip(FEATURE_NAMES, model_vector, strict=True)
}

# River HST expects bounded values. NumPy maps the positive log features
# with x/(1+x), and maps the signed ratio with a logistic function.
hst_vector = np.empty_like(model_vector)
hst_vector[:4] = model_vector[:4] / (1.0 + model_vector[:4])
bounded_ratio = np.clip(model_vector[4], -60.0, 60.0)
hst_vector[4] = 1.0 / (1.0 + np.exp(-bounded_ratio))
hst_features = {
    name: float(value)
    for name, value in zip(FEATURE_NAMES, hst_vector, strict=True)
}

return {
    "raw_amount": float(amount),
    "raw_vsize": float(vsize),
    "raw_inputs": float(inputs),
    "raw_outputs": float(outputs),
    "raw_ratio": float(output_input_ratio),
    "raw_complexity": float(inputs + outputs),
    "scaled": scaled,
    "hst": hst_features,
    "vector": model_vector.tolist(),
    "classify_transaction_shape": BitcoinAnomalyEngine.classify_transaction_shape(
        int(inputs),
        int(outputs),
```

©Joseph Aragbaiye Project, 2026

```
            ),
        }

    @staticmethod
    def classify_transaction_shape(input_count: int, output_count: int) -> str:
        """Describe transaction shape for explanation only; this label does not cast a model vote."""

        if (
            input_count >= STRUCTURE_MIN_COUNT
            and input_count >= STRUCTURE_DOMINANCE * max(1, output_count)
        ):
            return "CONSOLIDATION_LIKE"
        if (
            output_count >= STRUCTURE_MIN_COUNT
            and output_count >= STRUCTURE_DOMINANCE * max(1, input_count)
        ):
            return "FAN_OUT_LIKE"
        if input_count >= COINJOIN_MIN_COUNT and output_count >= COINJOIN_MIN_COUNT:
            ratio = output_count / max(1, input_count)
            if COINJOIN_RATIO_LOW <= ratio <= COINJOIN_RATIO_HIGH:
                return "BATCHING_OR_COINJOIN_LIKE"
        return "ORDINARY_TRANSFER_LIKE"

    @property
    def drift_mode_active(self) -> bool:
        return self.drift_transactions_remaining > 0

    @property
    def anomaly_cutoff(self) -> float:
        return self.current_anomaly_cutoff
```

©Joseph Aragbaiye Project, 2026

```
@property
def medium_risk_threshold(self) -> float:
    return self.current_medium_threshold

@property
def critical_risk_threshold(self) -> float:
    return self.current_critical_threshold

def set_adaptive_mode(self, mode: str, reason: str) -> None:
    """Switch the percentile and risk thresholds as the live stream behaviour changes."""

    settings = {
        "CALIBRATING": (
            STABLE_ANOMALY_PERCENTILE,
            STABLE_MEDIUM_RISK,
            STABLE_CRITICAL_RISK,
        ),
        "MORE_SENSITIVE": (
            SENSITIVE_ANOMALY_PERCENTILE,
            SENSITIVE_MEDIUM_RISK,
            SENSITIVE_CRITICAL_RISK,
        ),
        "STABLE": (
            STABLE_ANOMALY_PERCENTILE,
            STABLE_MEDIUM_RISK,
            STABLE_CRITICAL_RISK,
        ),
        "DRIFT_ADAPTATION": (
            DRIFT_ANOMALY_PERCENTILE,
            DRIFT_MEDIUM_RISK,
            DRIFT_CRITICAL_RISK,
```

©Joseph Aragbaiye Project, 2026

```
            ),
            "STRICT": (
                STRICT_ANOMALY_PERCENTILE,
                STRICT_MEDIUM_RISK,
                STRICT_CRITICAL_RISK,
            ),
        }
        if mode not in settings:
            raise ValueError(f"Unknown adaptive mode: {mode}")

        if mode != self.adaptive_mode:
            self.model_change_count += 1

        cutoff, medium, critical = settings[mode]
        self.adaptive_mode = mode
        self.adaptive_reason = reason
        self.current_anomaly_cutoff = cutoff
        self.current_medium_threshold = medium
        self.current_critical_threshold = critical

    def update_reliability_weights(self) -> None:
        """Nudge model weights using recent agreement, without allowing one model to decide alone."""

        if len(self.adaptive_history) < ADAPTIVE_MIN_SAMPLES:
            self.current_model_weights = {
                "HST": MODEL_WEIGHT,
                "ONLINE_IF": MODEL_WEIGHT,
                "BATCH_IF": MODEL_WEIGHT,
            }
            return
```

©Joseph Aragbaiye Project, 2026

```
    agreement: dict[str, list[bool]] = {
        "HST": [],
        "ONLINE_IF": [],
        "BATCH_IF": [],
    }
    for row in self.adaptive_history:
        majority_anomaly = row["anomaly_votes"] >= 2
        for model_name in agreement:
            agreement[model_name].append(
                bool(row[model_name]) == majority_anomaly
            )

    rates = {
        name: float(np.mean(values))
        for name, values in agreement.items()
    }
    mean_rate = float(np.mean(list(rates.values())))
    raw = {
        name: float(
            np.clip(
                MODEL_WEIGHT + 0.20 * (rate - mean_rate),
                MIN_MODEL_WEIGHT,
                MAX_MODEL_WEIGHT,
            )
        )
        for name, rate in rates.items()
    }
    total = sum(raw.values())
    self.current_model_weights = {
        name: weight / total
        for name, weight in raw.items()
```

©Joseph Aragbaiye Project, 2026

```
    }

def review_adaptive_need(self, force: bool = False) -> None:
    """Review recent Bitcoin decisions and change sensitivity only when the evidence crosses a trigger."""

    if self.rows_seen <= GLOBAL_WARMUP_SIZE:
        self.set_adaptive_mode(
            "CALIBRATING",
            "Waiting for the 400-transaction warm-up.",
        )
        return

    if self.drift_mode_active:
        return

    if (
        not force
        and self.rows_seen - self.last_adaptive_review_row
        < ADAPTIVE_REVIEW_INTERVAL
    ):
        return

    if len(self.adaptive_history) < ADAPTIVE_MIN_SAMPLES:
        self.set_adaptive_mode(
            "STABLE",
            "Warm-up complete; collecting adaptive review evidence.",
        )
        return

    rows = list(self.adaptive_history)
    self.confirmed_detection_rate = float(
```

©Joseph Aragbaiye Project, 2026

```
    np.mean([row["anomaly_votes"] >= 2 for row in rows])
  )
  self.single_model_rate = float(
    np.mean([row["anomaly_votes"] == 1 for row in rows])
  )
  self.high_risk_unconfirmed_rate = float(
    np.mean([row["high_risk_unconfirmed"] for row in rows])
  )
  self.update_reliability_weights()
  self.last_adaptive_review_row = self.rows_seen

  if self.confirmed_detection_rate >= HIGH_CONFIRMED_DETECTION_TRIGGER:
    self.set_adaptive_mode(
      "STRICT",
      "Confirmed anomaly rate is unusually high; sensitivity was "
      "reduced to control false positives.",
    )
  elif (
    self.high_risk_unconfirmed_rate
    >= HIGH_RISK_UNCONFIRMED_TRIGGER
    or self.single_model_rate
    >= SINGLE_MODEL_DISAGREEMENT_TRIGGER
  ):
    self.set_adaptive_mode(
      "MORE_SENSITIVE",
      "Recent high-risk unconfirmed transactions or repeated "
      "single-model disagreement indicate that sensitivity should "
      "increase.",
    )
  else:
    self.set_adaptive_mode(
```

©Joseph Aragbaiye Project, 2026

```
            "STABLE",
            "Recent detection and disagreement rates are within the "
            "expected operating range.",
        )

    def record_adaptive_observation(
        self,
        consensus: dict[str, Any],
        risk_score: float,
        hst_flag: bool,
        online_if_flag: bool,
        batch_if_flag: bool,
    ) -> None:
        """Keep this verdict for the next controller review; it cannot change its own decision."""

        if consensus["available_models"] < 3:
            return

        self.adaptive_history.append({
            "anomaly_votes": consensus["anomaly_votes"],
            "high_risk_unconfirmed": bool(
                risk_score >= self.medium_risk_threshold
                and consensus["anomaly_votes"] < 2
            ),
            "HST": hst_flag,
            "ONLINE_IF": online_if_flag,
            "BATCH_IF": batch_if_flag,
        })

    # HST: score first, then always learn
```

©Joseph Aragbaiye Project, 2026

```
def current_hst_threshold(self) -> float:
    """Read the HST cut-off from its recent Bitcoin score distribution."""

    if len(self.hst_scores) < MIN_SCORE_HISTORY:
        return 0.0

    return float(
        np.quantile(
            np.asarray(self.hst_scores, dtype=np.float64),
            self.anomaly_cutoff,
        )
    )

def hst_health(self) -> str:
    """Treat repeated HST scores as a health problem instead of silently trusting them."""

    if self.hst_last_error is not None:
        return "ERROR"
    if len(self.hst_recent_scores) < HST_MIN_HEALTH_SCORES:
        return "WARMING_REFERENCE_WINDOW"

    values = np.asarray(self.hst_recent_scores, dtype=np.float64)
    spread = float(np.ptp(values))
    unique = int(np.unique(np.round(values, HST_SCORE_ROUND_DECIMALS)).size)

    if unique <= HST_FLATLINE_UNIQUE or spread <= HST_FLATLINE_SPREAD:
        return "FLATLINED"
    return "HEALTHY"

def score_hst(self, features: dict[str, float]) -> dict[str, Any]:
    """Score the transaction before HST learns it, and keep failures separate from genuine zero scores."""
```

©Joseph Aragbaiye Project, 2026

```
started = time.perf_counter()
score: float | None = None

try:
    score = float(self.hst.score_one(features))
    if not np.isfinite(score):
        raise ValueError(f"HST returned invalid score {score}")
    self.hst_last_error = None
except Exception as error:
    self.hst_last_error = f"{type(error).__name__}: {error}"
    print(f"[!] HST scoring error: {self.hst_last_error}")

latency_ms = (time.perf_counter() - started) * 1_000.0
threshold = self.current_hst_threshold()
score_percentile = rolling_score_percentile(self.hst_scores, score)
health = self.hst_health()
ready = (
    score is not None
    and self.rows_seen > GLOBAL_WARMUP_SIZE
    and len(self.hst_scores) >= MIN_SCORE_HISTORY
    and health not in {"ERROR", "FLATLINED"}
)
flagged = bool(ready and score_percentile >= self.anomaly_cutoff)

return {
    "score": score,
    "percentile": score_percentile,
    "threshold": threshold,
    "flag": flagged,
    "ready": ready,
```

©Joseph Aragbaiye Project, 2026

```
        "health": health,
        "error": self.hst_last_error,
        "latency_ms": latency_ms,
    }

def learn_hst(
    self,
    features: dict[str, float],
    score: float | None,
) -> None:
    """Let HST learn the current transaction only after its pre-update score has been captured."""

    try:
        self.hst.learn_one(features)
        self.hst_learning_count += 1
    except Exception as error:
        self.hst_last_error = f"{type(error).__name__}: {error}"
        print(f"[!] HST learning error: {self.hst_last_error}")

    if score is not None:
        self.hst_scores.append(score)
        self.hst_recent_scores.append(score)

# Online Isolation Forest: score first, then always learn

def score_online_if(self, vector: list[float]) -> dict[str, Any]:
    started = time.perf_counter()
    score = self.online_if.score(vector)
    finished = time.perf_counter()

    if score is None:
```

©Joseph Aragbaiye Project, 2026

```
        return {
            "score": None,
            "percentile": 0.0,
            "flag": False,
            "ready": False,
            "health": self.online_if.health,
            "error": self.online_if.last_error,
            "latency_ms": None,
        }

    score_percentile = rolling_score_percentile(self.online_if_scores, score)
    ready = (
        self.online_if.is_ready
        and len(self.online_if_scores) >= ONLINE_IF_MIN_SCORE_HISTORY
        and self.online_if.last_error is None
    )
    flagged = bool(ready and score_percentile >= self.anomaly_cutoff)

    return {
        "score": score,
        "percentile": score_percentile,
        "flag": flagged,
        "ready": ready,
        "health": self.online_if.health,
        "error": self.online_if.last_error,
        "latency_ms": (finished - started) * 1_000.0,
    }

  def learn_online_if(
    self,
    vector: list[float],
```

©Joseph Aragbaiye Project, 2026

```
    score: float | None,
) -> None:
    self.online_if.learn(vector)

    if score is not None:
        self.online_if_scores.append(score)

# ADWIN drift detection

def detect_drift(self, features: dict[str, float]) -> list[str]:
    """Feed each transaction feature to its own ADWIN drift monitor."""

    changed_features: list[str] = []

    for feature_name, value in features.items():
        detector = self.adwin[feature_name]
        detector.update(value)

        if detector.drift_detected:
            changed_features.append(feature_name)

    if changed_features:
        self.drift_transactions_remaining = DRIFT_ADAPTATION_TRANSACTIONS
        self.last_drift_features = changed_features
        self.set_adaptive_mode(
            "DRIFT_ADAPTATION",
            "ADWIN detected a stream change in: "
            + ", ".join(changed_features)
            + ".",
        )
```

©Joseph Aragbaiye Project, 2026

```
    return changed_features

  def advance_drift_timer(self) -> None:
    if self.drift_transactions_remaining > 0:
      self.drift_transactions_remaining -= 1

  # K-Means behaviour grouping

  def update_clusters(self, vector: list[float]) -> tuple[int | None, str]:
    """Refresh behavioural clusters in small batches and label the current transaction group."""

    enough_rows = len(self.training_window) >= KMEANS_BATCH_SIZE
    time_to_update = self.rows_seen % KMEANS_BATCH_SIZE == 0

    if enough_rows and time_to_update:
      recent_rows = list(self.training_window)[-KMEANS_BATCH_SIZE:]
      self.kmeans.partial_fit(np.asarray(recent_rows, dtype=float))
      self.kmeans_ready = True

    if not self.kmeans_ready:
      return None, "Calibrating behavioural clusters..."

    cluster_id = int(
      self.kmeans.predict(np.asarray([vector], dtype=float))[0]
    )
    return cluster_id, self.cluster_label(cluster_id)

  def cluster_label(self, cluster_id: int) -> str:
    """Translate a numerical cluster into a Bitcoin transaction behaviour description."""

    centres = np.asarray(self.kmeans.cluster_centers_, dtype=np.float64)
```

©Joseph Aragbaiye Project, 2026

```
    centre = centres[cluster_id]

    amount = centre[0]
    inputs = centre[2]
    outputs = centre[3]
    median_amount = float(np.median(centres[:, 0]))
    median_inputs = float(np.median(centres[:, 2]))
    median_outputs = float(np.median(centres[:, 3]))

    if inputs > 1.5 * max(median_inputs, np.finfo(float).eps):
        if outputs <= median_outputs:
            return "Heuristic: input-heavy consolidation-like transfer"
    if outputs > 1.5 * max(median_outputs, np.finfo(float).eps):
        if inputs <= median_inputs:
            return "Heuristic: output-heavy fan-out-like transfer"
    if inputs > median_inputs and outputs > median_outputs:
        return "Heuristic: batching or CoinJoin-like structure"
    if amount > median_amount:
        return "Heuristic: higher-value ordinary transfer"
    return "Heuristic: lower-value ordinary transfer"

# Batch Isolation Forest training

def fit_batch_if(
    self,
    rows: list[list[float]],
    trained_at_row: int,
) -> None:
    """Fit the next Batch Isolation Forest off the scoring path, then replace the active copy safely."""

    try:
```

©Joseph Aragbaiye Project, 2026

```
    matrix = np.asarray(rows, dtype=float)

    if matrix.ndim != 2 or len(matrix) < INITIAL_BATCH_TRAINING_SIZE:
      return

    new_model = IsolationForest(
      n_estimators=BATCH_IF_TREES,
      contamination=BATCH_IF_CONTAMINATION,
      random_state=RANDOM_SEED,
      n_jobs=-1,
    )
    new_model.fit(matrix)

    new_reference_scores = -new_model.score_samples(matrix)
    new_explainer = shap.TreeExplainer(
      new_model,
      feature_names=FEATURE_NAMES,
    )

    with self.model_lock:
      self.batch_if = new_model
      self.batch_reference_scores = np.asarray(
        new_reference_scores,
        dtype=float,
      )
      self.batch_explainer = new_explainer
      self.last_batch_training_row = trained_at_row

  except Exception as error:
    print(f"[!] Batch Isolation Forest training failed: {error}")
```

©Joseph Aragbaiye Project, 2026

```
    finally:
        run_queued_training = False

        with self.training_lock:
            self.batch_retraining = False

            if self.retrain_queued:
                self.retrain_queued = False
                run_queued_training = True

        if run_queued_training:
            self.start_batch_retraining(force=True)

def train_initial_batch_model(self) -> None:
    """Fit the first Batch Isolation Forest once the 400-row calibration window is available."""

    if self.batch_if is not None:
        return

    if len(self.training_window) < INITIAL_BATCH_TRAINING_SIZE:
        return

    with self.training_lock:
        if self.batch_retraining:
            return
        self.batch_retraining = True

    initial_rows = list(self.training_window)[-INITIAL_BATCH_TRAINING_SIZE:]

    # Initial training is synchronous so the first model is immediately ready.
    self.fit_batch_if(initial_rows, self.rows_seen)
```

©Joseph Aragbaiye Project, 2026

```
def start_batch_retraining(self, force: bool = False) -> None:
    """
    Start background retraining only at exact 1,000-row boundaries.

    `force=True` is used only to run a boundary retraining that was queued
    because the previous training had not finished yet.
    """

    enough_rows = len(self.training_window) >= INITIAL_BATCH_TRAINING_SIZE
    at_boundary = (
        self.rows_seen > 0
        and self.rows_seen % BATCH_RETRAIN_INTERVAL == 0
    )

    if not enough_rows:
        return

    if not force and not at_boundary:
        return

    if not force and self.last_batch_training_row == self.rows_seen:
        return

    with self.training_lock:
        if self.batch_retraining:
            if at_boundary:
                self.retrain_queued = True
            return

        self.batch_retraining = True
```

©Joseph Aragbaiye Project, 2026

```
    recent_rows = list(self.training_window)[-BATCH_IF_WINDOW:]
    trained_at_row = self.rows_seen

    thread = threading.Thread(
        target=self.fit_batch_if,
        args=(recent_rows, trained_at_row),
        daemon=True,
    )
    thread.start()

# Batch Isolation Forest scoring and explanation

def score_batch_if(self, vector: list[float]) -> dict[str, Any]:
    """Score against the most recently completed Batch Isolation Forest."""

    with self.model_lock:
        model = self.batch_if
        explainer = self.batch_explainer
        reference_scores = self.batch_reference_scores.copy()

    if model is None:
        return {
            "score": None,
            "percentile": 0.0,
            "flag": False,
            "reason": "Batch IF is calibrating.",
            "latency_ms": None,
        }

    matrix = np.asarray([vector], dtype=float)
```

©Joseph Aragbaiye Project, 2026

```
    started = time.perf_counter()
    score = float(-model.score_samples(matrix)[0])
    latency_ms = (time.perf_counter() - started) * 1_000.0

    score_percentile = rolling_score_percentile(reference_scores, score)
    model_flag = model.predict(matrix)[0] == -1
    percentile_flag = score_percentile >= self.anomaly_cutoff
    flagged = bool(model_flag or percentile_flag)

    if not flagged or explainer is None:
        reason = "Batch IF remains within its recent baseline."
    else:
        reason = self.shap_reason(explainer, matrix, flagged=True)

    return {
        "score": score,
        "percentile": score_percentile,
        "flag": flagged,
        "reason": reason,
        "latency_ms": latency_ms,
    }

@staticmethod
def shap_reason(
    explainer: shap.TreeExplainer,
    matrix: np.ndarray,
    flagged: bool,
) -> str:
    """Name the transaction feature with the largest absolute SHAP contribution."""

    try:
```

©Joseph Aragbaiye Project, 2026

```
        shap_values = np.asarray(explainer.shap_values(matrix)).squeeze()
        shap_values = np.atleast_1d(shap_values).astype(float)

        if shap_values.size != len(FEATURE_NAMES):
            raise ValueError("Unexpected SHAP output shape.")

        contributions = dict(zip(FEATURE_NAMES, shap_values))
        strongest_feature = max(
            contributions,
            key=lambda name: abs(contributions[name]),
        )

        if flagged:
            return (
                "Batch IF flagged the transaction; strongest feature: "
                f"{strongest_feature}."
            )

        return (
            "Batch IF did not independently flag; strongest score "
            f"contribution: {strongest_feature}."
        )

    except Exception as error:
        return f"SHAP explanation unavailable ({error})."

def explain_batch_contribution(
    self,
    vector: list[float],
    existing_reason: str,
) -> str:
```

©Joseph Aragbaiye Project, 2026

```
    """Provide a Batch IF feature explanation even when HST and Online IF formed the majority."""

    with self.model_lock:
        explainer = self.batch_explainer

    if explainer is None:
        return existing_reason

    matrix = np.asarray([vector], dtype=float)
    return self.shap_reason(explainer, matrix, flagged=False)

# Consensus, weights, risk, and verdict

@staticmethod
def collect_model_votes(
    hst: dict[str, Any],
    online_if: dict[str, Any],
    batch_if: dict[str, Any],
    batch_available: bool,
) -> dict[str, Any]:
    """Record exactly which models agree and disagree.

    After the 400-transaction warm-up, every available model casts its own
    independent anomaly/normal vote. One model is allowed to disagree with
    the other two; the disagreement becomes part of the severity result.
    """

    model_flags: dict[str, bool] = {}
    if hst["ready"]:
        model_flags["HST"] = bool(hst["flag"])
    if online_if["ready"]:
```

©Joseph Aragbaiye Project, 2026

```
    model_flags["ONLINE_IF"] = bool(online_if["flag"])
  if batch_available:
    model_flags["BATCH_IF"] = bool(batch_if["flag"])

  available_models = len(model_flags)
  flagged = [name for name, flag in model_flags.items() if flag]
  normal = [name for name, flag in model_flags.items() if not flag]
  anomaly_votes = len(flagged)
  normal_votes = len(normal)

  if available_models < 3:
    label = "CALIBRATING"
    pattern = "WAITING_FOR_ALL_THREE_MODELS"
  elif anomaly_votes == 3:
    label = "ALL_MODELS_AGREE_ANOMALY"
    pattern = "THREE_ANOMALY_ZERO_NORMAL"
  elif anomaly_votes == 2:
    label = "MAJORITY_ANOMALY"
    pattern = "TWO_ANOMALY_ONE_NORMAL"
  elif anomaly_votes == 1:
    label = "SINGLE_MODEL_ANOMALY"
    pattern = "ONE_ANOMALY_TWO_NORMAL"
  else:
    label = "ALL_MODELS_AGREE_NORMAL"
    pattern = "ZERO_ANOMALY_THREE_NORMAL"

  return {
    "label": label,
    "pattern": pattern,
    "anomaly_votes": anomaly_votes,
    "normal_votes": normal_votes,
```

©Joseph Aragbaiye Project, 2026

```
        "available_models": available_models,
        "flagged_models": flagged,
        "normal_models": normal,
    }

  def effective_weights(
    self,
    hst_ready: bool,
    online_ready: bool,
    batch_ready: bool,
    batch_retraining: bool,
  ) -> dict[str, float]:
    """Use only ready detectors when calculating risk and renormalise their current weights."""

    weights = dict(self.current_model_weights)

    # The current Batch IF remains valid while a replacement model is
    # trained in the background, so it continues voting and keeps its weight.
    availability = {
        "HST": hst_ready,
        "ONLINE_IF": online_ready,
        "BATCH_IF": batch_ready,
    }

    for model_name, ready in availability.items():
        if not ready:
            weights[model_name] = 0.0

    total = sum(weights.values())
    if total > 0.0:
        weights = {
```

©Joseph Aragbaiye Project, 2026

```
            model_name: weight / total
            for model_name, weight in weights.items()
        }

    return weights

  def calculate_transaction_risk(
    self,
    hst: dict[str, Any],
    online_if: dict[str, Any],
    batch_if: dict[str, Any],
    batch_retraining: bool,
  ) -> dict[str, float]:
    """Combine the three detector percentiles into the operational risk score."""

    batch_score_exists = batch_if["score"] is not None
    weights = self.effective_weights(
        hst_ready=hst["ready"],
        online_ready=online_if["ready"],
        batch_ready=batch_score_exists,
        batch_retraining=batch_retraining,
    )

    weighted_score = (
        weights["HST"] * hst["percentile"]
        + weights["ONLINE_IF"] * online_if["percentile"]
        + weights["BATCH_IF"] * batch_if["percentile"]
    )

    anomaly_votes = sum(
        [
```

©Joseph Aragbaiye Project, 2026

```
            bool(hst["ready"] and hst["flag"]),
            bool(online_if["ready"] and online_if["flag"]),
            bool(
                batch_score_exists
                and batch_if["flag"]
            ),
        ]
    )

    # A single-model flag is disagreement, not a confirmed detection.
    if anomaly_votes >= 3:
        confidence_bonus = THREE_MODEL_CONFIDENCE_BONUS
    elif anomaly_votes == 2:
        confidence_bonus = TWO_MODEL_CONFIDENCE_BONUS
    else:
        confidence_bonus = 0.0

    risk_score = float(
        np.clip(weighted_score + confidence_bonus, 0.0, 1.0)
    )

    return {
        "score": risk_score,
        "hst_weight": weights["HST"],
        "online_if_weight": weights["ONLINE_IF"],
        "batch_if_weight": weights["BATCH_IF"],
    }

def decide_transaction_verdict(
    self,
    consensus: str,
```

©Joseph Aragbaiye Project, 2026

```
    anomaly_votes: int,
    risk_score: float,
) -> str:
    """Choose a verdict while enforcing two-model confirmation.

    0 flags -> NORMAL
    1 flag  -> MODEL_DISAGREEMENT (not a detection)
    2 flags -> LESS_CRITICAL, MEDIUM_RISK, or CRITICAL by risk score
    3 flags -> CRITICAL
    """

    if consensus == "CALIBRATING":
        return "CALIBRATING"
    if anomaly_votes >= 3:
        return "CRITICAL"
    if anomaly_votes == 2:
        if risk_score >= self.critical_risk_threshold:
            return "CRITICAL"
        if risk_score >= self.medium_risk_threshold:
            return "MEDIUM_RISK"
        return "LESS_CRITICAL"
    if anomaly_votes == 1:
        return "MODEL_DISAGREEMENT"
    return "NORMAL"

def explain_transaction_decision(
    self,
    verdict: str,
    consensus: str,
    risk_score: float,
    amount: float,
```

©Joseph Aragbaiye Project, 2026

```
    policy_review: bool,
    hst_flag: bool,
    online_if_flag: bool,
    batch_if_flag: bool,
    drift_features: list[str],
    model_reason: str,
) -> str:
    """Build the human-readable reason stored in SQLite and shown in Grafana."""

    anomaly_votes = sum([hst_flag, online_if_flag, batch_if_flag])
    if verdict == "CRITICAL":
        decision = (
            f"Critical: {anomaly_votes} models flagged the transaction; "
            f"combined risk={risk_score:.4f}."
        )
    elif verdict == "MEDIUM_RISK":
        decision = (
            f"Medium risk: two models flagged the transaction; "
            f"combined risk={risk_score:.4f}."
        )
    elif verdict == "LESS_CRITICAL":
        decision = (
            f"Less critical: two models flagged the transaction, but "
            f"combined risk={risk_score:.4f} stayed below the medium "
            "threshold."
        )
    elif verdict == "MODEL_DISAGREEMENT":
        decision = (
            "Model disagreement: only one model flagged the transaction. "
            "At least two model flags are required for detection."
        )
```

©Joseph Aragbaiye Project, 2026

```
elif verdict == "CALIBRATING":
    decision = (
        f"Calibrating: the first {GLOBAL_WARMUP_SIZE} transactions "
        "are used for model learning before verdict voting begins."
    )
else:
    decision = (
        f"Normal: none of the three models flagged the transaction; "
        f"combined risk={risk_score:.4f}."
    )

if policy_review:
    decision += (
        f" The amount {amount:.4f} BTC also crossed the "
        f"{POLICY_BTC_THRESHOLD:.4f} BTC policy-review threshold."
    )
if drift_features:
    decision += " ADWIN detected drift in: " + ", ".join(drift_features) + "."

votes = (
    f"Consensus={consensus}; HST flag={hst_flag}; "
    f"Online IF flag={online_if_flag}; Batch IF flag={batch_if_flag}."
)

learning = (
    "HST continues learning on every transaction without pause, "
    f"and Batch IF keeps its fixed {BATCH_RETRAIN_INTERVAL}-"
    "transaction retraining schedule."
)

return f"{decision} {votes} {model_reason} {learning}"
```

©Joseph Aragbaiye Project, 2026

```
  # Latency

  def record_model_latency(
      self,
      hst_ms: float,
      online_ms: float | None,
      batch_ms: float | None,
  ) -> dict[str, Any]:
      self.hst_latency.append(hst_ms)

      if online_ms is not None:
          self.online_if_latency.append(online_ms)

      if batch_ms is not None:
          self.batch_if_latency.append(batch_ms)

      current = {"HST": hst_ms}
      if online_ms is not None:
          current["Online IF"] = online_ms
      if batch_ms is not None:
          current["Batch IF"] = batch_ms

      fastest_model = min(current, key=current.get)
      fastest_time = current[fastest_model]
      slowest_time = max(current.values())
      ratio = slowest_time / fastest_time if fastest_time > 0 else None

      return {
          "hst_latency_ms": hst_ms,
          "online_if_latency_ms": online_ms,
```

©Joseph Aragbaiye Project, 2026

```
        "if_latency_ms": batch_ms,
        "avg_hst_latency_ms": rolling_mean(self.hst_latency),
        "avg_online_if_latency_ms": rolling_mean(
            self.online_if_latency
        ),
        "avg_if_latency_ms": rolling_mean(self.batch_if_latency),
        "faster_model": fastest_model,
        "latency_ratio": ratio,
    }

# 1,000-transaction statistics

def reset_report_counters(self) -> None:
    self.report_transactions = 0
    self.report_detections = 0
    self.report_hst_flags = 0
    self.report_online_flags = 0
    self.report_batch_flags = 0
    self.report_hst_ready = 0
    self.report_online_ready = 0
    self.report_batch_ready = 0
    self.report_drift_events = 0
    self.report_drift_rows = 0
    self.report_drift_features = {name: 0 for name in FEATURE_NAMES}
    self.report_consensus = {
        "ALL_MODELS_AGREE_ANOMALY": 0,
        "MAJORITY_ANOMALY": 0,
        "SINGLE_MODEL_ANOMALY": 0,
        "ALL_MODELS_AGREE_NORMAL": 0,
        "CALIBRATING": 0,
    }
```

©Joseph Aragbaiye Project, 2026

```
        self.report_verdicts: dict[str, int] = {}
        self.report_hst_latency_total = 0.0
        self.report_online_latency_total = 0.0
        self.report_online_latency_count = 0
        self.report_batch_latency_total = 0.0
        self.report_batch_latency_count = 0

    def update_report(
        self,
        result: dict[str, Any],
        drift_features: list[str],
    ) -> str | None:
        """Update counters and return a report after exactly 1,000 rows."""

        self.report_transactions += 1
        self.report_detections += int(result["detection_flag"])
        self.report_hst_flags += int(result["hst_flag"])
        self.report_online_flags += int(result["online_if_flag"])
        self.report_batch_flags += int(result["if_flag"])
        self.report_hst_ready += int(result["hst_available"])
        self.report_online_ready += int(result["online_if_available"])
        self.report_batch_ready += int(result["batch_if_available"])
        self.report_drift_events += int(bool(drift_features))
        self.report_drift_rows += int(self.drift_mode_active)

        consensus = result["consensus"]
        verdict = result["verdict"]
        self.report_consensus[consensus] = (
            self.report_consensus.get(consensus, 0) + 1
        )
        self.report_verdicts[verdict] = self.report_verdicts.get(verdict, 0) + 1
```

©Joseph Aragbaiye Project, 2026

```
for feature in drift_features:
    if feature in self.report_drift_features:
        self.report_drift_features[feature] += 1

self.report_hst_latency_total += result["hst_latency_ms"]

online_ms = result["online_if_latency_ms"]
if online_ms is not None:
    self.report_online_latency_total += online_ms
    self.report_online_latency_count += 1

batch_ms = result["if_latency_ms"]
if batch_ms is not None:
    self.report_batch_latency_total += batch_ms
    self.report_batch_latency_count += 1

if self.report_transactions < MODEL_COMPARISON_INTERVAL:
    return None

total = self.report_transactions
end_row = self.rows_seen
average_hst = self.report_hst_latency_total / total
average_online = (
    self.report_online_latency_total / self.report_online_latency_count
    if self.report_online_latency_count
    else None
)
average_batch = (
    self.report_batch_latency_total / self.report_batch_latency_count
    if self.report_batch_latency_count
```

©Joseph Aragbaiye Project, 2026

```
        else None
    )

    verdict_lines = [
        f"  {name:<22}: {count}"
        for name, count in sorted(self.report_verdicts.items())
    ]
    drift_text = ", ".join(
        f"{name}={count}"
        for name, count in self.report_drift_features.items()
    )

    lines = [
        "",
        "=" * 78,
        (
            "1,000-TRANSACTION THREE-MODEL STATISTICS: "
            f"rows {self.report_start_row}-{end_row}"
        ),
        "-" * 78,
        "MODEL AVAILABILITY AND FLAGS",
        (
            "  HST available/flags     : "
            f"{self.report_hst_ready}/{self.report_hst_flags}"
        ),
        (
            "  Online IF available/flags: "
            f"{self.report_online_ready}/{self.report_online_flags}"
        ),
        (
            "  Batch IF available/flags : "
```

©Joseph Aragbaiye Project, 2026

```
    f"{self.report_batch_ready}/{self.report_batch_flags}"
  ),
  "CONSENSUS",
  (
    "  All three anomaly      : "
    f"{self.report_consensus['ALL_MODELS_AGREE_ANOMALY']}"
  ),
  (
    "  Two anomaly, one normal  : "
    f"{self.report_consensus['MAJORITY_ANOMALY']}"
  ),
  (
    "  One anomaly, two normal  : "
    f"{self.report_consensus['SINGLE_MODEL_ANOMALY']}"
  ),
  (
    "  All three normal       : "
    f"{self.report_consensus['ALL_MODELS_AGREE_NORMAL']}"
  ),
  (
    "  Calibrating            : "
    f"{self.report_consensus['CALIBRATING']}"
  ),
  "DRIFT",
  f"  ADWIN drift events      : {self.report_drift_events}",
  f"  Adaptation-mode rows    : {self.report_drift_rows}",
  f"  Drift by feature        : {drift_text}",
  "VERDICTS",
  *verdict_lines,
  "LATENCY",
  f"  HST average             : {average_hst:.3f} ms",
```

©Joseph Aragbaiye Project, 2026

```
            (
                "  Online IF average       : "
                + (
                    f"{average_online:.3f} ms"
                    if average_online is not None
                    else "N/A (calibrating)"
                )
            ),
            (
                "  Batch IF average        : "
                + (
                    f"{average_batch:.3f} ms"
                    if average_batch is not None
                    else "N/A (calibrating)"
                )
            ),
            "=" * 78,
        ]

        report = "\n".join(lines)
        self.report_start_row = end_row + 1
        self.reset_report_counters()
        return report

    # Complete processing of one transaction

    def evaluate(
        self,
        tx: dict[str, Any],
        fingerprint: str,
    ) -> dict[str, Any]:
```

©Joseph Aragbaiye Project, 2026

```
    """Process one transaction using the current adaptive settings."""

    txid = str(tx.get("txid") or "UNKNOWN")
    timestamp = str(
        tx.get("timestamp")
        or datetime.now(timezone.utc).isoformat(timespec="seconds")
    )
    prepared = self.build_transaction_features(tx)
    features = prepared["scaled"]
    hst_features = prepared["hst"]
    vector = prepared["vector"]
    amount = prepared["raw_amount"]
    self.rows_seen += 1

    # Review completed historical decisions before scoring this row.
    # This ensures any change shown in the output was actually used.
    if not self.drift_mode_active and self.adaptive_mode == "DRIFT_ADAPTATION":
        self.review_adaptive_need(force=True)
    else:
        self.review_adaptive_need()

    # Drift is checked before model scoring so all three models use the same
    # sensitivity mode for the current transaction.
    drift_features = self.detect_drift(features)

    # Score each model before allowing it to learn the current transaction.
    hst_result = self.score_hst(hst_features)
    online_result = self.score_online_if(vector)
    batch_result = self.score_batch_if(vector)

    # Online learning never pauses.
```

©Joseph Aragbaiye Project, 2026

```
self.learn_online_if(vector, online_result["score"])
self.training_window.append(vector)
self.learn_hst(hst_features, hst_result["score"])

cluster_id, profile = self.update_clusters(vector)

# Transaction 400 completes initial Batch IF training. Transaction 401
# is the first row scored by that trained batch model.
self.train_initial_batch_model()

# Scheduled replacement remains fixed at 1,000-row boundaries.
self.start_batch_retraining()
with self.training_lock:
    is_batch_retraining = self.batch_retraining

batch_available = (
    self.rows_seen > GLOBAL_WARMUP_SIZE
    and batch_result["score"] is not None
)

consensus = self.collect_model_votes(
    hst=hst_result,
    online_if=online_result,
    batch_if=batch_result,
    batch_available=batch_available,
)
risk = self.calculate_transaction_risk(
    hst=hst_result,
    online_if=online_result,
    batch_if=batch_result,
    batch_retraining=is_batch_retraining,
```

©Joseph Aragbaiye Project, 2026

```
)

policy_review = amount >= POLICY_BTC_THRESHOLD
verdict = self.decide_transaction_verdict(
    consensus=consensus["label"],
    anomaly_votes=consensus["anomaly_votes"],
    risk_score=risk["score"],
)

model_reason = batch_result["reason"]
if (
    verdict in {"CRITICAL", "MEDIUM_RISK", "LESS_CRITICAL"}
    and not policy_review
    and not batch_result["flag"]
):
    model_reason = self.explain_batch_contribution(vector, model_reason)

explanation = self.explain_transaction_decision(
    verdict=verdict,
    consensus=consensus["label"],
    risk_score=risk["score"],
    amount=amount,
    policy_review=policy_review,
    hst_flag=hst_result["flag"],
    online_if_flag=online_result["flag"],
    batch_if_flag=batch_result["flag"],
    drift_features=drift_features,
    model_reason=model_reason,
)
explanation += (
    f" Adaptive controller={self.adaptive_mode}; "
```

©Joseph Aragbaiye Project, 2026

```
    f"cutoff={self.anomaly_cutoff:.2f}; reason={self.adaptive_reason}"
)

if self.drift_mode_active:
    adaptation_mode = (
        f"{self.adaptive_mode} "
        f"({self.drift_transactions_remaining} transactions remaining)"
    )
else:
    adaptation_mode = self.adaptive_mode

if drift_features:
    drift_label = ", ".join(drift_features)
elif self.drift_mode_active and self.last_drift_features:
    drift_label = (
        "Adapting after drift in: "
        + ", ".join(self.last_drift_features)
    )
else:
    drift_label = "Stable network baseline"

detection_flag = consensus["anomaly_votes"] >= 2
latency = self.record_model_latency(
    hst_ms=hst_result["latency_ms"],
    online_ms=online_result["latency_ms"],
    batch_ms=batch_result["latency_ms"],
)

result = {
    "txid": txid,
    "transaction_fingerprint": fingerprint,
```

©Joseph Aragbaiye Project, 2026

```
"timestamp": timestamp,
"verdict": verdict,
"detection_flag": detection_flag,
"anomaly_check_status": (
   "WARMING_UP" if verdict == "CALIBRATING" else "CHECKED"
),
"risk_score": risk["score"],
"amount_btc": amount,
"vsize_raw": prepared["raw_vsize"],
"input_count_raw": prepared["raw_inputs"],
"output_count_raw": prepared["raw_outputs"],
"output_input_ratio": prepared["raw_ratio"],
"complexity_raw": prepared["raw_complexity"],
"classify_transaction_shape": prepared["classify_transaction_shape"],
"consensus": consensus["label"],
"agreement_pattern": consensus["pattern"],
"flagged_models": ", ".join(consensus["flagged_models"]) or "NONE",
"normal_models": ", ".join(consensus["normal_models"]) or "NONE",
"anomaly_votes": consensus["anomaly_votes"],
"normal_votes": consensus["normal_votes"],
"available_models": consensus["available_models"],
"policy_review": policy_review,
"hst_score": hst_result["score"],
"hst_percentile": hst_result["percentile"],
"hst_threshold": hst_result["threshold"],
"hst_flag": hst_result["flag"],
"hst_available": hst_result["ready"],
"hst_health": hst_result["health"],
"hst_error": hst_result["error"],
"hst_learning_state": "LEARNING_CONTINUOUSLY",
"hst_learning_count": self.hst_learning_count,
```

©Joseph Aragbaiye Project, 2026

```
"online_if_score": online_result["score"],
"online_if_percentile": online_result["percentile"],
"online_if_flag": online_result["flag"],
"online_if_available": online_result["ready"],
"online_if_model": ONLINE_IF_MODEL_NAME,
"online_if_health": online_result["health"],
"online_if_error": online_result["error"],
"if_score": batch_result["score"],
"if_percentile": batch_result["percentile"],
"if_flag": batch_result["flag"],
"batch_if_available": batch_available,
"batch_if_retraining": is_batch_retraining,
"cluster_id": cluster_id,
"profile": profile,
"drift": drift_label,
"adaptation_mode": adaptation_mode,
"adaptive_reason": self.adaptive_reason,
"anomaly_cutoff": self.anomaly_cutoff,
"model_change_count": self.model_change_count,
"confirmed_detection_rate": self.confirmed_detection_rate,
"single_model_rate": self.single_model_rate,
"high_risk_unconfirmed_rate": self.high_risk_unconfirmed_rate,
"hst_weight": risk["hst_weight"],
"online_if_weight": risk["online_if_weight"],
"if_weight": risk["batch_if_weight"],
"critical_threshold": self.critical_risk_threshold,
"medium_threshold": self.medium_risk_threshold,
"if_window": BATCH_IF_WINDOW,
"retrain_interval": BATCH_RETRAIN_INTERVAL,
"shap_reason": explanation,
**latency,
```

©Joseph Aragbaiye Project, 2026

```
        }

        result["comparison_report"] = self.update_report(result, drift_features)

        # This observation can influence a later review, never its own verdict.
        self.record_adaptive_observation(
            consensus=consensus,
            risk_score=risk["score"],
            hst_flag=hst_result["flag"],
            online_if_flag=online_result["flag"],
            batch_if_flag=batch_result["flag"],
        )
        self.advance_drift_timer()
        return result

# Console output used while testing the live stream

def format_transaction_result(result: dict[str, Any]) -> str:
    """Format one live Bitcoin decision for terminal inspection during implementation tests."""

    hst_score = display_metric(result["hst_score"], decimals=12)
    online_score = display_metric(result["online_if_score"])
    batch_score = display_metric(result["if_score"])

    cluster = result["cluster_id"]
    cluster_text = str(cluster) if cluster is not None else "Calibrating"

    online_latency = display_metric(
        result["online_if_latency_ms"],
        decimals=3,
    )
```

©Joseph Aragbaiye Project, 2026

```
    batch_latency = display_metric(
        result["if_latency_ms"],
        decimals=3,
    )
    average_online_latency = display_metric(
        result["avg_online_if_latency_ms"],
        decimals=3,
    )
    average_batch_latency = display_metric(
        result["avg_if_latency_ms"],
        decimals=3,
    )

    if result["latency_ratio"] is None:
        speed_text = result["faster_model"]
    else:
        speed_text = (
            f"{result['faster_model']} is fastest; "
            f"slowest/fastest={result['latency_ratio']:.1f}x"
        )

    return f"""
{'-' * 78}
Transaction ID    : {result['txid']}
Verdict           : {result['verdict']}
Detection Flag    : {result['detection_flag']}
Anomaly Check     : {result['anomaly_check_status']}
Consensus         : {result['consensus']} \
(anomaly={result['anomaly_votes']}, normal={result['normal_votes']}, \
available={result['available_models']})
Agreement Pattern  : {result['agreement_pattern']}
```

©Joseph Aragbaiye Project, 2026

```
Flagged Models    : {result['flagged_models']}
Normal Models     : {result['normal_models']}
Risk Score        : {result['risk_score']:.4f} \
(medium={result['medium_threshold']:.2f}, critical={result['critical_threshold']:.2f})

HST               : score={hst_score}, \
percentile={result['hst_percentile']:.4f}, \
threshold={result['hst_threshold']:.4f}, flag={result['hst_flag']}
HST Health        : {result['hst_health']} \
(error={result['hst_error'] or 'NONE'})
HST Learning      : {result['hst_learning_state']} \
(learned={result['hst_learning_count']})
Online IF         : model={result['online_if_model']}, score={online_score}, \
percentile={result['online_if_percentile']:.4f}, flag={result['online_if_flag']}
Online IF Health  : {result['online_if_health']} \
(error={result['online_if_error'] or 'NONE'})
Batch IF          : score={batch_score}, \
percentile={result['if_percentile']:.4f}, flag={result['if_flag']}, \
retraining={result['batch_if_retraining']}

Latency this row  : HST={result['hst_latency_ms']:.3f} ms, \
Online IF={online_latency} ms, Batch IF={batch_latency} ms
Rolling latency   : HST={result['avg_hst_latency_ms']:.3f} ms, \
Online IF={average_online_latency} ms, Batch IF={average_batch_latency} ms
Speed comparison  : {speed_text}

Model weights     : HST={result['hst_weight']:.4f}, \
Online IF={result['online_if_weight']:.4f}, Batch IF={result['if_weight']:.4f}
Anomaly cutoff    : {result['anomaly_cutoff']:.2f}
Adaptive rates    : confirmed={result['confirmed_detection_rate']:.3f}, \
single={result['single_model_rate']:.3f}, \
```

©Joseph Aragbaiye Project, 2026

```
high-risk-unconfirmed={result['high_risk_unconfirmed_rate']:.3f}
Model changes     : {result['model_change_count']}
Batch IF schedule  : window={result['if_window']}, \
retrain every {result['retrain_interval']} transactions
Adaptation mode    : {result['adaptation_mode']}
Adaptive reason    : {result['adaptive_reason']}
Transaction shape  : inputs={result['input_count_raw']:.0f}, \
outputs={result['output_count_raw']:.0f}, \
ratio={result['output_input_ratio']:.4f}, label={result['classify_transaction_shape']}
Cluster group     : {cluster_text} - {result['profile']}
Network drift     : {result['drift']}
Explanation       : {result['shap_reason']}
{'-' * 78}
""".strip()



def print_latency_summary(engine: BitcoinAnomalyEngine) -> None:
    """Print the rolling detector latency averages collected during this run."""

    if not engine.hst_latency:
        return

    print(
        f"\n[*] Latency summary "
        f"(last {len(engine.hst_latency)} transactions):"
    )
    print(f"    HST avg latency      : {np.mean(engine.hst_latency):.3f} ms")

    online_average = rolling_mean(engine.online_if_latency)
    if online_average is None:
        print("    Online IF avg latency : N/A (never finished warming up)")
```

©Joseph Aragbaiye Project, 2026

```
    else:
        print(f"    Online IF avg latency : {online_average:.3f} ms")

    batch_average = rolling_mean(engine.batch_if_latency)
    if batch_average is None:
        print("    Batch IF avg latency  : N/A (never finished warming up)")
    else:
        print(f"    Batch IF avg latency  : {batch_average:.3f} ms")

# Kafka consumer entry point

def run_bitcoin_anomaly_pipeline() -> None:
    """Consume btc-tx messages, reject duplicates, score them, and persist each finished verdict."""

    prepare_results_database()
    engine = BitcoinAnomalyEngine()
    duplicate_guard = TransactionDuplicateGuard()

    consumer = KafkaConsumer(
        KAFKA_TOPIC,
        bootstrap_servers=KAFKA_SERVERS,
        value_deserializer=lambda raw: json.loads(raw.decode("utf-8")),
        auto_offset_reset="latest",
        enable_auto_commit=True,
        group_id=KAFKA_GROUP_ID,
    )

    print(
        "[*] Bitcoin anomaly pipeline using CapyMOA Online-iForest "
        f"and listening to Kafka topic '{KAFKA_TOPIC}' on {KAFKA_SERVERS}."
    )
```

©Joseph Aragbaiye Project, 2026

```
try:
    for message in consumer:
        transaction = message.value

        if not isinstance(transaction, dict):
            print("[!] Ignored message because it was not a JSON object.")
            continue

        txid = str(transaction.get("txid") or "").strip()
        if not txid:
            print("[!] Ignored transaction without a txid.")
            continue

        fingerprint = transaction_fingerprint(transaction)

        memory_reason = duplicate_guard.duplicate_reason(txid, fingerprint)
        if memory_reason is not None:
            print(
                f"[*] Skipped duplicate transaction {txid} "
                f"before learning ({memory_reason})."
            )
            continue

        if txid_already_saved(txid):
            print(
                f"[*] Skipped duplicate transaction {txid} "
                "before learning (SQLITE_TXID_DUPLICATE)."
            )
            duplicate_guard.remember(txid, fingerprint)
            continue
```

©Joseph Aragbaiye Project, 2026

```
        if transaction_body_already_saved(fingerprint):
            print(
                f"[*] Skipped duplicate transaction {txid} "
                "before learning (SQLITE_ROW_DUPLICATE)."
            )
            duplicate_guard.remember(txid, fingerprint)
            continue

        # Reserve the row in memory before any detector, drift monitor,
        # cluster, latency history, or training method receives it.
        duplicate_guard.remember(txid, fingerprint)
        result = engine.evaluate(transaction, fingerprint)

        if not save_transaction_result(result):
            print(f"[*] Transaction {txid} was saved by another process.")
            continue

        print(format_transaction_result(result))

        comparison_report = result["comparison_report"]
        if comparison_report is not None:
            print(comparison_report)
            email_model_report(comparison_report, engine.rows_seen)

        if result["verdict"] in EMAIL_ALERT_VERDICTS:
            email_transaction_alert(result)

    except KeyboardInterrupt:
        print("\n[*] Pipeline stopped safely.")
```

©Joseph Aragbaiye Project, 2026

```
    except Exception as error:
        print(f"\n[!] Pipeline stopped because of an error: {error}")
        raise

    finally:
        print_latency_summary(engine)
        consumer.close()

if __name__ == "__main__":
    run_bitcoin_anomaly_pipeline()
```

Grafana Installation and SQLite Connection

Grafana is used to view the results produced by the Bitcoin machine learning system. The machine learning program saves results in SQLite. Grafana reads the SQLite database and shows the information in a dashboard.

This guide starts from the Ubuntu terminal in WSL2. Run each command one at a time. The example user name is admp. If your user name is different, use your own name in the file path.

Step 1. Open Ubuntu in WSL2

Why: Grafana will be installed inside Ubuntu.

Step 2. Update Ubuntu

sudo apt update

Why: This refreshes the software list before installation.

Step 3. Install the tools needed for Grafana

sudo apt install -y wget gpg

Why: These tools help Ubuntu download and check Grafana packages.

Step 4. Create the Grafana key folder

sudo mkdir -p /etc/apt/keyrings

sudo wget -O /etc/apt/keyrings/grafana.asc https://apt.grafana.com/gpg-full.key

sudo chmod 644 /etc/apt/keyrings/grafana.asc

Why: This allows Ubuntu to trust the Grafana package source.

Step 5. Add the Grafana software source

©Joseph Aragbaiye Project, 2026

echo "deb [signed-by=/etc/apt/keyrings/grafana.asc] https://apt.grafana.com stable main" | sudo tee /etc/apt/sources.list.d/grafana.list

Why: This tells Ubuntu where to get Grafana.

Step 6. Update Ubuntu again

sudo apt update

Why: Ubuntu can now see the Grafana package.

Step 7. Install Grafana

sudo apt install -y grafana

Why: This installs the Grafana server.

Step 8. Start Grafana

sudo systemctl enable --now grafana-server

Why: This starts Grafana and allows it to start again automatically.

Step 9. Check Grafana

systemctl status grafana-server --no-pager

Why: The result should show that Grafana is active and running.

Step 10. Check port 3000

ss -ltnp | grep 3000

Why: Grafana normally uses port 3000.

Step 11. Open Grafana in your browser

Why: This opens the Grafana web page.

Open your Windows browser and enter http://localhost:3000

For the first login, use admin as the user name and admin as the password. Grafana may ask you to create a new password.

Step 12. Install SQLite

sudo apt install -y sqlite3

Why: This lets you check the project database from the terminal.

Step 13. Install the SQLite plugin for Grafana

sudo grafana-cli plugins install frser-sqlite-datasource

Why: This plugin lets Grafana read an SQLite database.

Step 14. Restart Grafana or start Grafana

sudo systemctl start grafana-server

sudo systemctl restart grafana-server

©Joseph Aragbaiye Project, 2026

Check if Grafana is runung : sudo systemctl status grafana-server --no-pager

Why: Grafana must restart before it can use the new plugin.

Step 15. Check the SQLite plugin

sudo grafana-cli plugins ls

Why: The result should include frser-sqlite-datasource.

Step 16. Find the SQLite database

find ~/bitcoin-fraud -maxdepth 3 -type f \(-name "*.db" -o -name "*.sqlite" -o -name "*.sqlite3" \)

Why: Grafana needs the full path of the database file.

An example path is /home/admp/bitcoin-fraud/bitcoin_fraud.db

Step 17. Check the database

sqlite3 /home/admp/bitcoin-fraud/bitcoin_fraud.db ".tables"

Why: This confirms that SQLite can open the database.

Use your real database path if it is different.

Step 18. Allow Grafana to read a database inside the home folder

sudo mkdir -p /etc/systemd/system/grafana-server.service.d

sudo nano /etc/systemd/system/grafana-server.service.d/bitcoin-fraud.conf

Why: Grafana can be blocked from reading files inside the Linux home folder.

Inside the file, enter the following two lines, then save and close the file.

[Service]

ProtectHome=false

Step 19. Reload the Grafana service

sudo systemctl daemon-reload

sudo systemctl restart grafana-server

systemctl show grafana-server -p ProtectHome

Why: The last command should show ProtectHome=false.

Step 20. Give Grafana permission to read the project folder

sudo apt install -y acl

sudo setfacl -m u:grafana:x /home/admp

sudo setfacl -R -m u:grafana:rX /home/admp/bitcoin-fraud

Why: Grafana runs with its own user account and needs permission to read the SQLite file.

If your Linux user name is not admp, replace admp with your own user name.

©Joseph Aragbaiye Project, 2026

Step 21. Open the Grafana connection page

Why: This is where the SQLite data source is added.

In Grafana, open Connections. Choose Add new connection. Search for SQLite.

Step 22. Add the SQLite data source

Why: This connects Grafana to the project database.

Choose SQLite. Select Add new data source. Give it a simple name such as Bitcoin SQLite.

Step 23. Enter the database path

Why: Grafana needs the exact location of the SQLite file.

For example, enter /home/admp/bitcoin-fraud/bitcoin_fraud.db

Step 24. Save and test

Why: A successful test means Grafana can open the database.

Choose Save and test.

Step 25. Check the table names

sqlite3 /home/admp/bitcoin-fraud/bitcoin_fraud.db ".tables"

Why: You need the table name before writing a Grafana query.

Step 26. Test a simple query in Grafana

Why: This confirms that Grafana can read rows from SQLite.

Open Explore. Select the Bitcoin SQLite data source. Enter a query that matches your table name.

SELECT * FROM transactions LIMIT 20;

If your table is not called transactions, replace transactions with the real table name.

Step 27. Create a simple dashboard

Why: This allows the database values to be shown on the screen.

Open Dashboards. Create a new dashboard. Add a visualization. Select the Bitcoin SQLite data source.

SELECT COUNT(*) AS total_transactions FROM transactions;

This example shows the number of rows in the transaction table. Later you can create panels for anomaly scores, drift information, model results and alerts.

Step 28. Set the dashboard refresh time

Why: This helps Grafana show new values while the machine learning program is running.

Use a simple refresh time such as 5 seconds or 10 seconds.

Simple checks if Grafana does not work

If Grafana does not open, run:

sudo systemctl restart grafana-server

©Joseph Aragbaiye Project, 2026

systemctl status grafana-server --no-pager

If you need to see recent Grafana errors, run:

sudo journalctl -u grafana-server -n 50 --no-pager

If Grafana cannot find the SQLite file, run:

ls -l /home/admp/bitcoin-fraud/bitcoin_fraud.db

If the database is inside the home folder, check this setting:

systemctl show grafana-server -p ProtectHome

The result should be:

ProtectHome=false

If the SQLite plugin does not appear, restart Grafana:

sudo systemctl restart grafana-server

If Grafana shows no data, test the database directly:

sqlite3 /home/admp/bitcoin-fraud/bitcoin_fraud.db "SELECT COUNT(*) FROM transactions;"

Configure Grafana Dashboard:

1. Confirm SQLite before building the dashboard

```
sudo -u grafana sqlite3 /home/admp/bitcoin-fraud-data/bitcoin_analytics_live.db
"SELECT COUNT(*) FROM transaction_verdicts;"

sudo -u grafana sqlite3 /home/admp/bitcoin-fraud-data/bitcoin_analytics_live.db
"SELECT rowid, verdict, risk_score, anomaly_votes, detection_flag FROM
transaction_verdicts ORDER BY rowid DESC LIMIT 1;"
```

If both commands return data, the datasource path and permissions are ready.

2. Create a completely new dashboard

1. Open http://localhost:3000 and sign in.
2. Open Dashboards.
3. Choose New, then New dashboard.
4. Choose Add visualization.
5. Select Bitcoin Fraud Live SQLite.
6. Create the first panel, save it, return to the dashboard and add the next panel.
7. Save the new dashboard as Bitcoin Fraud Investigator Live — V4.

Do not use Import dashboard and do not clone the old dashboard.

4. Dashboard-level settings

- Default time range: Last 6 hours.
- Auto refresh: 1s to match the supplied dashboard captures. Use 5s only if performance becomes unnecessarily heavy.
- Use the light Grafana appearance shown in the screenshots.

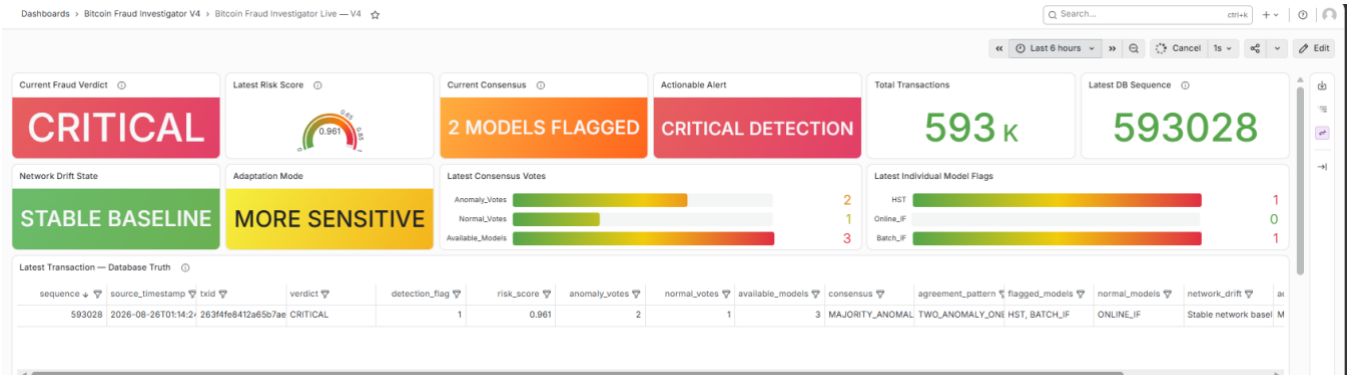
©Joseph Aragbaiye Project, 2026

- Keep the panel titles exactly as written in this guide.
- Do not hard-code example values such as 0.746, 593028 or CRITICAL.

©Joseph Aragbaiye Project, 2026

5. Build the top status row exactly

Create six panels across the first row. The first, third and fourth use large full-background Stat cards. The second is the semicircular Gauge. The fifth and sixth are large numeric Stat cards.



.

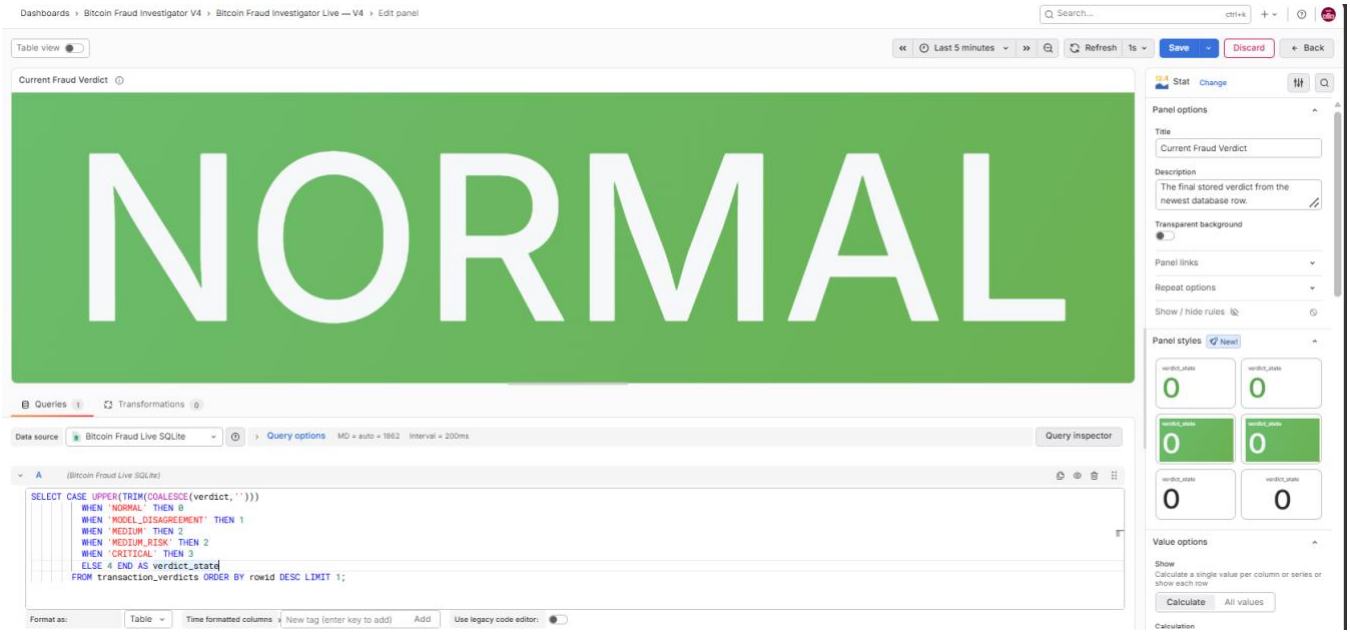
Panel 1. Current Fraud Verdict

Visualization: Stat. Set Color mode to Background solid. Keep the large full-background text presentation.

```
SELECT CASE UPPER(TRIM(COALESCE(verdict,'')))
    WHEN 'NORMAL' THEN 0
    WHEN 'MODEL_DISAGREEMENT' THEN 1
    WHEN 'LESS_CRITICAL' THEN 1
    WHEN 'MEDIUM' THEN 2
    WHEN 'MEDIUM_RISK' THEN 2
    WHEN 'CRITICAL' THEN 3
    WHEN 'CALIBRATING' THEN 4
    ELSE 5
END AS verdict_state
FROM transaction_verdicts
ORDER BY rowid DESC
LIMIT 1;
```

Value mappings: 0 NORMAL; 1 MODEL DISAGREEMENT; 2 MEDIUM RISK; 3 CRITICAL; 4 CALIBRATING; 5 UNKNOWN. Use green for NORMAL, warning colours for review states and red for CRITICAL.

©Joseph Aragbaiye Project, 2026



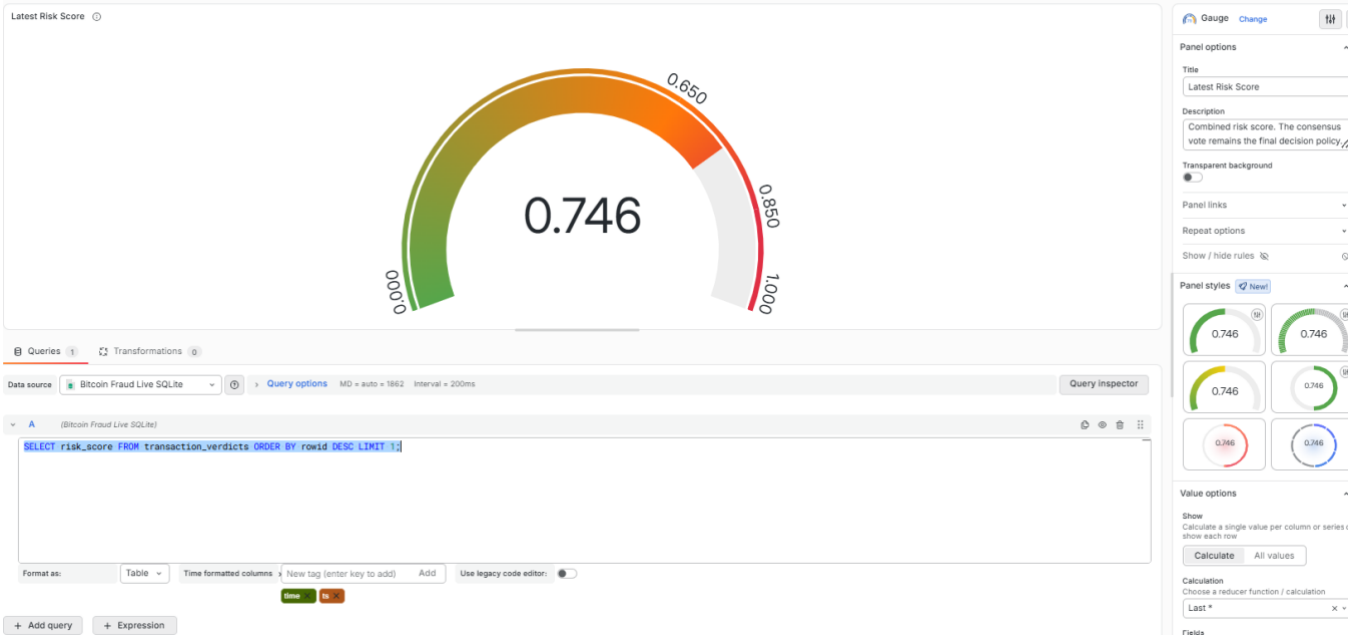
Reference: Current Fraud Verdict editor and full-background Stat design.

Panel 2. Latest Risk Score

Visualization: Gauge. Keep the semicircular gauge and central numeric value.

```
SELECT risk_score
FROM transaction_verdicts
ORDER BY rowid DESC
LIMIT 1;
```

- Minimum 0.
- Maximum 1.
- Decimals 3.
- Keep visible threshold markers around 0.650 and 0.850, ending at 1.000.
- Treat the value as an anomaly/risk score, not a fraud probability.



Reference: Latest Risk Score Gauge design.

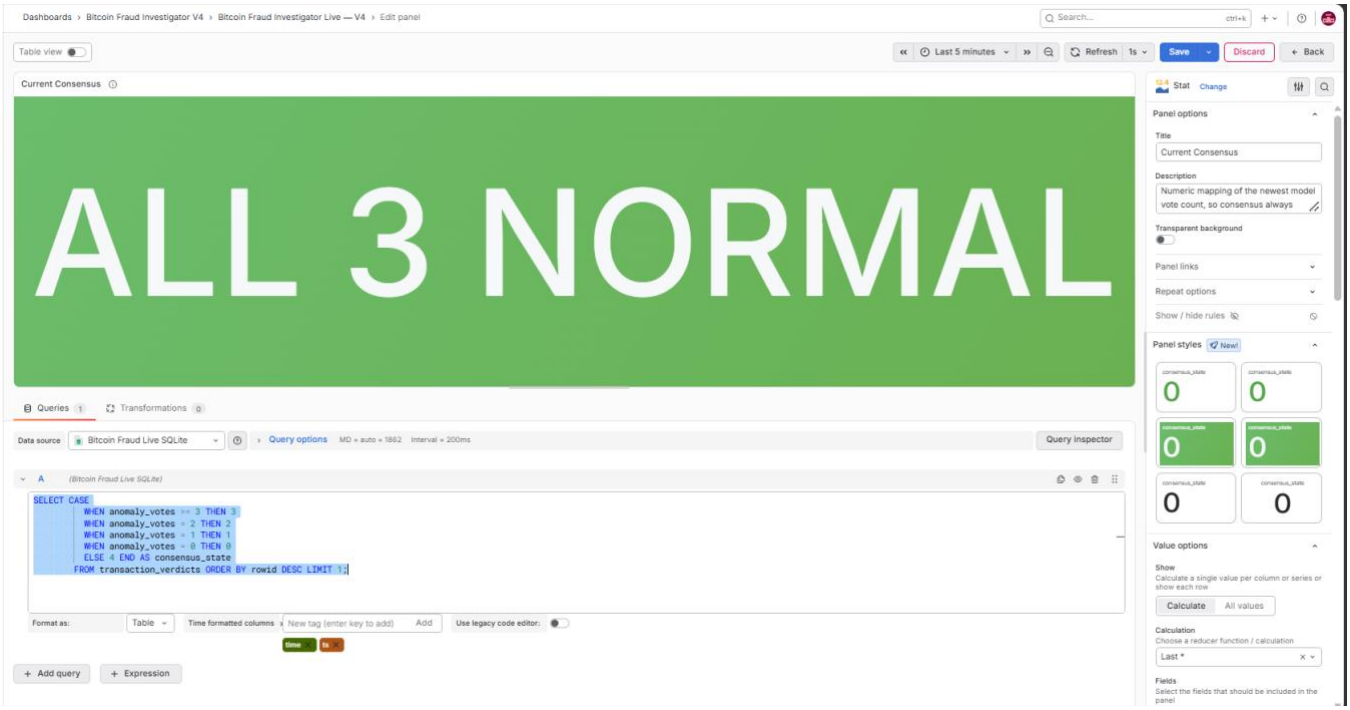
©Joseph Aragbaiye Project, 2026

Panel 3. Current Consensus

Visualization: Stat with full coloured background.

```
SELECT CASE
    WHEN anomaly_votes >= 3 THEN 3
    WHEN anomaly_votes = 2 THEN 2
    WHEN anomaly_votes = 1 THEN 1
    ELSE 0
END AS consensus_state
FROM transaction_verdicts
ORDER BY rowid DESC
LIMIT 1;
```

Value mappings: 0 ALL 3 NORMAL; 1 1 MODEL FLAGGED; 2 2 MODELS FLAGGED; 3 ALL 3 FLAGGED.



Reference: Current Consensus Stat design.

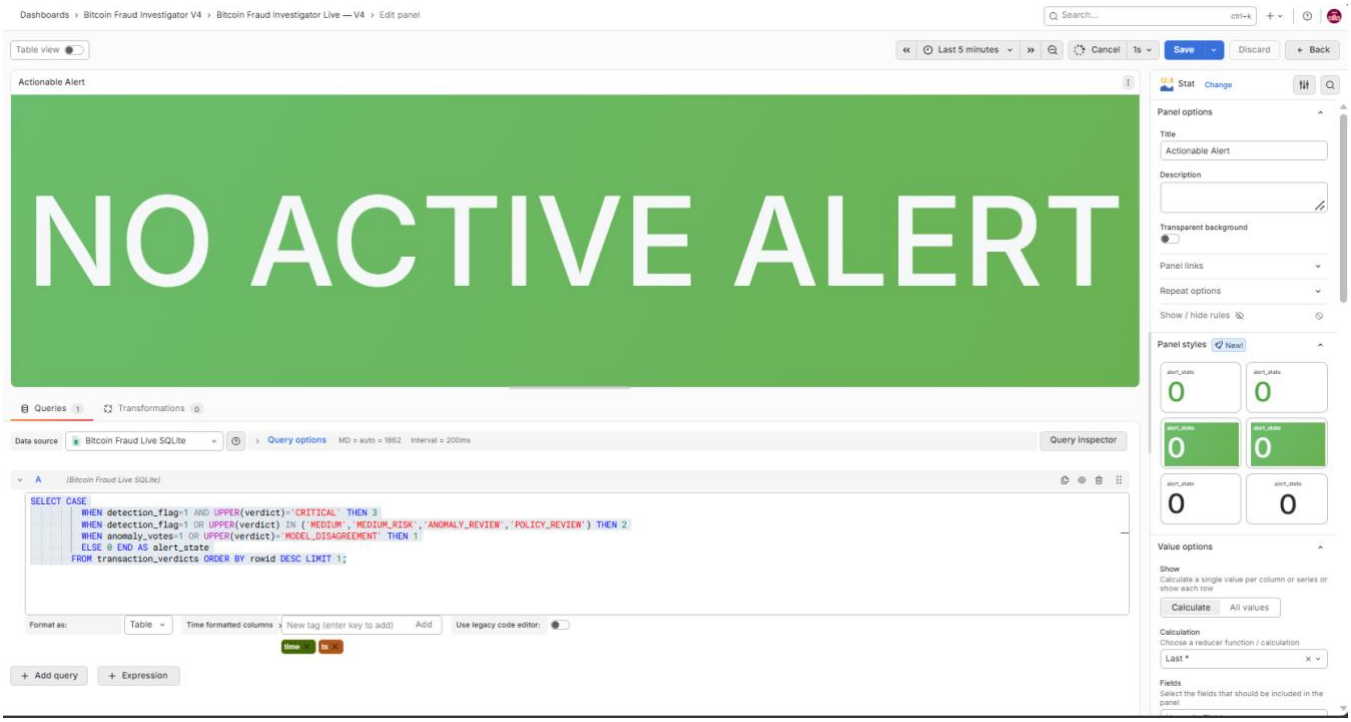
Panel 4. Actionable Alert

Visualization: Stat with full coloured background.

```
SELECT CASE
    WHEN detection_flag=1 AND UPPER(TRIM(verdict))='CRITICAL' THEN 3
    WHEN detection_flag=1 OR UPPER(TRIM(verdict)) IN
('MEDIUM','MEDIUM_RISK','ANOMALY_REVIEW','POLICY_REVIEW') THEN 2
    WHEN anomaly_votes=1 OR UPPER(TRIM(verdict)) IN
('MODEL_DISAGREEMENT','LESS_CRITICAL') THEN 1
    ELSE 0
END AS alert_state
FROM transaction_verdicts
ORDER BY rowid DESC
LIMIT 1;
```

Value mappings: 0 NO ACTIVE ALERT; 1 WATCH; 2 REVIEW; 3 CRITICAL DETECTION. Use green, warning colours and red in the same order.

©Joseph Aragbaiye Project, 2026



Reference: Actionable Alert Stat design.

Panel 5. Total Transactions

Visualization: Stat. Show only the large count and use compact number formatting.

```
SELECT COUNT(*) AS total_transactions
FROM transaction_verdicts;
```

Panel 6. Latest DB Sequence

Visualization: Stat. Show the newest sequence as a large integer.

```
SELECT MAX(sequence) AS latest_sequence
FROM transaction_verdicts;
```

If the current table does not contain sequence, use MAX(rowid) instead.

©Joseph Aragbaiye Project, 2026

6. Build the second status row exactly

Panel 7. Network Drift State

Visualization: Stat with full background.

```
SELECT CASE
    WHEN LOWER(COALESCE(network_drift,'')) LIKE '%stable%' THEN 0
    WHEN LOWER(COALESCE(network_drift,'')) LIKE '%drift%' THEN 1
    ELSE 2
END AS drift_state
FROM transaction_verdicts
ORDER BY rowid DESC
LIMIT 1;
```

Value mappings: 0 STABLE BASELINE; 1 DRIFT ACTIVE; 2 UNKNOWN.

Panel 8. Adaptation Mode

Visualization: Stat with full background. Show the latest stored controller state.

```
SELECT COALESCE(adaptation_mode,'UNKNOWN') AS adaptation_mode
FROM transaction_verdicts
ORDER BY rowid DESC
LIMIT 1;
```

Create text mappings for MORE_SENSITIVE, DRIFT_ADAPTATION, STRICT and STABLE so the background state changes with the stored value.

Panel 9. Latest Consensus Votes

Visualization: Bar gauge with three horizontal rows.

```
SELECT
    anomaly_votes AS Anomaly_Votes,
    normal_votes AS Normal_Votes,
    available_models AS Available_Models
FROM transaction_verdicts
ORDER BY rowid DESC
LIMIT 1;
```

Set maximum to 3 and keep the numeric values visible on the right.

Panel 10. Latest Individual Model Flags

Visualization: Bar gauge with HST, Online_IF and Batch_IF.

```
SELECT
    hst_flag AS HST,
    online_if_flag AS Online_IF,
    batch_if_flag AS Batch_IF
FROM transaction_verdicts
ORDER BY rowid DESC
LIMIT 1;
```

Set minimum 0 and maximum 1.

Panel 11. Latest Transaction — Database Truth

Visualization: Table across the full dashboard width.

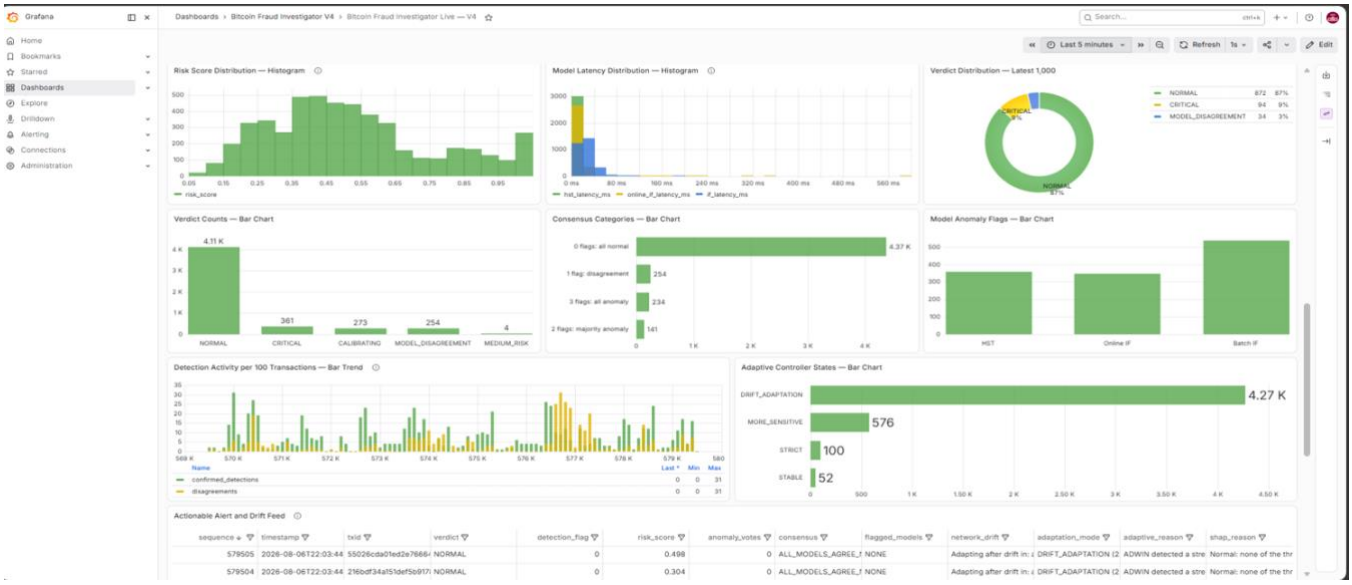
©Joseph Aragbaiye Project, 2026

```
SELECT
    sequence,
    timestamp,
    txid,
    verdict,
    detection_flag,
    ROUND(risk_score,3) AS risk_score,
    anomaly_votes,
    normal_votes,
    available_models,
    consensus,
    agreement_pattern,
    flagged_models,
    normal_models,
    network_drift,
    adaptation_mode
FROM transaction_verdicts
ORDER BY rowid DESC
LIMIT 1;
```

Keep horizontal scrolling rather than deleting evidence fields.

©Joseph Aragbaiye Project, 2026

7. Build the distribution and operational analytics section



Reference: exact analytics area.

Panel 12. Risk Score Distribution — Histogram

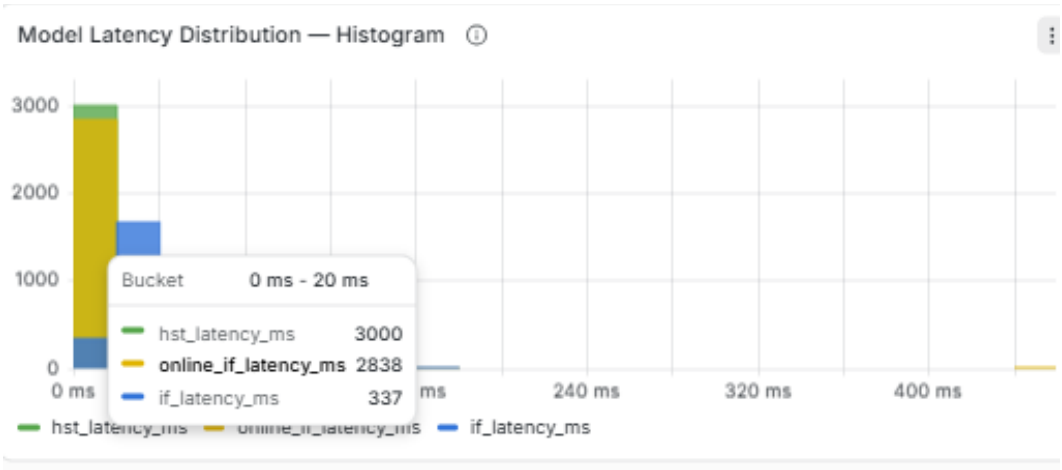
Visualization: Histogram.

```
SELECT risk_score
FROM transaction_verdicts
ORDER BY rowid DESC
LIMIT 5000;
```

Panel 13. Model Latency Distribution — Histogram

Visualization: Histogram with three fields, using milliseconds.

```
SELECT
    hst_ms AS hst_latency_ms,
    online_if_ms AS online_if_latency_ms,
    batch_if_ms AS if_latency_ms
FROM transaction_verdicts
ORDER BY rowid DESC
LIMIT 3000;
```



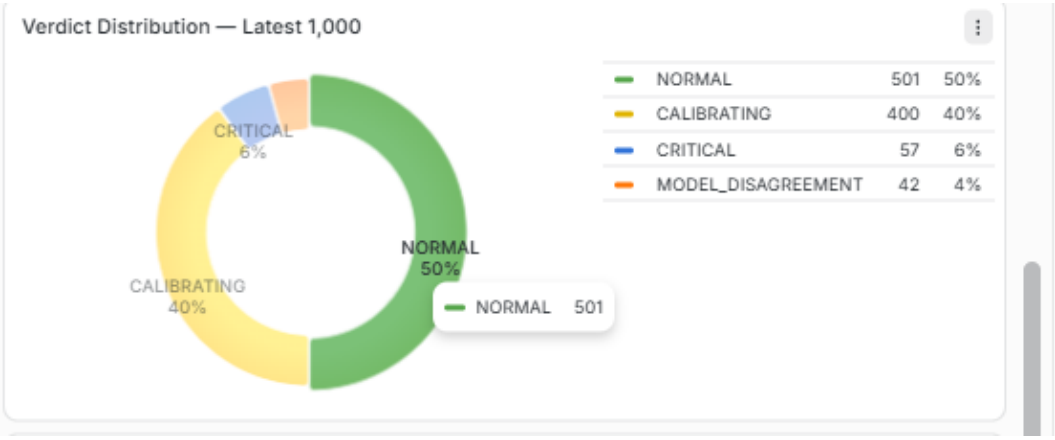
Reference: Model Latency Distribution histogram.

©Joseph Aragbaiye Project, 2026

Panel 14. Verdict Distribution — Latest 1,000

Visualization: Pie chart in Donut mode. Show labels, counts and percentages.

```
SELECT verdict, COUNT(*) AS count
FROM (
    SELECT verdict
    FROM transaction_verdicts
    ORDER BY rowid DESC
    LIMIT 1000
)
GROUP BY verdict
ORDER BY count DESC;
```



Reference: Verdict Distribution donut.

Panel 15. Verdict Counts — Bar Chart

Visualization: vertical Bar chart.

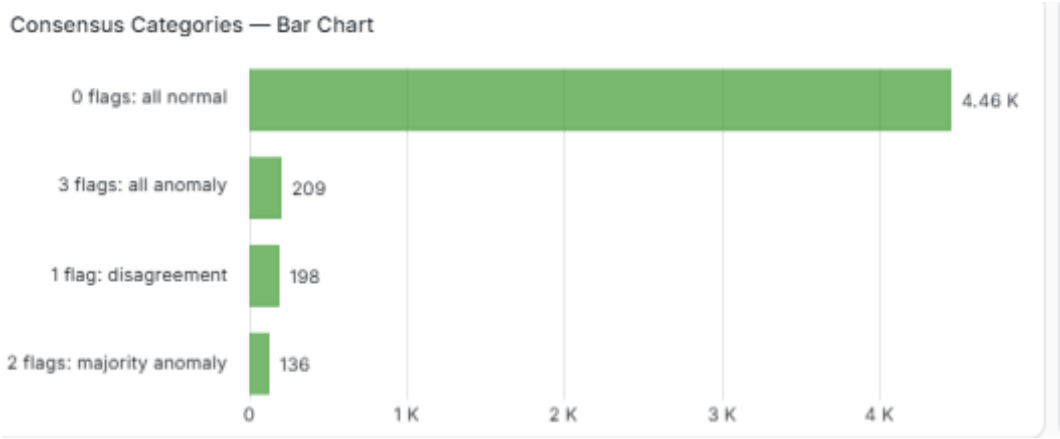
```
SELECT verdict, COUNT(*) AS count
FROM (
    SELECT verdict
    FROM transaction_verdicts
    ORDER BY rowid DESC
    LIMIT 5000
)
GROUP BY verdict
ORDER BY count DESC;
```

Panel 16. Consensus Categories — Bar Chart

Visualization: horizontal Bar chart.

©Joseph Aragbaiye Project, 2026

```
SELECT
    CASE
        WHEN anomaly_votes=0 THEN '0 flags: all normal'
        WHEN anomaly_votes=1 THEN '1 flag: disagreement'
        WHEN anomaly_votes=2 THEN '2 flags: majority anomaly'
        ELSE '3 flags: all anomaly'
    END AS consensus_category,
    COUNT(*) AS count
FROM (
    SELECT anomaly_votes
    FROM transaction_verdicts
    ORDER BY rowid DESC
    LIMIT 5000
)
GROUP BY consensus_category
ORDER BY count DESC;
```



Reference: Consensus Categories horizontal bar chart.

Panel 17. Model Anomaly Flags — Bar Chart

Visualization: three vertical bars.

```
WITH recent AS (
    SELECT hst_flag, online_if_flag, batch_if_flag
    FROM transaction_verdicts
    ORDER BY rowid DESC
    LIMIT 1000
)
SELECT 'HST' AS model, SUM(hst_flag) AS flags FROM recent
UNION ALL
SELECT 'Online IF', SUM(online_if_flag) FROM recent
UNION ALL
SELECT 'Batch IF', SUM(batch_if_flag) FROM recent;
```

Panel 18. Detection Activity per 100 Transactions — Bar Trend

Visualization: Bar chart with two series.

©Joseph Aragbaiye Project, 2026

```
WITH recent AS (
    SELECT sequence, detection_flag, verdict
    FROM transaction_verdicts
    ORDER BY rowid DESC
    LIMIT 10000
)
SELECT
    CAST(sequence/100 AS INTEGER)*100 AS sequence_bucket,
    SUM(CASE WHEN detection_flag=1 THEN 1 ELSE 0 END) AS confirmed_detections,
    SUM(CASE WHEN verdict='MODEL_DISAGREEMENT' THEN 1 ELSE 0 END) AS disagreements
FROM recent
GROUP BY sequence_bucket
ORDER BY sequence_bucket;
```

Panel 19. Adaptive Controller States — Bar Chart

Visualization: horizontal Bar chart.

```
SELECT adaptation_mode, COUNT(*) AS count
FROM (
    SELECT adaptation_mode
    FROM transaction_verdicts
    ORDER BY rowid DESC
    LIMIT 5000
)
GROUP BY adaptation_mode
ORDER BY count DESC;
```

Panel 20. Actionable Alert and Drift Feed

Visualization: Table.

```
SELECT
    sequence,
    timestamp,
    txid,
    verdict,
    detection_flag,
    ROUND(risk_score,3) AS risk_score,
    anomaly_votes,
    consensus,
    flagged_models,
    network_drift,
    adaptation_mode,
    adaptive_reason,
    shap_reason
FROM transaction_verdicts
WHERE detection_flag=1
   OR verdict='MODEL_DISAGREEMENT'
   OR LOWER(COALESCE(network_drift,'')) LIKE '%drift%'
ORDER BY rowid DESC
LIMIT 100;
```

©Joseph Aragbaiye Project, 2026

8. Build the relationship panels exactly



Reference: Risk vs Bitcoin Amount and transaction-complexity area.

Panel 21. Risk vs Bitcoin Amount — Scatter Plot

Visualization: XY chart / Scatter. X axis amount_btc; Y axis risk_score.

```
SELECT amount_btc, risk_score
FROM transaction_verdicts
WHERE amount_btc IS NOT NULL AND risk_score IS NOT NULL
ORDER BY rowid DESC
LIMIT 1000;
```

Panel 22. Risk vs Transaction Complexity — Scatter Plot

The reference capture visibly uses two wide horizontal bars even though the panel title says Scatter Plot. To reproduce that exact appearance, use a Bar gauge with the newest complexity and risk_score values.

```
SELECT complexity, risk_score
FROM transaction_verdicts
ORDER BY rowid DESC
LIMIT 1;
```

Show complexity and risk_score as two horizontal fields with values on the right.

©Joseph Aragbaiye Project, 2026

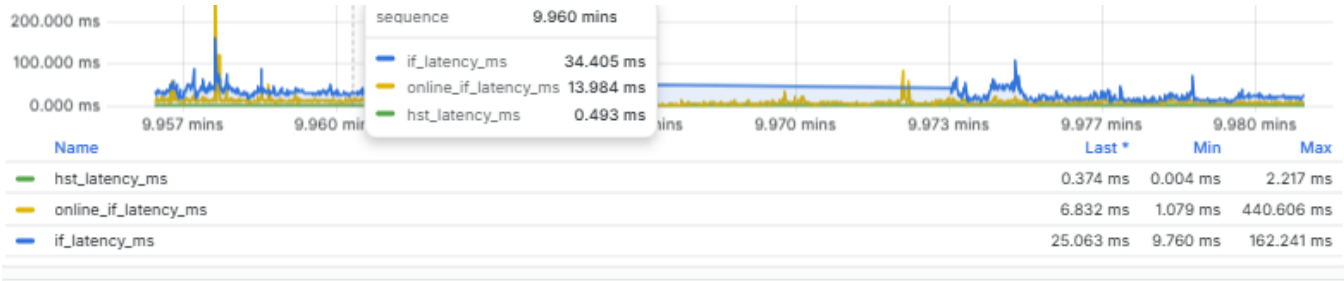
9. Build the supporting model trend panels

Panel 23. Model Latency Trend

Visualization: Time series with three model latency lines.

```
SELECT
    sequence AS time,
    hst_ms AS hst_latency_ms,
    online_if_ms AS online_if_latency_ms,
    batch_if_ms AS if_latency_ms
FROM transaction_verdicts
ORDER BY rowid DESC
LIMIT 1000;
```

If Grafana insists on a time-formatted field, use timestamp AS time and mark timestamp as the time column.



Reference: model latency trend.

Panel 24. Model Weights

Visualization: Time series with three weight series.

```
SELECT
    sequence AS time,
    hst_weight,
    online_if_weight,
    if_weight
FROM transaction_verdicts
ORDER BY rowid DESC
LIMIT 1000;
```



Reference: HST, Online IF and IF model weight display.

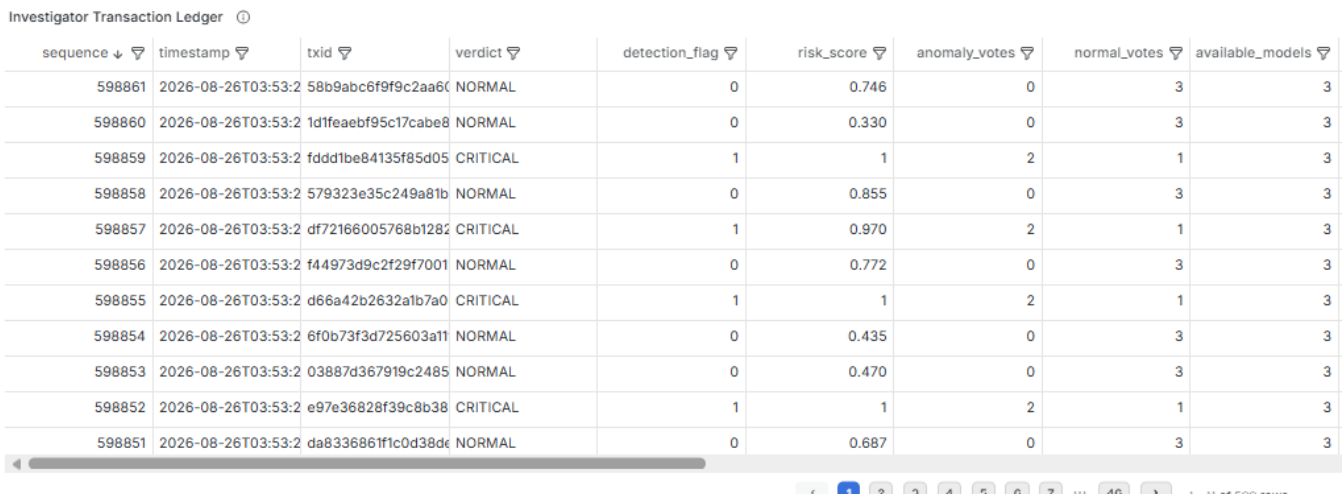
©Joseph Aragbaiye Project, 2026

10. Build the Investigator Transaction Ledger

Visualization: Table. Keep it wide and allow horizontal scrolling.

```
SELECT
    sequence,
    timestamp,
    txid,
    verdict,
    detection_flag,
    ROUND(risk_score,3) AS risk_score,
    anomaly_votes,
    normal_votes,
    available_models,
    hst_score,
    hst_percentile,
    hst_threshold,
    hst_flag,
    hst_health,
    online_if_score,
    online_if_percentile,
    online_if_flag,
    batch_if_score,
    batch_if_percentile,
    batch_if_flag,
    network_drift,
    adaptation_mode,
    hst_ms,
    online_if_ms,
    batch_if_ms,
    faster_model,
    latency_ratio,
    amount_btc
FROM transaction_verdicts
ORDER BY rowid DESC
LIMIT 500;
```

If any field name differs in the live schema, run PRAGMA table_info(transaction_verdicts) and substitute the real name. Preserve the evidence order rather than removing fields.



Investigator Transaction Ledger

sequence	timestamp	txid	verdict	detection_flag	risk_score	anomaly_votes	normal_votes	available_models
598861	2026-08-26T03:53:2	58b9abc6f9f9c2aa6(	NORMAL	0	0.746	0	3	3
598860	2026-08-26T03:53:2	1d1feaebf95c17cabe8	NORMAL	0	0.330	0	3	3
598859	2026-08-26T03:53:2	fddd1be84135f85d05	CRITICAL	1	1	2	1	3
598858	2026-08-26T03:53:2	579323e35c249a81b	NORMAL	0	0.855	0	3	3
598857	2026-08-26T03:53:2	df72166005768b1282	CRITICAL	1	0.970	2	1	3
598856	2026-08-26T03:53:2	f44973d9c2f29f7001	NORMAL	0	0.772	0	3	3
598855	2026-08-26T03:53:2	d66a42b2632a1b7a0	CRITICAL	1	1	2	1	3
598854	2026-08-26T03:53:2	6f0b73f3d725603a11	NORMAL	0	0.435	0	3	3
598853	2026-08-26T03:53:2	03887d367919c2485	NORMAL	0	0.470	0	3	3
598852	2026-08-26T03:53:2	e97e36828f39c8b38	CRITICAL	1	1	2	1	3
598851	2026-08-26T03:53:2	da8336861f1c0d38d(	NORMAL	0	0.687	0	3	3

Reference: Investigator Transaction Ledger - verdict and vote fields.

©Joseph Aragbaiye Project, 2026

action Ledger

hst_score	hst_percentile	hst_threshold	hst_flag	hst_health	online_if_score	nline_if_percentile
0.731	0.742	0.933	0	HEALTHY	0.270	0.732
0.494	0.446	0.933	0	HEALTHY	0.237	0.315
0.972	0.978	0.932	1	HEALTHY	0.354	0.924
0.876	0.860	0.932	0	HEALTHY	0.291	0.842
0.944	0.944	0.930	1	HEALTHY	0.304	0.871
0.784	0.788	0.930	0	HEALTHY	0.270	0.738
0.967	0.973	0.929	1	HEALTHY	0.343	0.919
0.498	0.481	0.929	0	HEALTHY	0.251	0.553
0.498	0.481	0.929	0	HEALTHY	0.251	0.555
0.965	0.968	0.929	1	HEALTHY	0.362	0.928
0.773	0.778	0.929	0	HEALTHY	0.247	0.462

Reference: Investigator Transaction Ledger - HST and Online IF fields.

network_drift	adaptation_mode	hst_ms	online_if_ms	batch_if_ms	faster_model	latency_ratio	amount_btc
Adapting after drift in	DRIFT_ADAPTATION	0.374	6.83	25.1	HST	66.9	0.00260
Adapting after drift in	DRIFT_ADAPTATION	0.367	9.06	22.2	HST	60.5	0.00225
Adapting after drift in	DRIFT_ADAPTATION	0.211	8.79	21.1	HST	100	214
Adapting after drift in	DRIFT_ADAPTATION	0.228	7.41	17.1	HST	75.3	0.00505
Adapting after drift in	DRIFT_ADAPTATION	0.179	6.57	15.0	HST	83.8	0.488
Adapting after drift in	DRIFT_ADAPTATION	0.296	6.78	21.3	HST	72.0	0.0161
Adapting after drift in	DRIFT_ADAPTATION	0.144	8.78	21.1	HST	147	0.797
Adapting after drift in	DRIFT_ADAPTATION	0.299	8.52	20.7	HST	69.1	0.00822
Adapting after drift in	DRIFT_ADAPTATION	0.295	11.2	22.8	HST	77.4	0.0309
Adapting after drift in	DRIFT_ADAPTATION	0.201	9.43	19.1	HST	95.2	1.25
Adapting after drift in	DRIFT_ADAPTATION	0.333	7.54	24.0	HST	71.9	0.00424

Reference: Investigator Transaction Ledger - drift, adaptation, latency and amount fields.

©Joseph Aragbaiye Project, 2026

11. Arrange the new dashboard to match the reference

- Row 1: Current Fraud Verdict | Latest Risk Score | Current Consensus | Actionable Alert | Total Transactions | Latest DB Sequence.
- Row 2: Network Drift State | Adaptation Mode | Latest Consensus Votes | Latest Individual Model Flags.
- Row 3: Latest Transaction — Database Truth across the full width.
- Analytics row: Risk Score Distribution | Model Latency Distribution | Verdict Distribution — Latest 1,000.
- Next row: Verdict Counts | Consensus Categories | Model Anomaly Flags.
- Next row: Detection Activity per 100 Transactions | Adaptive Controller States.
- Next row: Actionable Alert and Drift Feed across the width.
- Relationship row: Risk vs Bitcoin Amount | Risk vs Transaction Complexity.
- Supporting trends: Model Latency Trend | Model Weights.
- Final evidence area: Investigator Transaction Ledger across the full width.

Use the supplied screenshots as the visual truth while dragging and resizing. The new dashboard should look structurally the same, while all values are live.

12. Save and make the fresh dashboard live

1. Choose Save dashboard and keep the name Bitcoin Fraud Investigator Live — V4.
2. Set refresh to 1s to match the reference. If the machine becomes unnecessarily busy, change only the refresh rate to 5s; do not redesign the panels.
3. Run the ML pipeline and watch Latest DB Sequence and Total Transactions increase.
4. Check that Current Fraud Verdict, Latest Risk Score, Current Consensus and Actionable Alert follow the newest SQLite row.

13. Prove live ML → SQLite → Grafana operation

```
watch -n 2 'sqlite3 /home/admp/bitcoin-fraud-data/bitcoin_analytics_live.db "SELECT
sequence, timestamp, verdict, ROUND(risk_score,4), anomaly_votes, detection_flag FROM
transaction_verdicts ORDER BY rowid DESC LIMIT 5;"'
```

The newest terminal row should match the newest values shown by Grafana. If SQLite changes but Grafana does not, inspect the datasource or panel query. If SQLite does not change, the fault is before Grafana.

14. Final design rule

Preserve the exact dashboard design shown in the screenshots, but never hard-code the screenshot values. Every verdict, risk score, model vote, drift state, latency, transaction count and investigator row must come from transaction_verdicts. An anomaly or CRITICAL state is evidence for review, not proof of fraud.

©Joseph Aragbaiye Project, 2026